

重庆大学本科学生毕业论文（设计）

康养中心信息化管理系统的设计与实现



学 生：刘铠

学 号：20215295

指导教师：石锐

专 业：计算机科学与技术

重庆大学计算机学院

2025 年 5 月

Undergraduate Thesis (Design) of Chongqing University

**Design and Implementation of an
Information Management System for Health
and Wellness Centers**



By

LIU Kai

Supervised by

Prof. SHI Rui

Computer Science and Technology

College of Computer Science

Chongqing University

May, 2025

摘 要

随着我国人口老龄化程度加深，65 岁以上老年人口已占全国总人口的 15.4%（国家统计局，2023），预计到 2070 年将达到 41.3%。传统养老服务模式如家庭养老、机构养老和社区养老均面临效率低、数据分散、服务单一、人力资源短缺、资源利用率低、个性化服务不足等问题。老年群体呈现慢性病高发、术后康复周期延长等复合健康需求特征，约 86%以上的老年人患有至少一种慢性病，更有 65%以上的老年人存在多病共存现象，而传统康复服务存在资源分散、连续性不足等痛点。为此，国家《智慧健康养老产业发展行动计划（2021-2025 年）》明确提出构建覆盖“预防-医疗-康复-护理-安宁疗护”全周期的智慧健康养老服务系统，要求通过物联网、人工智能等技术实现健康数据动态监测与康复资源精准调度。本课题旨在为老年人提供全方位的健康管理服务，包括健康信息检测、疾病预防、心理健康等，拟设计并实现一套基于 B/S 架构的康养机构信息管理系统，以客户信息管理、膳食管理及推荐、健康数据动态分析为核心，通过大模型、代谢综合征（MetS）预测模型的接入，力图解决以下问题：数据孤岛——整合健康档案、护理记录等多源数据，实现标准化管理；服务精准化——结合个性化膳食推荐与健康风险预警，提升客户满意度；智能化程度不足——利用大语言模型（LLM），提供智能化建议。

本系统基于 Web 架构采用前后端分离模式开发，前端运用 Vue3 框架构建动态交互界面，后端通过 Django 框架实现业务逻辑处理，数据库系统采用 MySQL 进行结构化数据存储，结合 Redis 实现高频数据的缓存优化，构建了一个高效协同的管理平台。系统针对康养机构的业务特征，重点设计了七大核心功能模块：客户信息管理模块、护理计划管理模块、健康档案管理模块、膳食计划管理模块、排班管理模块、预约服务管理模块、通知提醒模块。该系统创新性地将健康评估数据与运营服务深度耦合，通过数据可视化分析辅助决策，优化了传统康养机构多部门协同效率。

关键词：康养中心；信息管理系统；老龄化；大语言模型；代谢综合征预测

ABSTRACT

With the deepening population aging trend in China, the proportion of elderly people aged 65 and above has reached 15.4% of the total population (National Bureau of Statistics, 2023), projected to increase to 41.3% by 2070. Traditional elderly care models such as family-based, institutional, and community-based care face challenges including low efficiency, fragmented data, limited services, human resource shortages, low resource utilization, and insufficient personalized services. The elderly population exhibits complex health needs characterized by a high prevalence of chronic diseases and prolonged postoperative rehabilitation periods, with approximately 86% of seniors suffering from at least one chronic disease and over 65% experiencing multimorbidity, while traditional rehabilitation services suffer from resource fragmentation and service discontinuity. The national Smart Health and Elderly Care Industry Development Action Plan (2021-2025) explicitly proposes establishing a comprehensive smart health management system covering the full cycle of "prevention-medical treatment-rehabilitation-nursing-hospice care", mandating the implementation of dynamic health data monitoring and precision scheduling of rehabilitation resources through IoT and AI technologies. This study aims to provide comprehensive health management services for the elderly, including health information monitoring, disease prevention, and psychological health support, by designing and implementing a B/S architecture-based information management system for health and wellness institutions. With a focus on client information management, dietary management and recommendations, and dynamic health data analysis, the system incorporates large language models and metabolic syndrome (MetS) prediction models to address three key challenges: data silos - integrating multi-source data such as health records and care documentation for standardized management; service precision - combining personalized dietary recommendations with health risk warnings to improve client satisfaction; and insufficient intelligence - utilizing large-scale pre-trained language models (LLMs) to provide intelligent recommendations.

The system employs a Web-based architecture with front-end and back-end separation, utilizing Vue3 framework for dynamic interfaces and Django framework for backend services. MySQL handles structured data storage complemented by Redis for

high-frequency data caching, collectively forming an efficient collaborative platform. Specifically designed for rehabilitation institutions, the system features seven core modules: Client Information Management, Care Plan Management, Health Records Management, Dietary Planning, Staff Scheduling, Service Reservation, and Notification Alerts. It innovatively achieves deep coupling between health assessment data and operational services, enhancing multi-department coordination efficiency through data visualization analytics to support decision-making.

Key words: Health and Wellness Centers; Information Management System; Aging;
Large Language Models (LLMs); Metabolic Syndrome (MetS)
Prediction

目 录

摘要	I
ABSTRACT	II
1 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	1
1.2.1 国内研究现状	1
1.2.2 国外研究现状	2
1.3 研究内容	2
2 需求分析	4
2.1 客户信息管理模块	4
2.2 护理计划管理模块	5
2.3 健康档案管理模块	7
2.4 膳食计划管理模块	8
2.5 排班管理模块	10
2.6 预约服务管理模块	12
2.7 通知提醒模块	14
2.8 其它功能说明	15
2.8.1 登录注册模块	15
2.8.2 用户管理模块	16
2.8.3 个人信息管理模块	17
2.8.4 报表展示模块	18
2.9 非功能需求	19
2.9.1 性能需求分析	19
2.9.2 安全性需求分析	19
2.10 本章小结	20
3 系统技术架构	21
3.1 平台设计	21
3.2 关键技术概述	22
3.2.1 前端技术栈: Vue 3	22
3.2.2 后端技术栈: Django + Django REST Framework(DRF)	22

3.2.3 数据库设计与优化: Mysql	23
3.2.4 缓存系统设计: Redis	23
3.2.5 系统安全	24
3.3 本章小结	24
4 系统数据库设计与实现	25
4.1 数据库实体关系设计	25
4.2 数据库物理结构设计	28
4.3 本章小结	36
5 系统功能模块设计与实现	38
5.1 接口鉴权功能及相关安全特性的实现	38
5.1.1 基于会话的身份验证机制	38
5.1.2 基于角色的访问控制 (RBAC)	39
5.1.3 中间件层保护	40
5.1.4 密码安全管理	40
5.2 登录、用户管理和个人资料及通知模块	40
5.3 首页报表展示模块	43
5.4 客户档案管理模块	44
5.5 健康档案管理模块	45
5.6 护理计划管理模块	47
5.7 预约服务管理模块	49
5.8 排班管理模块	52
5.9 膳食计划管理模块	54
5.10 本章小结	58
6 智能服务模块研究与实现	59
6.1 大模型的应用	59
6.1.1 个性化饮食推荐	59
6.1.2 健康指标评估与建议	60
6.2 代谢综合征预测	61
6.2.1 方法与材料	61
6.2.2 实验结果	64
6.3 本章小结	68
7 系统测试	69
7.1 测试环境	69
7.2 功能测试	70

7.3 性能测试73

7.3.1 用户数与并发量估算73

7.3.2 负载测试73

7.3.3 压力测试76

7.4 本章小结79

8 总结与展望81

8.1 工作总结81

8.2 不足与展望82

参考文献83

附录 A：后端关键代码86

附录 B：前端关键代码98

附录 C：代谢综合征预测代码109

致谢116

原创性声明和使用授权书

1 绪论

1.1 研究背景与意义

随着我国人口老龄化程度加深，65 岁以上老年人口已占全国总人口的 15.4%（国家统计局，2023），预计 2070 年将达到 41.3%^[1]。传统养老模式如家庭养老、机构养老、社区养老等存在着服务效率低、数据信息分散不互通、服务质量与资源单一、缺少人力投入、利用效率低、个性化程度不足等诸多问题^[2]。而老年人群体具有高慢性病患率、术后恢复期长等复合需求特征，如何高效整合养老和康复服务资源满足老年人的复合需求已成为当下的热点公众议题。据调查显示约有 86% 以上的老人患有一种或多种慢性病，其中有 65% 以上的老人存在多病共存的现象，而传统康养服务又存在着碎片化、不连续等痛点^[3]。为此工业和信息化部、民政部、国家卫生健康三方共同署名发布了《智慧健康养老产业发展行动计划（2021-2025 年）》，明确指出要构建“预防-医疗-康复-护理-安宁疗护”全周期一体化的智慧养老服务体系，提出要充分利用物联网、人工智能等相关信息技术实现动态健康监测和精准调配各类康复资源的要求^[4]。

本课题旨在为老年人提供全方位的服务管理包括健康信息检测、疾病预防等^[5]，希望能设计开发出一套 B/S 架构的康养机构信息管理系统，以客户信息管理、膳食管理及推荐、健康数据动态分析为重点，解决三个问题：

1. 数据孤岛：整合健康档案、护理记录等多源数据，实现标准化管理。
2. 服务精准化：个性化膳食推荐与健康预警相结合。
3. 智能化程度低：利用大语言模型（LLM）提供智能建议。

1.2 国内外研究现状

1.2.1 国内研究现状

国内关于养老方面系统的研究表现在医养结合、智能化管理与数据可视化等方面。医养结合方面，王等（2022）提出的 H 养老院信息系统整合了在线问诊、个性化点餐、娱乐场所预约等功能，探索了医疗与养护的创新融合^[6]。智能化管理领域，李（2019）开发的老年人康养庄园智慧管理系统使用人工智能和物联网技术，利用可穿戴设备实现了生命体征监测和预警功能，并辅以智能门禁、服务呼叫等功能提升了机构服务效率^[7]。数据可视化方面，张与梁（2024）基于 ECharts 构建的康养数据可视化系统实现了医疗指标实时监测与趋势分析，能够直观地展示复杂疗养数据并提供决策支持^[8]。此外面向特定人群的信息管理系统也取得了进展，如罗（2021）针对老年痴呆群体设计的智慧康养平台构建了包含一般病史、ABC

症候群等十余项临床数据的数据库，并利用统计法和马尔科夫模型分析了认知能力的影响因素^[9]。而吴（2019）构建的养老院信息系统在实现健康档案、膳食与费用管理等功能的基础上结合了物联网技术，给老年人配备的腕表能够准确定位在院内及院外的位置，并提供一键呼叫功能^[10]。

1.2.2 国外研究现状

国际对智慧养老开展的研究较早，重点多放在智能健康管理、物联网医疗以及数字健康平台等方面^[11]。智能健康管理领域，马来西亚有团队设计出一款老年照护 App，这款 App 不但实现了对老年人远程的体征监测，而且还能够对医疗预约以及药物情况进行管理（Dahari 等，2022）^[12]。另外，在泰国老龄社区展开试点的一款智能健康管理软件，它通过对历史病历加以管理，同时提供公共卫生服务中心以及医院的相关信息，并且还播放适合老年人观看的运动视频，以此来提升老年人自身对健康进行管理的能力（Ladkoom 等，2024）^[13]。在物联网医疗方面，新型的可穿戴设备具备多种功能，像是能够捕捉血压、血氧等人体指标，还能对运动状态进行监控，也可以对慢性病进行观测等（Mahajan 等，2024）^[14]。TrueLoo 智能马桶圈能够自动记录排尿事件，对粪便和尿液的特征进行监测并予以报告，而且还能联网把相关数据发送给护理人员，这样一来就大大减轻了患者以及护理人员的负担，使得护理质量也得到了提高（Glenn 等，2024）^[15]。在数字健康平台的研究当中，希腊所开发的护理平台可以利用人工智能去分析老年人的行为模式，进而定制出适合的护理计划（Kouroubali 等，2022）^[16]。日本的 Urban Renaissance 机构在其住房社区里建立了智能医疗系统，借助该系统能够让行动不便的老龄住民在家就可以获得护理服务以及医疗支持（ZHANG & HU，2022）^[17]。在实际推广的过程中，这些智能化的系统或者产品，它们依然面临着不少挑战，比如个人信息安全方面的顾虑、用户接受程度的问题以及技术整合等方面的难题（Maeder & Morgan，2020）^[18]。

1.3 研究内容

本文基于老龄化社会背景下康养服务的现实需求，参考现有国内外相关系统，结合医学和信息技术领域知识，构建了一个面向客户和服务提供人员的多视角康养中心信息管理系统。系统选用 B/S 架构，使用前后端分离模式进行开发，并整合了 LLM 与代谢综合征（MetS）预测模型，具体研究内容如下：

（1）需求分析。考虑到老年群体健康管理有着自身的特殊之处，所以要对康养机构在诸如客户管理、健康监测以及膳食管理等方面所涉及的业务流程展开分析，并且要细致梳理系统当中的各类用户，像是管理员、护理人员（也就是员工）、老年人（也就是客户）等，进而明确他们各自的功能需求以及权限划分情

况，最终确定系统的整体目标以及关键性能指标到底是怎样的。

（2）系统技术架构。分析并且挑选契合康养信息系统的关键技术栈，以此来搭建前后端分离的系统架构模式。凭借着对技术栈展开合理的配置，促使系统各个组件之间能够达成高效的协同，让数据得以顺畅流转，进而给后续功能模块的开发筑牢相应的技术根基。

（3）系统数据库设计与实现。依据业务方面的具体需求来规划设计数据模型，进而构建起涵盖用户信息、健康档案、膳食数据以及排班预约等诸多数据实体，同时明确它们相互之间关系的数据库结构。在存储结构化信息时，主要选用 MySQL 数据库。并且为减轻数据库所承受的负载，会结合 Redis 缓存技术对常用视图结果予以缓存。

（4）系统功能模块设计与实现。主要围绕客户信息管理、健康数据分析以及膳食管理推荐这三个核心要点，从而达成七大功能模块的实现。其一，客户信息模块能够实现康养中心客户基础信息的录入、查询、统计与分析。其二，护理计划管理模块帮助医护人员制定并调整个性化护理方案，也能对执行情况予以跟踪。其三，健康档案管理模块会对客户的健康数据加以整合。其四，膳食计划管理模块会把客户的健康数据和已有的膳食记录相结合，以此来给出个性化的膳食推荐内容。其五，排班管理模块可帮助管理员进行排班，实现人力资源的合理配置。其六，预约服务管理模块能够提供便捷的服务预约功能，同时还可对服务进度进行跟踪。其七，提醒模块会自动推送与健康相关的提醒内容以及活动通知等信息。

（5）智能服务模块研究与实现。智能服务模块方面的研究与实现工作主要聚焦于两点。其一，对大语言模型在康养管理领域的应用展开探索，力求达成诸如实现智能建议这类功能的目标。其二，针对代谢综合征预测模型于健康评估以及风险预警当中的应用加以研究，借助机器学习算法来细致分析老年人的健康数据，进而给出具有个性化特点的干预建议。

（6）系统测试。根据应用的功能模块设计，制定测试方案，对系统各模块进行测试验证其是否能够满足需求，验证系统是否满足关键性能指标。

2 需求分析

本章将详细阐述康养中心信息管理系统的需求分析过程，明确系统旨在解决的核心问题，并针对管理员、员工和客户三类不同角色，深入剖析他们在客户信息管理、护理计划、健康档案、膳食计划、排班、预约服务及通知提醒等七大核心功能模块以及登录注册、用户管理、个人信息管理等辅助模块中的具体功能需求与权限划分，同时定义系统的非功能性需求，为后续的系统设计与实现奠定基础。

2.1 客户信息管理模块

客户信息管理模块主要功能点为查询和修改客户档案，三种角色的用户对都能访问该模块，不同角色的用户权限不同，用例图如图 2.1 所示。

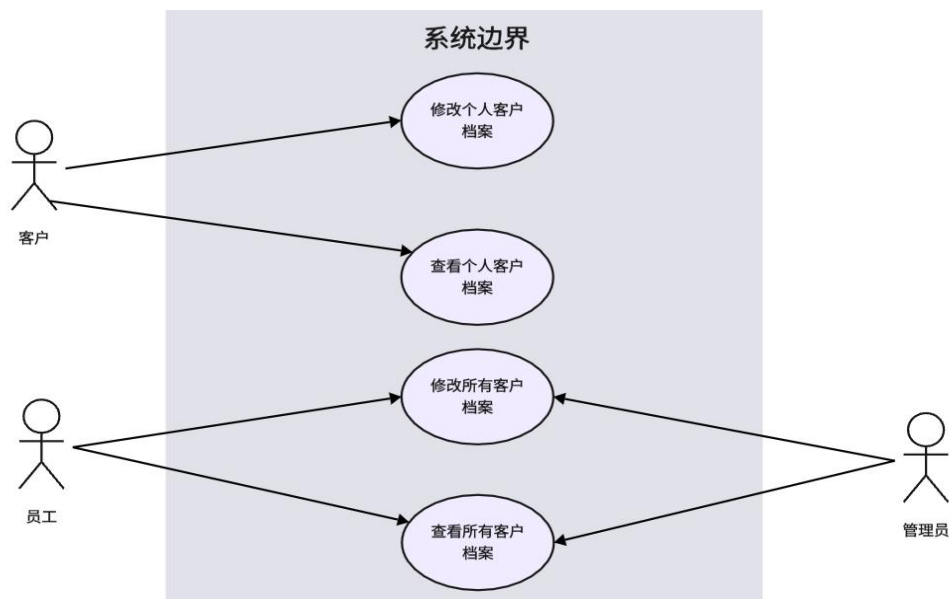


图 2.1 客户信息管理模块用例图

在该模块中，管理员和员工两种角色的权限一致，这里我们将这两种角色合并分析具体功能，给出表 2.1 和表 2.2。

表 2.1 客户信息管理模块客户功能说明表

客户功能	功能说明
查看个人客户档案	客户仅能够查看自己的客户档案，内容包括姓名、出生日期、护理需求等。

续表 2.1 客户信息管理模块客户功能说明表

客户功能	功能说明
修改个人客户档案	客户仅能够修改自己的客户档案，能修改除护理等级外的其它所有内容。

表 2.2 客户信息管理模块管理员/员工功能说明表

管理员/员工功能	功能说明
查看所有客户档案	管理员和员工能够查看所有客户的客户档案。
修改所有客户档案	管理员和员工能够修改所有客户的客户档案，能修改客户档案中的所有字段。

客户档案所存储的内容中，除了用户的个人信息如姓名、性别、出生日期等常见内容和护理等级、护理需求等康养语境下的需求内容外，还需要包含客户的忌口和爱吃食材用于后续的膳食计划管理模块中的膳食推荐功能，以及婚姻、收入范围两个字段用于后续的 MetS 预测模型的数据输入。

2.2 护理计划管理模块

护理计划的主要功能点为对护理计划的增删改查，对护理计划绑定的服务预约和计划目标的增删改查，查询护理计划报告。三种角色都能访问该模块，不同角色权限不同，用例图如图 2.2 所示。

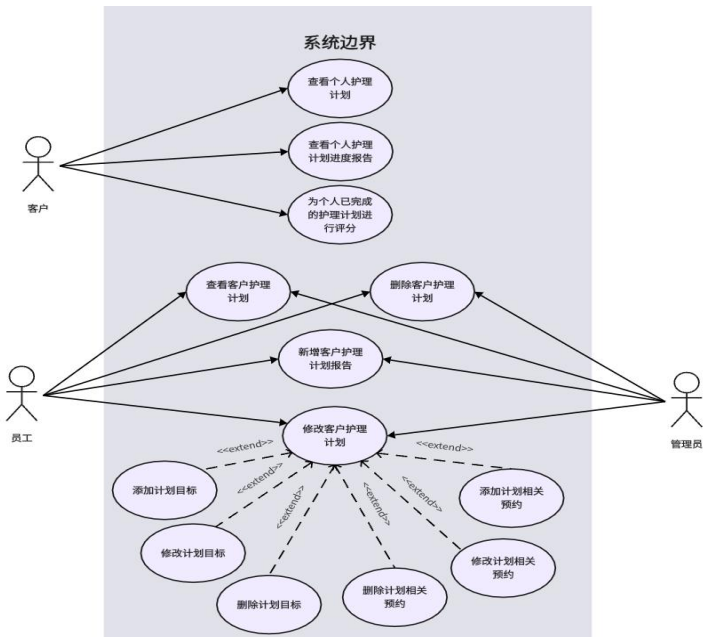


图 2.2 护理计划管理模块用例图

在该模块中，管理员和员工权限一致，这里合并讨论其功能并给出表 2.3 和表 2.4。

表 2.3 护理计划管理模块客户功能说明表

客户功能	功能说明
查看个人护理计划	客户只能查看本人的护理计划，护理计划内容包括护理计划基础字段如：开始日期，结束日期，计划类型等，和护理计划绑定内容如：预期达成目标，服务预约。
查看个人护理计划进度报告	客户只能查看本人的护理计划进度报告，护理计划报告除了护理计划基础字段外，还会展示护理计划中的不同状态的服务预约数量和不同状态的计划目标数量。
为个人已完成的护理计划评分	客户能为本人的计划状态为已结束的护理计划进行评分，分数区间为 1~10 的整数部分。

表 2.4 护理计划管理模块管理员/员工功能说明表

管理员/员工功能	功能说明
查看客户护理计划	管理员和员工能够查看所有客户的护理计划。
删除客户护理计划	管理员和员工能够删除所有客户的护理计划。
新增客户护理计划	管理员和员工能够新增所有客户的护理计划，新增时只需填写护理计划的基础字段。
修改客户护理计划	管理员和员工能够修改所有客户的护理计划，修改时除了能够修改护理计划的基础字段外，还能够修改关联的计划和预约。
添加计划目标	管理员和员工能够为某一护理计划添加计划目标，计划目标字段包含计划描述和计划状态。
修改计划目标	管理员和员工能够修改某一护理计划的计划目标。
删除计划目标	管理员和员工能够删除某一护理计划的计划目标。
添加计划相关预约	管理员和员工能够为某一护理计划添加服务预约。服务预约详情见 2.6，这里是在基础预约上添加了对护理计划的关联。
修改计划相关预约	管理员和员工能够修改某一护理计划的服务预约。
删除计划相关预约	管理员和员工能够删除某一护理计划的服务预约。

对于该模块客户拥有查看自己的护理计划和护理计划报告的权限以及能够为自己的护理计划评分。而管理员和员工拥有除了评分外的所有权限。

护理计划中应包含的字段有计划类型，状态，开始日期，结束日期，计划责任人等基础内容，除此之外还需要包含计划的目标和关联的服务预约。计划目标需要能够设置多个，服务预约也能设置多个。

2.3 健康档案管理模块

健康档案管理模块的主要功能点是对健康档案的增删改查，还包括根据健康档案中的各项指标给出患 MetS 概率和智能评估结果。三种角色都能访问该模块，不同角色权限不同，用例图见图 2.3。

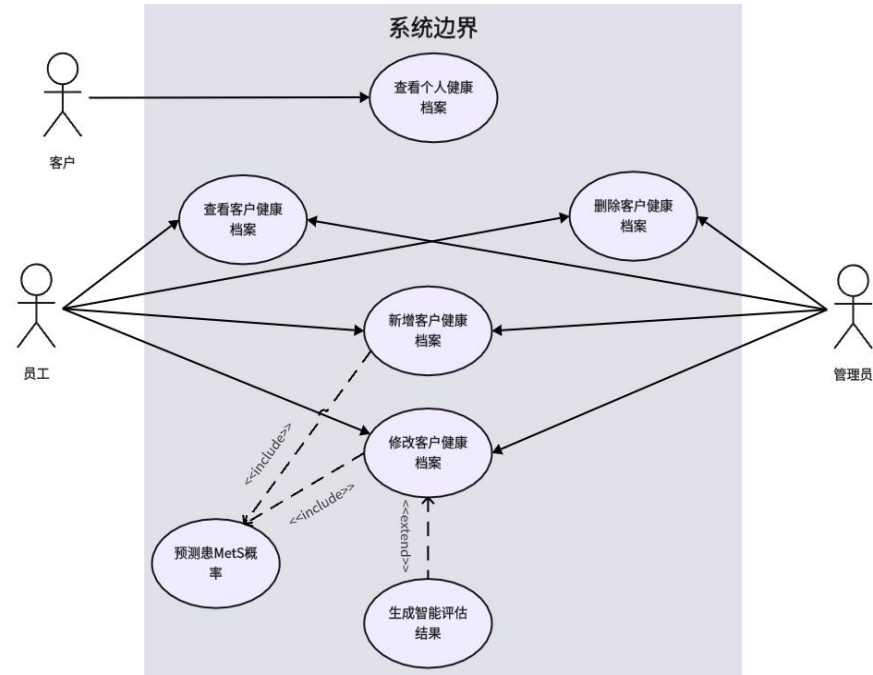


图 2.3 健康档案管理模块用例图

在该模块中，管理员和员工权限一致，这里合并讨论其功能，并给出表 2.5 和表 2.6。

表 2.5 健康档案管理模块客户功能说明表

客户功能	功能说明
查看个人健康档案	客户仅能够查看个人的健康档案，内容包括记录时间、记录员工、评估结果、身体各项数据等健康档案基本字段，除此之外还包括智能评估结果和预测患 MetS 概率。

表 2.6 健康档案管理模块管理员/员工功能说明表

管理员/员工功能	功能说明
查看客户健康档案	管理员和员工能够查看所有客户的健康档案。
删除客户健康档案	管理员和员工能够删除所有客户的健康档案。
新增客户健康档案	管理员和员工能够为所有客户新增健康档案，能填充的内容为健康档案基本字段。
修改客户健康档案	管理员和员工能够修改所有客户的健康档案，能修改的内容为健康档案基本字段，并可以生成智能评估结果。
预测患 MetS 概率	在新增或修改客户健康档案时，后台会凭借输入的身体指标数据预测客户患 MetS 概率并展示出来。
生成智能评估结果	在修改客户健康档案时，可以凭身体指标数据和预测患 MetS 概率生成智能评估结果。

健康档案应包含的字段有记录时间、记录员工、评估结果、身体各项指标等基础内容，除此之外还需要有预测患 MetS 概率以及智能评估结果。这里预测患 MetS 概率需要根据身体各项指标进行预测。管理员和员工能够选择生成智能评估结果，会根据身体各项指标以及预测患 MetS 概率生成并展示在具体的健康档案详情中。

2.4 膳食计划管理模块

膳食计划管理模块主要功能点是对膳食计划的增删改查，以及食材库和食谱库的增删改查。三种角色都能访问该模块，不同角色权限不同，用例图分图 2.4 和图 2.5 给出。

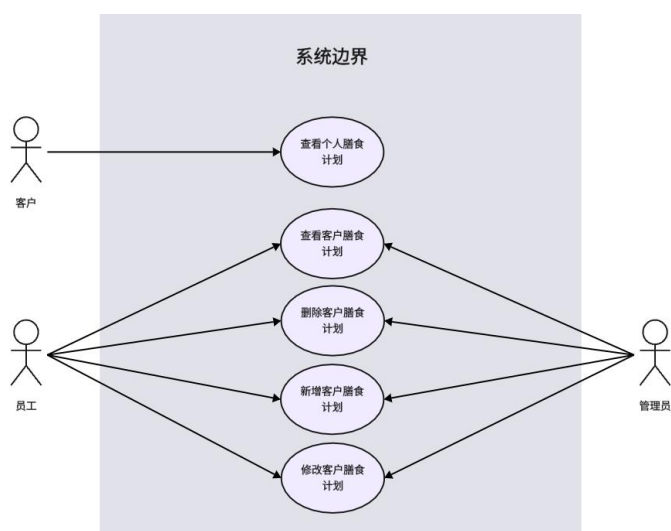


图 2.4 膳食计划管理模块（膳食计划部分）用例图

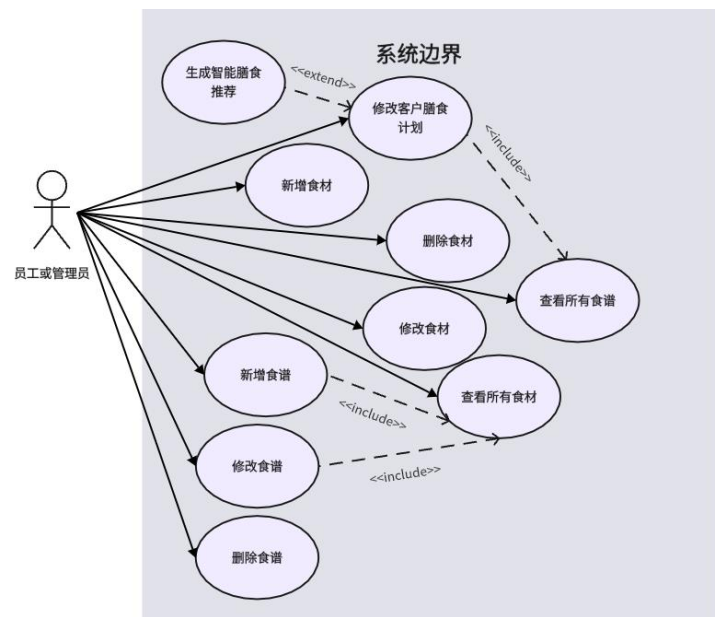


图 2.5 膳食计划管理模块（食材库和食谱库部分）用例图

在该模块中管理员和员工权限一致，合并讨论其功能，并给出表 2.7 和表 2.8。

表 2.7 膳食计划管理模块客户功能说明表

客户功能	功能说明
查看个人膳食计划	客户仅能够查看个人的膳食计划，内容包括计划制定日期、制定人、营养需求、三餐安排、实际营养摄入等膳食计划基本字段，还包括智能膳食推荐。

表 2.8 膳食计划管理模块管理员/员工功能说明表

管理员/员工功能	功能说明
查看客户膳食计划	管理员和员工能够查看所有客户的膳食计划。
删除客户膳食计划	管理员和员工能够删除所有客户的膳食计划。
新增客户膳食计划	管理员和员工能够为所有客户新建膳食计划，新建时仅填充制定人、制定日期、营养需求字段。
修改客户膳食计划	管理员和员工能够修改所有客户的膳食计划，在修改时除可以修改制定人、制定日期、营养需求等基础字段外，还可以从食谱库中选取具体的菜品填入三餐安排中，并可以选择生成智能膳食推荐。当三餐安排中被填入具体的菜品后，后台会计算出实际营养摄入。

续表 2.8 膳食计划管理模块管理员/员工功能说明表

管理员/员工功能	功能说明
查看所有食材	管理员和员工可以查看所有食材，食材包含内容有单位数量、描述、单位数量下的营养成分。
新增食材	管理员和员工可以新增食材。
修改食材	管理员和员工可以修改食材。
删除食材	管理员和员工可以删除食材。
查看所有食谱	管理员和员工可以查看所有食谱，食谱包含内容有菜品名称、食材清单、营养成分。
新增食谱	管理员和员工可以新增食谱，新增时需要填写菜品名称，然后从食材库中选取具体重量的食材填入食材清单中，后台会根据食材清单计算出该菜品的营养成分。
修改食谱	管理员和员工可以修改食谱，与新增食谱逻辑类似。
删除食谱	管理员和员工可以删除食谱。

膳食计划管理模块具体可以分为三个子模块：食材库模块；食谱库模块；膳食计划模块。其中客户只能访问膳食计划模块查看自己的膳食计划，而管理员和员工拥有所有权限。

食材库中的食材包含的字段有食材名，食材描述，营养成分。这里的营养成分为了简化处理，只包含了热量、脂肪、蛋白质、碳水化合物四种成分。后续食谱库、膳食计划中的营养部分也只包含了这四种成分。

食谱库中的食谱包含的字段有菜品名，食材清单，营养成分。这里的食材清单填入的食材为食材库中已有食材，而营养成分是根据食材库中食材的基本信息及填写的食材清单计算得来。

膳食计划包含的字段有制定人、制定日期、客户名、客户忌口和爱吃食材、营养需求、三餐安排等。这里三餐安排分为早餐、午餐、晚餐，填入时会从食谱库中选取具体的菜品填入具体某餐，一餐可以安排多个菜品。当三餐安排被填充后，会根据食谱库中的信息和具体填入的菜品计算出实际的营养摄入。管理员和员工可以选择生成智能膳食建议，会根据客户的忌口和爱吃食材、客户的一些基本信息和客户前三天的膳食计划结合食谱库生成智能膳食建议并展示在具体膳食计划详情中。

2.5 排班管理模块

排班管理模块的主要功能点是对排班记录的增删改查，以及排班模板的增删改

查。该模块只有员工和管理员能够访问，两种角色权限不同，用例图见图 2.6。

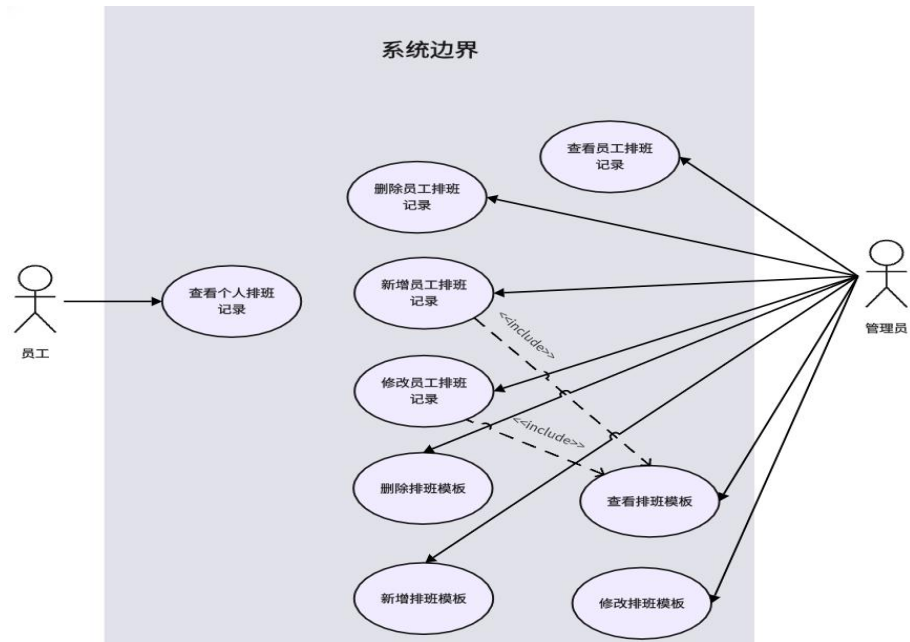


图 2.6 排班管理模块用例图

该模块中员工和管理员权限不同，分别讨论其功能，给出表 2.9 和表 2.10。

表 2.9 排班管理模块员工功能说明表	
员工功能	功能说明
查看个人排班记录	员工可以查看个人的排班记录，排班记录内容包括排班日期，排班模板及模板详情。

表 2.10 排班管理模块管理员功能说明表	
管理员功能	功能说明
查看员工排班记录	管理员可以查看所有员工的排班记录。
删除员工排班记录	管理员可以删除所有员工的排班记录。
新增员工排班记录	管理员可以为所有员工新增排班记录，新增时需要通过查询排班模板库获取可选排班模板填入。
修改员工排班记录	管理员可以修改所有员工的排班记录，逻辑与新增时类似。
查看排班模板	管理员可以查看所有排班模板，排班模板内容包括班次名称，时间段。
删除排班模板	管理员可以删除排班模板。

续表 2.10 排班管理模块管理员功能说明表

管理员功能	功能说明
新增排班模板	管理员可以新增排班模板，需要填充排班模板中的所有字段。
修改排班模板	管理员可以修改排班模板，逻辑与新增时类似。

排班管理模块可以划分为两个子模块：排班模板模块，排班记录模块。员工仅能访问排班记录模块且只能查看自己的排班记录，而管理员可以访问这两个模块并拥有全部权限。

排班模板模块中的字段有班次名称和时间段。主要作用为统一排班模块模板中的时间段，不用重复输入时间段。

排班记录模块中的字段有员工、日期和排班模板。通过查询并添加排班模板模块中的排班模板就能统一时间段，便于管理员工的排班记录。

2.6 预约服务管理模块

预约服务模块的主要功能点是对预约的增删改查，以及对服务的增删改查。三种角色都能够访问该模块，不同角色权限不同，用例图见图 2.7。

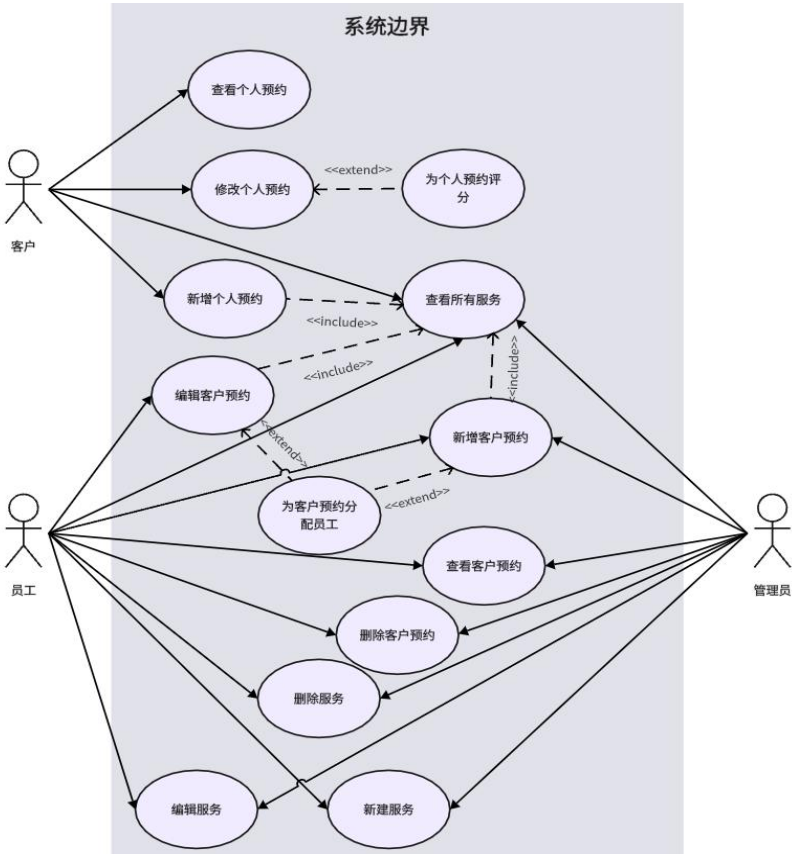


图 2.7 预约服务管理模块用例图

该模块中管理员和员工权限一致，这里合并讨论其功能并给出表 2.11 和表 2.12。

表 2.11 预约服务管理模块客户功能表

客户功能	功能说明
查看个人预约	客户仅能够查看自己的预约，预约内容包含客户名、服务项目、预约日期、预约时间、服务员工、状态等字段。
修改个人预约	客户修改功能有限，当预约状态为待确认时能够将状态修改为已取消。当预约状态为已结束时，客户能为预约评分。
为个人预约评分	客户能够为预约状态为已结束的预约进行评分，评分范围为 1~10 的整数。
新增个人预约	客户能够新增预约，新增预约时需要从服务库中选取服务项目并填写其它字段。新增预约后预约状态为待确认，且客户不可修改该状态。
查看所有服务	客户能够查看所有提供的服务，服务内容包含服务名称和服务持续时间。

表 2.12 预约服务管理模块管理员/员工功能表

管理员/员工功能	功能说明
查看客户预约	管理员和员工能够查看所有客户的预约。
新增客户预约	管理员和员工能够为所有客户新增预约，同客户新增个人预约类似，这里可以指定客户和服务员工以及预约状态，同时能将预约和护理计划绑定。
编辑客户预约	管理员和员工能够编辑所有客户的预约，逻辑和新增客户预约类似。
为客户预约分配员工	管理员和员工能够为客户预约分配服务员工，只能将预约分配给排班时间包含了预约预定时间到结束时间段且该时间段无其它预约的员工。
删除客户预约	管理员和员工能够删除所有客户的预约。
查看所有服务	管理员和员工能够查看所有提供的服务。
删除服务	管理员和员工能够删除服务。
新增服务	管理员和员工能够新增服务。
编辑服务	管理员和员工能够编辑服务。

预约服务模块可以分为两个子模块：服务模块和预约模块。客户能够查看服务模块中的所有服务，而管理员和员工拥有服务模块的所有权限。对于预约模块，

客户能够查看自己的预约、新增预约、取消预约、为预约评分，而管理员和员工拥有预约模块除了评分外的所有权限。

服务模块中的服务包含的内容有服务名称和服务持续时长。

预约模块中的预约包含的内容有客户名、预约时间、结束时间、服务项目、预约日期、服务员工、状态、满意度、护理计划。这里的结束时间不由用户填充，根据服务项目得持续时长和预定时间计算得来；护理计划字段若不为空则表明绑定了某护理计划，若为空则表明为独立预约，没有绑定任何护理计划。

2.7 通知提醒模块

通知提醒模块的主要功能点是查看通知以及标记通知已读。不同角色的所有用户都能访问该模块且对该模块的权限一致，用例图见图 2.8。

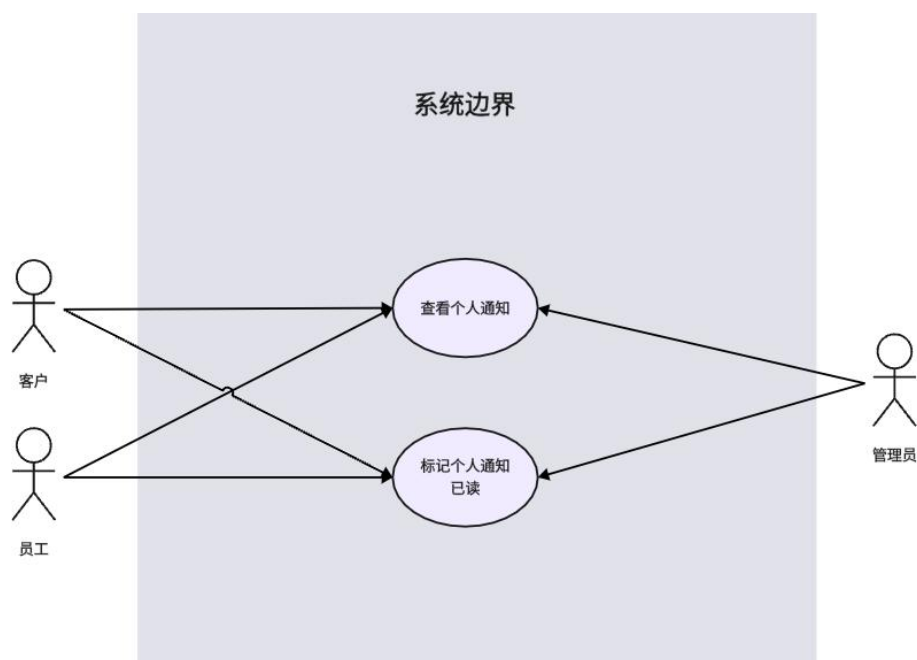


图 2.8 通知提醒模块用例图

在该模块中三种角色功能一致，这里合并讨论其功能，并给出表 2.13。

表 2.13 通知提醒模块管理员/员工/客户功能表

管理员/员工/客户功能	功能说明
查看个人通知	所有角色都能够查看个人通知。
标记个人通知已读	所有角色都能够标记个人通知为已读。

在该模块中所有角色拥有权限相同，仅有查看通知及标记通知为已读的权限。这里采用预定义通知策略，通知内容和触发条件由系统预先配置，用户无法修改或扩展。

预期设定的通知有：a) 定时通知管理员当前有多少预约未指定服务员工，提醒其及时指派；b) 在即将到预约预定时间时通知服务员工和客户，提醒其及时参与预约；c) 当护理计划未处于已完成状态时，每日向该计划的负责员工发出通知，提醒其监督该计划顺利执行。

2.8 其它功能说明

2.8.1 登录注册模块

登录注册模块的主要功能点为注册、登录和重置密码。任何用户都需要通过登录进入系统，其它所有模块的用例都包含了用户登录，用例图见图 2.9。

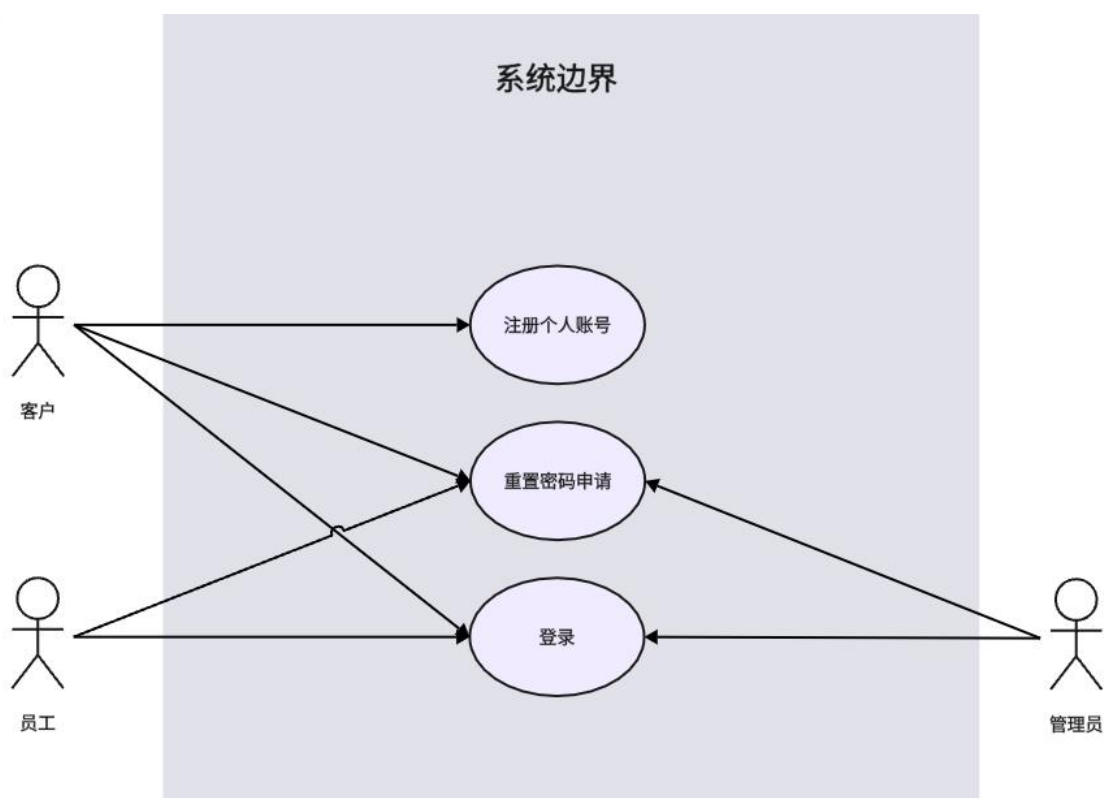


图 2.9 登录注册模块用例图

在登录注册模块中，由于此时还未登录系统确认角色，所以功能无差异。这里合并讨论三种角色的功能，并给出表 2.14。

表 2.14 登录注册模块管理员/员工/客户功能表

功能	角色	功能说明
注册个人账号	客户	在注册时，注册账号默认角色为客户。对于管理员、员工的注册放在 2.8.2 中讨论。
重置密码申请	客户、员工、 管理员	当用户忘记密码时，可以提出重置密码申请，输入用户名即可提出申请。关于批准并重置密码在 2.8.2 中讨论。
登录	客户、员工、 管理员	不论用户为何种角色，都需要通过该模块进行登录并进入系统。

在该模块中没有完整的实体用于展示，而是提供一个窗口用于登进系统。登录需要的字段有用户名和密码；重置密码需要的字段有用户名；客户注册时需要的字段有：用户名，姓名，邮箱，密码。

2.8.2 用户管理模块

用户管理模块的主要功能点为对所有用户的管理，只有管理员角色的用户才能访问该模块并拥有该模块的全部功能权限。用例图见图 2.10。

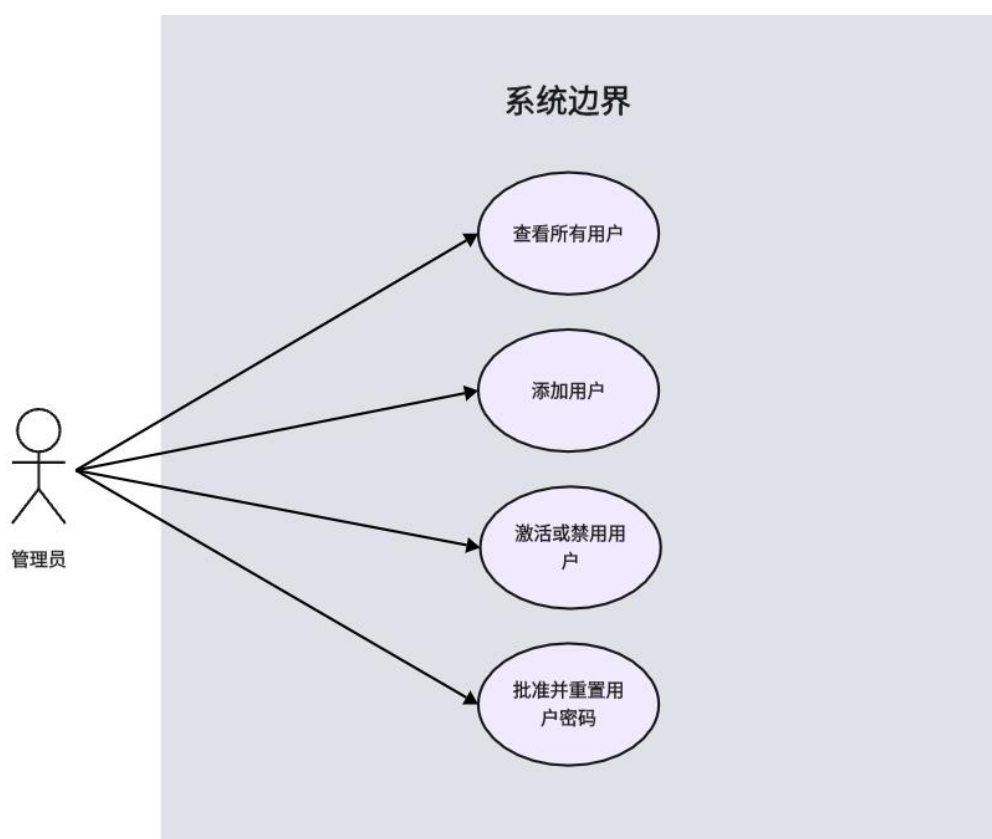


图 2.10 用户管理模块用例图

此模块的管理员功能说明表见表 2.15。

表 2.15 用户管理模块管理员功能表

管理员功能	功能说明
查看所有用户	能够查看所有用户的账号信息，内容包括用户名、姓名、邮箱、用户身份、最后登录时间、注册时间、账号激活状态、重置密码请求。
添加用户	能够添加任意角色的用户，这里可以添加员工和管理员角色的用户。
激活或禁用用户	能够激活或禁用用户账号，被禁用的用户账号将无法再登录系统。
批准并重置用户密码	在登录注册模块中提交的重置密码申请，在本模块中可以被批准并将密码重置为初始密码，初始密码为 123456。

2.8.3 个人信息管理模块

个人信息管理模块的主要功能点是查询和修改自己的个人信息，所有角色都能够访问，用例图见图 2.11。

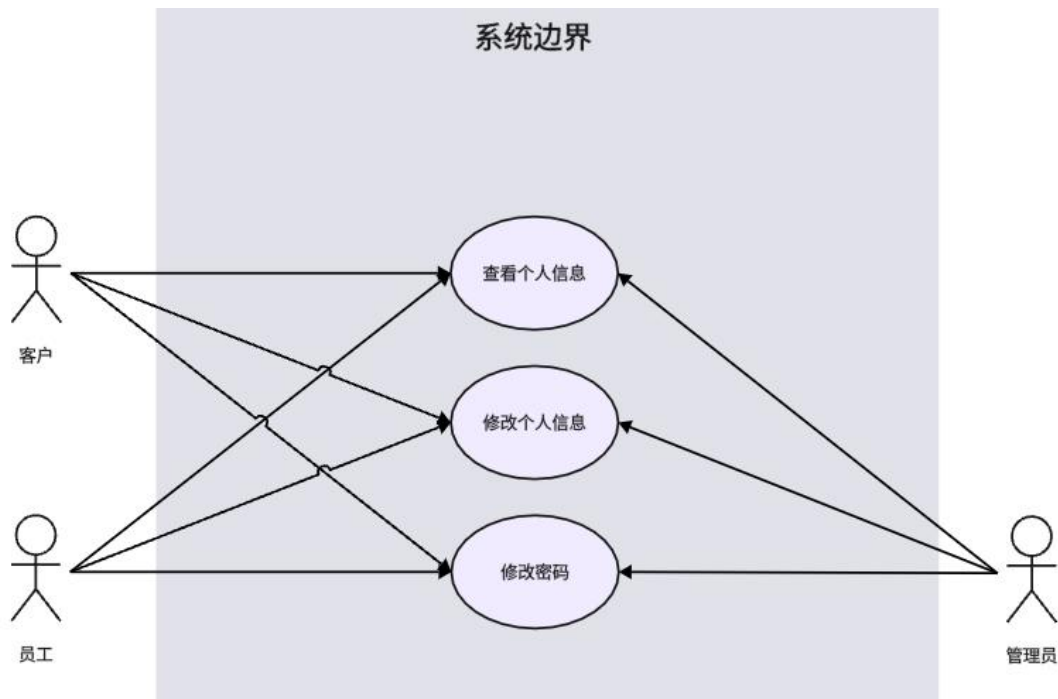


图 2.11 个人信息模块用例图

在该模块中所有角色的功能一致，这里合并讨论其功能并给出表 2.16。

表 2.16 个人信息管理模块管理员/客户/员工功能表

管理员/客户/员工功能	功能说明
查看个人信息	所有角色都能够查看自己账号的个人信息，能够被查看的个人信息包括用户名，姓名，邮箱，角色。
修改个人信息	所有角色都能够修改自己账号的个人信息，能够被修改的个人信息包括用户名，姓名，邮箱。
修改密码	所有角色都能够修改自己账号的密码，修改密码需要输入旧密码和新密码，旧密码验证成功才能更新为新密码。

2.8.4 报表展示模块

报表展示模块的主要功能是将各类数据展示出来，三种角色都能够访问该模块，不同角色的权限不同，用例图见图 2.12。

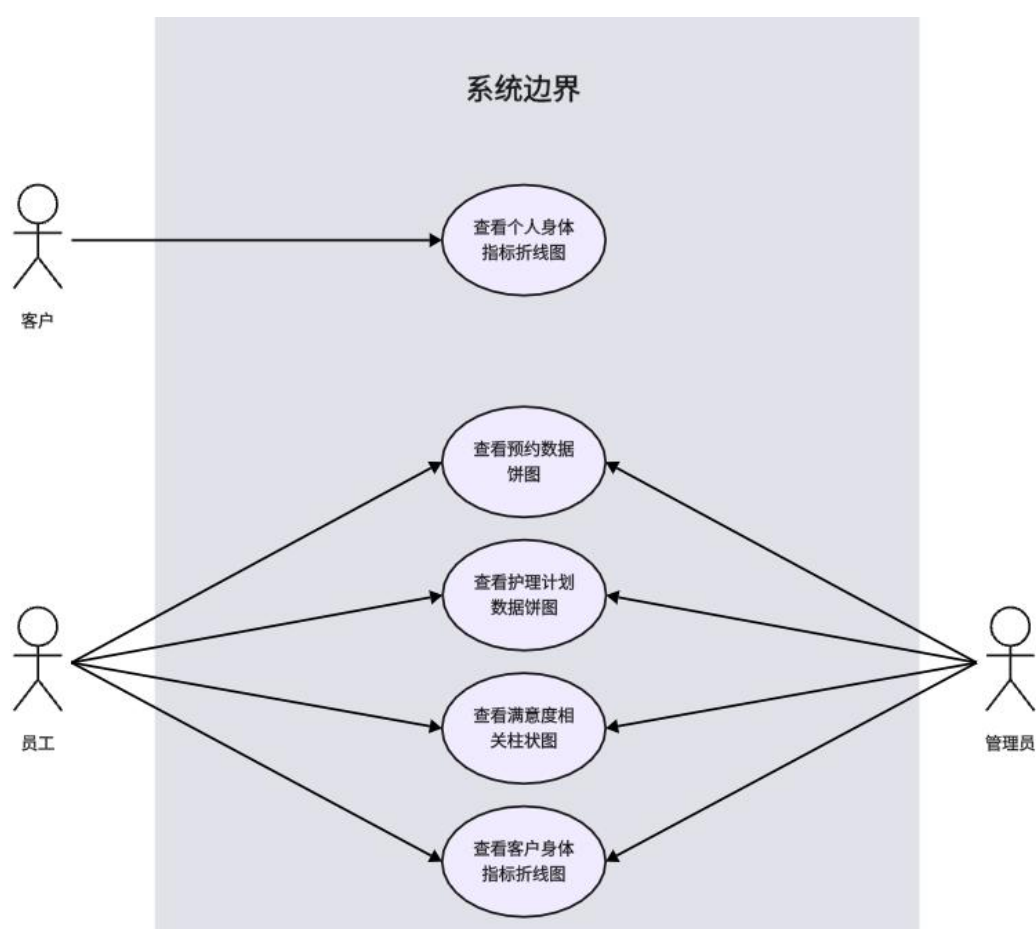


图 2.12 报表展示模块用例图

在该模块中员工和管理员权限一致，我们合并讨论其功能并给出表 2.17 和表

2.18。

表 2.17 报表展示模块客户功能说明表

客户功能	功能说明
查看个人身体指标折线图	客户可以在该模块中查看自己在健康档案模块中记录的身体指标随日期变化的折线图。

表 2.18 报表展示模块管理员/员工功能说明表

管理员/员工功能	功能说明
查看预约数据饼图	管理员和员工能够查看包括了所有预约的满意度饼图、预约状态饼图和预约的服务类型饼图。
查看护理计划饼图	管理员和员工能够查看包括了所有护理计划的计划满意度饼图、计划状态饼图和计划目标状态饼图。
查看满意度相关柱状图	管理员和员工能够查看预约满意度的均值随时间变化的趋势柱状图、护理计划满意度的均值随时间变化的趋势柱状图和所有曾被评分的服务的满意度均值的柱状图，其中前两个图能够选择日、月、年三种粒度的横坐标。
查看客户身体指标折线图	管理员和员工能够查看所有客户的身体指标的折线图。

2.9 非功能需求

非功能需求定义了系统应具备的质量属性，是衡量系统成功与否的重要标准。

2.9.1 性能需求分析

系统的性能直接影响用户体验和业务处理效率，相关的性能指标如下：

响应时间：对于核心操作（如登录、查询信息、提交表单），平均响应时间应小于 2 秒。对于复杂查询或数据处理操作，平均响应时间应小于 5 秒。

并发用户数：根据用户量估算（见 7.3 节），系统应能稳定支持至少 50 个并发用户的正常操作。

吞吐量：在正常负载下（50 并发用户），系统应能维持稳定的事务处理能力（Transactions Per Second, TPS），具体目标值将在 7.3 节中确定。

2.9.2 安全性需求分析

系统需要有保护用户数据的能力，具体体现在如下方面：

身份的认证方面，所有用户均务必凭借用户名以及密码来完成身份方面的验证

操作，唯有如此才能够对系统予以访问。对于密码而言，其需要以加密的方式进行存储，并且要强制性地设定相应的复杂度方面的要求。

访问控制方面，系统要实现基于角色的访问控制，也就是 RBAC 模式。像管理员、员工、客户这些不同角色的用户，各自有着不一样的操作权限。如此一来，便能保证用户仅仅可以访问处在自身权限范围之内的功能以及数据。

数据传输安全方面，前后端之间传输数据时得运用 HTTPS 协议来加以加密，其目的在于防止数据在传输期间被窃听，亦或是出现数据被篡改的情况。

系统需具备一定的防护能力，以便能抵御那些常见的 Web 攻击，像是 SQL 注入、跨站脚本攻击（也就是 XSS）以及跨站请求伪造（即 CSRF）等这类攻击形式。

系统需对关键操作日志予以记录，像是登录操作以及重要数据修改之类的操作日志都应记录下来，如此一来，便能够便于后续开展安全审计工作以及进行问题追踪事宜。

2.10 本章小结

本章通过细致的需求分析，明确了康养中心信息管理系统的核心目标与待解决问题，并利用 UML 用例图清晰地展示了管理员、员工、客户三类用户在七大核心功能模块及三个辅助功能模块中的具体操作权限与功能需求，最后还界定了系统在性能和安全性方面的非功能性要求，为系统架构设计和功能实现提供了明确指引。

3 系统技术架构

本章节将分为平台设计和关键技术概述两个部分进行介绍。在满足现有需求的基础上选择能够快速开发和扩展性强的系统架构和相关技术是本文设计并实现的康养中心信息管理系统的核心目标。针对老年群体健康管理的特殊性和康养机构业务流程的复杂性，系统架构需兼顾数据安全性、访问效率与用户友好性。本章将详细阐述系统的技术选型依据、架构设计思路以及各层次技术组件的功能与协作方式，展示如何通过合理的技术架构设计解决康养机构在信息管理中的数据孤岛、服务精准化与智能化程度不足等关键问题。同时，本章还将介绍系统如何通过前后端分离架构实现业务逻辑与用户界面的解耦，以及如何通过多层数据存储策略优化系统性能，为后续功能模块的实现奠定技术基础。

3.1 平台设计

本系统采用 B/S（浏览器/服务器）架构模式，相较于传统 C/S 架构，B/S 模式具有跨平台性强、维护成本低、部署便捷等优势^[19]。系统的整体架构由前端展示层、业务逻辑层以及数据持久层这三个部分构成，其运用的是前后端分离的开发模式，在各个不同的层级之间，是凭借着标准化的接口来实现数据的交互活动的。

系统的物理架构遵循的是“1+2+1”这样的模式，具体来说就是包含 1 个客户端，这里的客户端指的是 Web 浏览器，还有 2 个服务端组件，这 2 个组件分别是 Web 服务器以及应用服务器，另外再加上 1 个数据库服务器。要知道的是，因为本系统一直都是在开发环境下运行的，所以并没有去配置像 Nginx 和 Apache 这类常见的 Web 服务器，反而是采用了 Django 自身所带的开发服务器来运行整个系统。

在逻辑架构方面，系统达成了功能模块化以及数据流程标准化的状态，各个模块之间借助 RESTful API 接口来完成数据的交换操作^[20]。前端运用的是单页面应用（SPA）模式，后端则采用了模块化的设计方式，把业务功能拆解成相对独立的服务组件，如此一来便增强了系统在扩展性与维护性方面的表现。

就康养机构数据处理所呈现出的特点而言，系统运用了一种双层数据存储的策略。具体来讲，MySQL 承担起了存储那些具有持久化特性的结构化数据的职责，像用户信息、健康档案之类的数据就由它来负责存储。而 Redis，它扮演的角色是分布式缓存中间件，主要负责对高频访问视图结果进行缓存，比如说客户列表、报表数据等等，同时它还会缓存会话信息等属于热点的数据，并且还充当 Celery 的中间人（broker）^[21]。除此之外，系统还会借助 JSONField 来存储半结构化数据，

像营养信息以及紧急联系人信息就是通过这种方式存储的。这样的一种多层存储设计，很好地在数据一致性和访问效率之间达成了平衡，能够充分满足康养机构在数据处理方面所呈现出的多样化需求。

3.2 关键技术概述

3.2.1 前端技术栈：Vue 3

系统的前端是借助 Vue 3 框架来展开开发工作的，该框架通过运用组合式 API 以及<script setup>语法，达成了组件化、具备响应式特点的开发模式^[22]。在构建工具方面选用的是 Vite，它能够给予更快的开发服务器启动速度，并且实现热模块替换。UI 层运用了 Element Plus 组件库，从而完成了契合老年用户需求的界面设计，像是具备大字体、有着高对比度以及呈现简洁布局等方面的特性。前端的数据状态管理运用的是 Vuex 4，而路由管理则是使用 Vue Router 4。此外，系统还将 ECharts 数据可视化库整合了进来，以此用来展示图表以及统计信息，同时利用 Axios 来处理 HTTP 请求，进而实现和后端 API 的交互。

Vue 3 框架运用了组合式 API 以及<script setup>语法，相较于 Vue 2 的选项式 API 而言，其在代码组织性与复用性方面表现得更为出色。组合式 API 能够让开发者以更为灵活的方式去组织组件逻辑，进而使得代码的可维护性以及可测试性都得到了提升。

Vite 作为新出现的一代构建工具，其运用原生 ES 模块导入的方式，达成了按需编译的效果，实实在在地提升了开发服务器启动速度以及热更新方面的性能。相较于传统的 webpack 而言，Vite 所带来的开发体验要更为出色，其构建速度也明显更快一些。

Element Plus 组件库是依托 Vue 3 来开发的，它配备了颇为丰富的 UI 组件，同时还具备主题定制方面的能力，这就使得它尤其契合用来构建企业级的应用。该组件库在组件设计这块儿遵循着无障碍的标准，从而给老年用户给予了很不错的可访问性方面的支持。

3.2.2 后端技术栈：Django + Django REST Framework (DRF)

后端方面，选用 Python 语言来进行开发，同时运用 Django 5.1.6 框架。在这一过程中，借助 Django 具备的 ORM（对象关系映射）功能，使得数据库操作得以简化。系统的 API 接口是依靠 Django REST Framework 3.15.2 来实现的，并且它支持基于 Token 的认证机制，如此一来，便能够保障数据传输的安全性。为了能够妥善处理定时任务，系统把 Celery 5.4.0 任务队列给集成了进来，从而实现了像针对未分配员工发出预约通知、对即将到来的预约给予提醒等这类定时任务。而 API 文档则是通过 drf-yasg (Swagger)来达成的，其能够给予交互式接口文档。

Django 框架秉持着“内置电池”的设计理念，实实在在地给出了一整套完备的 Web 开发解决方案。它所具备的 ORM 系统能够对多种数据库后端予以支持，借助对象关系映射这一方式，把数据库操作给简化了，进而让开发效率得以提升。

Django REST Framework 有着颇为强大的序列化器以及视图集，这对 RESTful API 的快速开发起到了有力的支撑作用。其认证系统能够支持多种多样的认证方式，像 Token 认证、Session 认证等等，如此一来便有力地确保了 API 的安全性。

Celery 作为分布式任务队列，支持异步任务处理和定时任务调度。其任务调度机制灵活，可以处理复杂的任务依赖关系，适合处理康养系统中的各类定时通知任务。

3.2.3 数据库设计与优化：Mysql

系统数据层把 MySQL 当作主要的关系型数据库来用，在设计的时候遵循第三范式（3NF）的设计原则，以此来保证数据能保持一致性^[23]。在数据库设计环节，着重关注了老年健康数据所具有的特殊性，还构建了好些相互关联的数据模型，像 Users（用户）、Roles（角色）、Clients（客户）、CarePlans（护理计划）、HealthMetrics（健康指标）、DietPlans（饮食计划）等等。面对高并发访问的场景，实施了较为合理的索引优化策略，使得查询效率得以提升。

MySQL 是当下主流的关系型数据库，它具备事务支持的特点，还能保证数据的一致性等诸多特性。它的索引机制能够支持好几种不同的索引类型，要是能合理地去设计索引的话，那么查询性能就可以得到颇为明显的提升。

数据库在设计之时遵循着第三范式（3NF）这一规范。它借助于消除数据所存在的冗余情况以及依赖关系等手段，有力地确保了数据所具备的一致性以及完整性特质。这种进行数据库设计的方式，虽说有可能会使得查询的复杂程度有所增加，然而却能够极为有效地防止数据出现异常的状况。

3.2.4 缓存系统设计：Redis

系统将 Redis 集成进来，使其充当分布式缓存中间件，同时充当 Celery 的消息中间人，由此达成了两项核心功能。其一，能够对常用视图结果以及会话信息等这类热点数据进行缓存操作，以此来缓解数据库所承受的负载压力；其二，担当起 Celery 任务队列的中间人角色，为系统里定时任务的调度与执行给予有力支持。如此的设计安排，极大地增强了系统针对高频请求做出响应的能力，并且还对任务处理的流程起到了优化完善的作用。

Redis 作为一款高性能的内存数据库，它所具备的键值存储模式以及多种多样的数据结构，像字符串、列表、集合、有序集合等等，是极为契合充当视图缓存以及消息队列的载体的。Redis 的原子操作这一特性，能够确保在高并发的那种环境之下，数据可以保持一致性。并且其持久化机制也使得系统在经历重启之后，

有能力将关键数据予以恢复。

系统所运用的视图缓存策略，是专门针对那些访问频率颇高然而变动相对较少的数据来设置合理的缓存过期时间的。如此一来，既能减轻数据库所承受的压力，又可以保证数据能够得到及时的更新。而在 Celery 任务队列方面，Redis 为其给出了可靠的消息传递机制，从而确保像通知提醒这类定时任务能够以高效且可靠的方式得以执行。

3.2.5 系统安全

就安全层面而言，系统成功构建起了一种以 RBAC（基于角色的访问控制）为基础的权限管理机制，与此同时，还融合了诸如 CSRF 保护以及密码安全存储这类技术，以此来对敏感健康信息予以保护。

RBAC 权限模型实现了角色跟权限的分离，由此给出了颇为灵活的访问控制机制。该模型在扩展方面以及维护方面都比较容易，能够较好地应对康养系统当中那复杂多样的权限需求。

CSRF 保护机制会去验证请求到底是从哪里来的，以此来防止出现跨站请求伪造这样的攻击情况。密码安全存储方面则是运用了加盐哈希算法，通过这种方式切实确保了用户数据具备应有的安全性。

3.3 本章小结

本章对康养中心信息管理系统的技术架构予以介绍，其内容主要涵盖平台设计以及关键技术概述这两个部分。就平台设计来讲，采用的是 B/S 架构，借助前后端分离的方式，将其划分成展示层、逻辑层和数据层。在物理架构方面，呈现的是“1+2+1”模式，从逻辑层面而言，实现了数据流的模块化与标准化，并且在数据存储上，通过结合 MySQL 和 Redis，对数据的一致性与效率加以优化。至于关键技术这一方面，前端运用的是 Vue 3、Vite 以及 Element Plus，后端则是以 Django 和 DRF 为基础，同时辅以 Redis 缓存和任务队列，在安全方面，采用 RBAC 和 CSRF 来提供保护。如此这般，可解决数据孤岛以及服务智能化不足等相关问题，进而对快速开发与扩展给予相应支持。

4 系统数据库设计与实现

4.1 数据库实体关系设计

数据库设计需要在明晰需求的前提下,参考数据库设计的三范式^[24]并考虑数据库在生产环境下可能承受的压力,设计一个尽可能减少数据冗余且符合实际生产需要的数据库。

本章使用 Crow's foot (乌鸦脚) 方法进行 E-R 图的建模^[25]。Crow's foot 方法和 Chen 方法的区别在于, Crow's foot 方法将实体用被上下分割的矩形来表示,上半部分存放实体名称,下半部分存放实体的属性。而一对一、一对多、多对多等关系则用类似鸭脚的图案来表示。这种方法避免了 Chen 方法中若实体属性过多会导致椭圆拥挤的情况。系统的整体 E-R 图如图 4.1 所示。

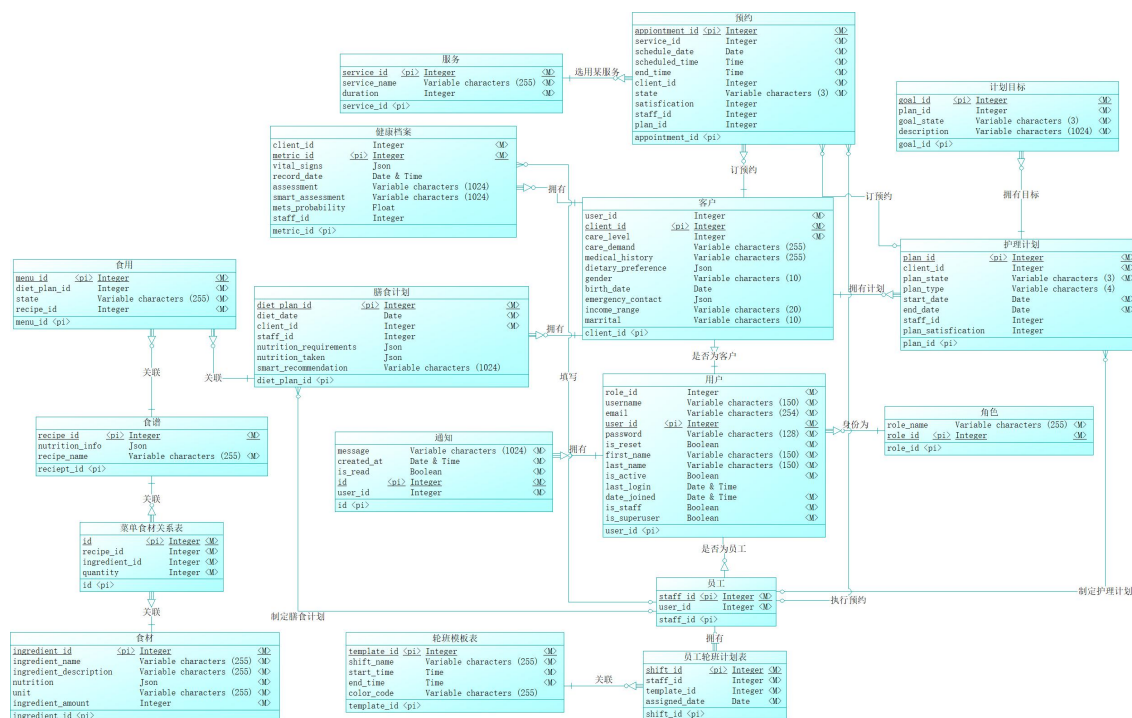


图 4.1 系统整体 E-R 图

角色相关实体的局部 E-R 图如图 4.2 所示。

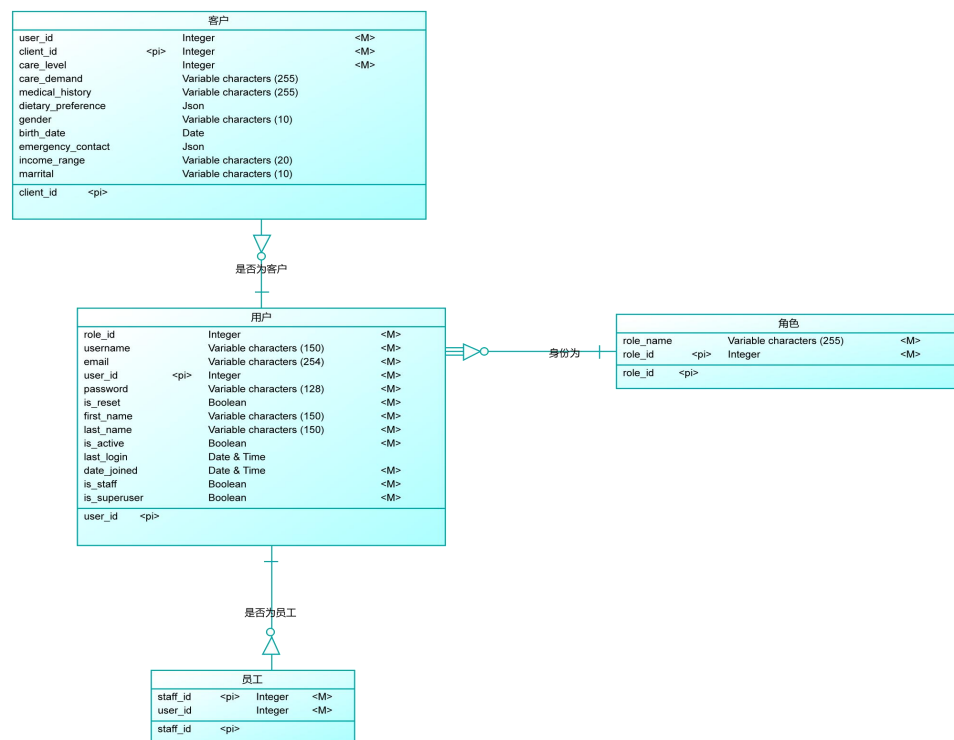


图 4.2 角色相关实体的局部 E-R 图

膳食计划、排班管理和通知的相关实体的局部 E-R 图如图 4.3 所示。

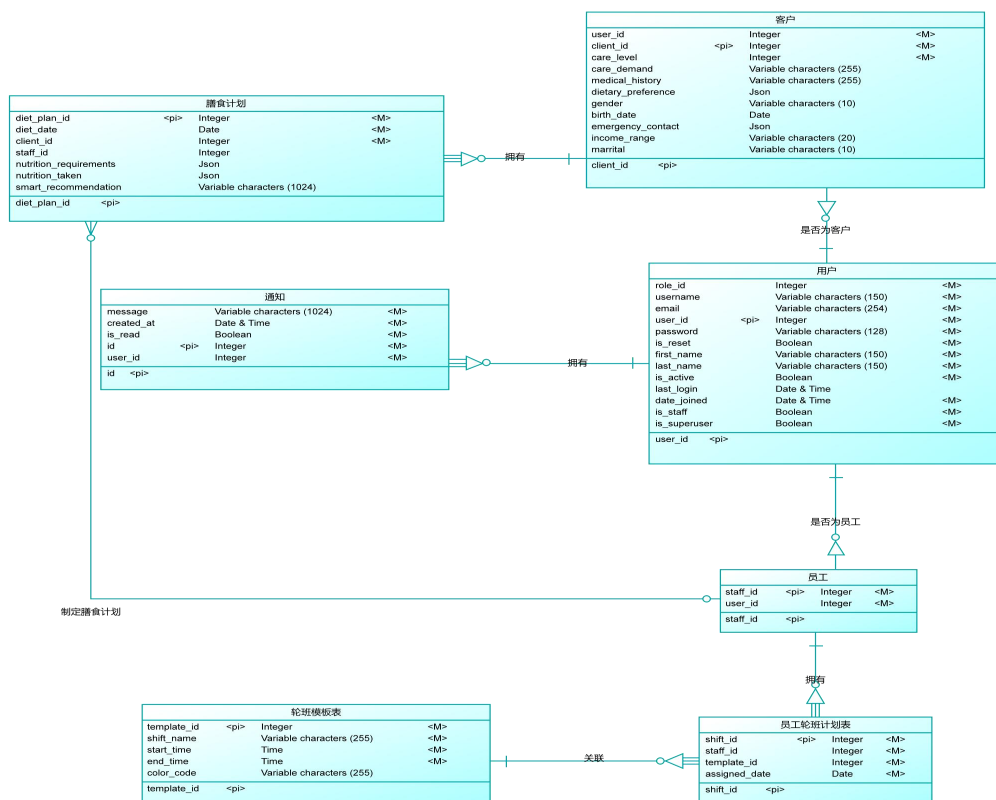


图 4.3 膳食计划、排班管理和通知相关实体的局部 E-R 图

膳食计划内部相关实体的局部 E-R 图如图 4.4 所示。

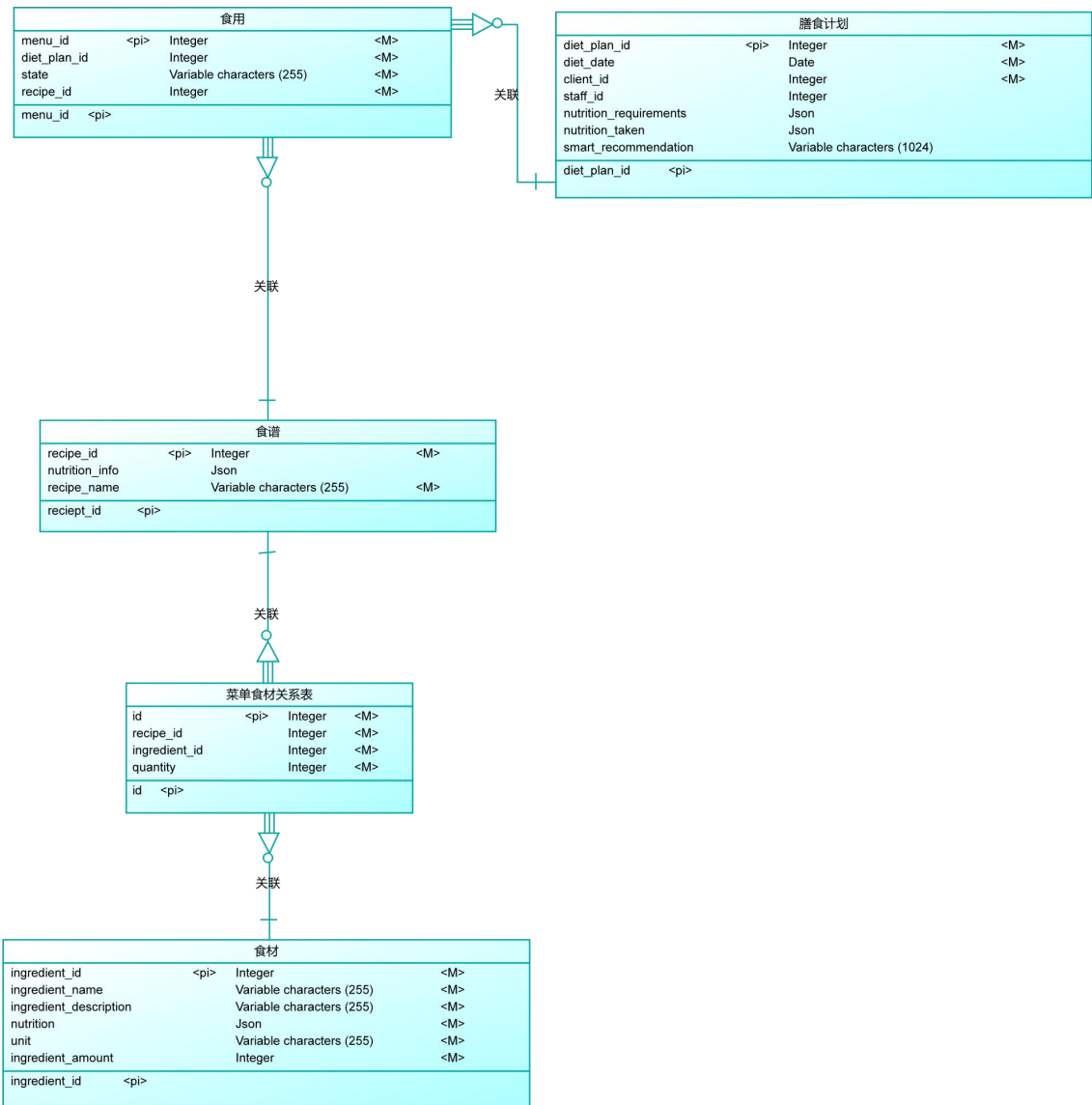


图 4.4 膳食计划内部相关实体的局部 E-R 图

预约、护理计划和健康档案相关实体的局部 E-R 图如图 4.5 所示。

完全兼容 Django 认证系统的标准协议。字段说明如表 4.1 所示。

表 4.1 用户表

序号	名称	类型	含义	是否必填	备注
1	id	bigint	用户 id	是	主键，自增
2	password	varchar	密码	是	存放密码的加盐 哈希值
3	last_login	datetime	上次登录时间	否	
4	username	varchar	用户名	是	唯一
5	first_name	varchar	名	是	
6	last_name	varchar	姓	是	
7	email	varchar	邮箱	是	
8	is_active	tinyint	用户是否激活	是	
9	date_joined	datetime	账号注册时间	是	
10	is_staff	tinyint	是否为员工	是	继承获得，非必需
11	is_superuser	tinyint	是否为管理员	是	继承获得，非必需
12	role_id	int	用户的角色 id	是	外键，角色表主键
13	is_reset	tinyint	是否重置密码	是	

(2) 员工表

员工表存储员工的基本信息。为了简化设计，本系统忽略了员工个人基本信息的存储。字段说明如表 4.2 所示。

表 4.2 员工表

序号	名称	类型	含义	是否必填	备注
1	staff_id	int	员工 id	是	主键，自增
2	user_id	int	用户 id	是	外键，用户表主键

(3) 客户表

客户表存储客户的基本信息。字段说明如表 4.3 所示。

表 4.3 客户表

序号	名称	类型	含义	是否必填	备注
1	client_id	int	客户 id	是	主键，自增
2	care_level	int	护理等级	否	
3	care_demand	varchar	护理需求	否	
4	medical_history	varchar	病历	否	
5	dietary_preference	json	饮食偏好	否	
6	birth_date	date	生日	否	
7	emergency_contact	json	紧急联系人	否	
8	gender	varchar	性别	是	
9	income_range	varchar	薪酬等级	是	取值：“小于 3000 元/月”， “3000-6000 元/月”（默认）， “大于 6000 元/月”
10	marital	varchar	结婚状态	是	取值：“已婚”， “未婚”（默认）， “离婚”，“丧偶”，“分居”
11	user_id	bigint	用户 id	是	外键，用户表主键

(4) 角色表

角色表存储角色信息。字段说明如表 4.4 所示。

表 4.4 角色表

序号	名称	类型	含义	是否必填	备注
1	role_id	int	角色 id	是	主键
2	role_name	varchar	角色名	是	

(5) 通知表

通知表存储通知信息。字段说明如表 4.5 所示。

表 4.5 通知表

序号	名称	类型	含义	是否必填	备注
1	id	bigint	通知 id	是	主键，自增
2	message	longtext	通知内容	是	
3	created_at	datetime	创建时间	是	
4	is_read	tinyint	是否已读	是	
5	user_id	bigint	用户 id	是	外键，用户表主键

(6) 员工排班表

员工排班表存储员工排班信息。字段说明如表 4.6 所示。

表 4.6 员工排班表

序号	名称	类型	含义	是否必填	备注
1	shift_id	int	排班 id	是	主键，自增
2	assigned_date	date	排班日期	是	
3	template_id	int	模板 id	是	外键，排班模板表主键
4	staff_id	int	员工 id	是	外键，员工表主键

(7) 排班模板表

排班模板表存储排班模板信息。字段说明如表 4.7 所示。

表 4.7 排班模板表

序号	名称	类型	含义	是否必填	备注
1	template_id	int	模板 id	是	主键，自增
2	shift_name	varchar	模板名	是	
3	start_time	time	开始时间	是	
4	end_time	time	结束时间	是	

续表 4.7 排班模板表

序号	名称	类型	含义	是否必填	备注
5	color_code	varchar	颜色码	否	样式，非必需

(8) 膳食计划表

膳食计划表存储膳食计划基本信息。字段说明如表 4.8 所示。

表 4.8 膳食计划表

序号	名称	类型	含义	是否必填	备注
1	diet_plan_id	int	膳食计划 id	是	主键，自增
2	diet_date	date	计划日期	是	
3	nutrition_requirements	json	营养需求	否	
4	nutrition_taken	json	营养实际摄入	否	
5	smart_recommendation	longtext	智能建议	否	
6	client_id	int	客户 id	是	外键，客户表主键
7	staff_id	int	员工 id（制定人）	否	外键，员工表主键

(9) 食谱表

食谱表存储食谱基本信息。字段说明如表 4.9 所示。

表 4.9 食谱表

序号	名称	类型	含义	是否必填	备注
1	recipe_id	int	食谱 id	是	主键，自增
2	recipe_name	varchar	食谱名称（菜品名）	是	
3	nutrition_info	json	菜品营养信息	否	

(10) 食材表

食材表存储食材信息。字段说明如表 4.10 所示。

表 4.10 食材表

序号	名称	类型	含义	是否必填	备注
1	ingredient_id	int	食材 id	是	主键，自增
2	ingredient_name	varchar	食材名	是	
3	ingredient_description	varchar	食材描述	是	
4	ingredient_amount	int	食材数量	是	
5	unit	varchar	单位	是	默认为“克”
6	nutrition	json	营养信息	是	

(11) 食用表

食用表存储膳食计划和食谱表关系。字段说明如表 4.11 所示。

表 4.11 食用表

序号	名称	类型	含义	是否必填	备注
1	id	bigint	食用 id	是	主键，自增
2	diet_plan_id	int	膳食计划 id	是	外键，膳食计划表主键
3	recipe_id	int	菜谱 id	是	外键，菜谱表主键
4	type	varchar	三餐信息	否	取值：“早餐”，“午餐”，“晚餐”

(12) 菜单食材关系表

菜单食材关系表存储食谱表和食材表关系。字段说明如表 4.12 所示。

表 4.12 菜单食材关系表

序号	名称	类型	含义	是否必填	备注
1	id	int	该表 id	是	主键，自增
2	ingredient_id	int	食材 id	是	外键，食材表主键
3	recipe_id	int	食谱 id	是	外键，食谱表主键

续表 4.12 菜单食材关系表

序号	名称	类型	含义	是否必填	备注
4	quantity	decimal	倍数	是	食谱中食材数量 依靠该字段和食 材表 ingredient_amount 相乘算出

(13) 健康档案表

健康档案表存储健康档案信息。字段说明如表 4.13 所示。

表 4.13 健康档案表

序号	名称	类型	含义	是否必填	备注
1	metric_id	int	档案 id	是	主键，自增
2	vital_signs	json	身体指标	否	
3	record_date	date	记录日期	否	
4	assessment	varchar	评估	否	
5	mets_probability	double	患 MetS 概率	否	
6	smart_assessment	varchar	智能评估	否	
7	staff_id	int	员工 id (评估人)	否	外键，员工表主 键
8	client_id	int	客户 id	是	外键，客户表主 键

(14) 护理计划表

护理计划表存储护理计划基本信息。字段说明如表 4.14 所示。

表 4.14 护理计划表

序号	名称	类型	含义	是否必填	备注
1	plan_id	int	计划 id	是	主键，自增
2	plan_state	varchar	计划状态	是	取值：“待开始”， “进行中”，“已 完成”

续表 4.14 护理计划表

序号	名称	类型	含义	是否必填	备注
3	plan_type	varchar	计划类型	否	取值：“康复护理”，“日常护理”
4	start_date	date	计划开始日期	是	
5	end_date	date	计划结束日期	是	
6	plan_satisfaction	int	计划满意度	否	
7	staff_id	int	员工 id（制定人）	否	外键，员工表主键
8	client_id	int	客户 id	是	外键，客户表主键

(15) 计划目标表

计划目标表存储护理计划的目标。字段说明如表 4.15 所示。

表 4.15 计划目标表

序号	名称	类型	含义	是否必填	备注
1	goal_id	int	目标 id	是	主键，自增
2	goal_state	varchar	目标状态	是	取值：“已达成”，“未达成”（默认）
3	description	varchar	目标描述	否	
4	plan_id	int	护理计划 id	是	外键，护理计划表主键

(16) 预约表

预约表存储预约信息。字段说明如表 4.16 所示。

表 4.16 预约表

序号	名称	类型	含义	是否必填	备注
1	appointment_id	int	预约 id	是	主键，自增
2	schedule_date	date	预约日期	是	

续表 4.16 预约表

序号	名称	类型	含义	是否必填	备注
3	schedule_time	time	预约时间	是	
4	end_time	time	预计结束时间	否	
5	state	varchar	预约状态	是	取值：“待确认”（默认），“已预约”，“已结束”，“已取消”
6	service_id	int	服务 id	是	外键，服务表主键
7	satisfaction	int	预约满意度	否	
8	client_id	int	客户 id	是	外键，客户表主键
9	staff_id	int	员工 id（服务人）	否	外键，员工表主键
10	plan_id	int	护理计划 id（若绑定护理计划则有值）	否	外键，护理计划表主键

(17) 服务表

服务表存储服务信息。字段说明如表 4.17 所示。

表 4.17 服务表

序号	名称	类型	含义	是否必填	备注
1	service_id	int	服务 id	是	主键，自增
2	service_name	varchar	服务名	是	
3	duration	int	持续时长（单位为分钟）	否	

4.3 本章小结

本章细致地对康养中心信息管理系统的数据库设计与实现加以阐述。起初，运用 Crow's foot 方法来开展实体关系（E-R）建模工作，将系统的整体 E-R 图呈现出来，同时把角色、膳食计划、排班管理、通知、预约、护理计划以及健康档案

等诸多相关实体的局部 E-R 图也予以展示，以此保证数据库设计既能削减数据冗余的状况，又可契合实际生产方面的需求。随后，在数据库物理结构设计环节选用了 MySQL，并且借助 Django ORM 模型驱动设计来自动生成表结构，详尽地罗列出用户、员工、客户、角色、通知、排班、膳食计划、食谱、食材、健康档案、护理计划、预约以及服务等总共 17 个关键数据表的字段信息、类型、所蕴含的含义以及约束条件等内容，从而为系统功能的实现筑牢了稳固的数据根基。

5 系统功能模块设计与实现

基于需求分析和数据库设计，本章将设计和实现需求分析中提及的各模块功能。出于篇幅考虑，本章将有选择地描述各模块关键功能的设计与实现，并忽略后端接口的具体实现而只叙述一些关键设计。关键代码实现可见附录 A 和附录 B。

5.1 接口鉴权功能及相关安全特性的实现

在 B/S 架构中，系统采用多层次安全机制实现接口鉴权与会话管理，虽部分设计存在优化空间，但能够有效保障基础业务场景的安全性。

5.1.1 基于会话的身份验证机制

系统利用 Django 内置会话机制实现用户状态管理，满足基础会话跟踪需求：

(1) 会话存储与管理

配置 SESSION_ENGINE 为缓存存储，相比于数据库存储能够在高并发场景下有更好的性能表现，配合 Django 的会话加密与防篡改机制，已完全满足中小规模系统的可靠性要求。

默认会话过期时间（SESSION_COOKIE_AGE）使用框架推荐值，确保基础安全性。Django 设置文件中相关配置图 5.1 所示。

```
152 # session配置
153 SESSION_ENGINE = 'django.contrib.sessions.backends.cache' # 使用数据库(db)存储 session
154 SESSION_COOKIE_AGE = 60 * 60 * 24 * 7 * 2 # 会话过期时间(单位:秒), 默认两周
155 SESSION_EXPIRE_AT_BROWSER_CLOSE = False # 浏览器关闭时会话不会失效
```

图 5.1 session 配置

(2) 登录流程实现

通过 Django 的 login 函数绑定会话，虽未集成 OAuth2 等现代化协议，但标准会话管理足够支撑当前用户名/密码认证模式的业务需求。支持@login_required 装饰器快速实现视图保护，兼顾开发效率。在登录视图中的使用如图 5.2 所示。

```
188 # 检查密码是否匹配
189 if check_password(password, user.password):
190     from django.contrib.auth import login as django_login
191     # 设置用户会话
192     django_login(request, user) # 这里会使用 Django 的 Session 来记录用户的登录状态 @login_required
```

图 5.2 登录视图中设置会话

5.1.2 基于角色的访问控制（RBAC）

系统通过硬编码角色权限类实现差异化访问控制，虽牺牲了动态权限管理的灵活性，但对于角色类型固定、权限规则稳定的早期业务场景，显著降低了开发复杂度。

(1) 自定义权限类

系统实现了三种权限类：a) **IsAdmin**：只允许管理员角色访问；b) **IsStaff**：只允许员工角色访问；c) **IsClient**：只允许客户角色访问其自身资源。这里展示 **IsClient** 权限类的实现，如图 5.3 所示。

```
26 class IsClient(permissions.BasePermission):
27     """
28     客户角色的用户访问自身资源
29     """
30     4 个用法 (4 个动态)  Viitttoooo
31     def has_permission(self, request, view):
32         # 确保用户已登录，并且是客户
33         if request.user and request.user.is_authenticated and request.user.is_active:
34             # 尝试从 URL 路径参数中获取 client_id
35             client_id = view.kwargs.get('client_id')
36
37             # 如果路径参数中没有 client_id，则尝试从查询参数中获取
38             if client_id is None:
39                 client_id = int(request.query_params.get('client_id'))
40             # 如果两种方式都获取不到 client_id，则拒绝访问
41             if client_id is None:
42                 return False
43             return (request.user.role.role_name == '客户' and
44                     client_id == Clients.objects.get(user_id=request.user.id).client_id)
45         return False
```

图 5.3 IsClient 权限类

(2) 权限验证流程

在使用权限类进行权限验证时有两种方式，一种是使用 DRF 提供的鉴权装饰器，另一种则是自行检查。两种实现分别见图 5.4 和图 5.5。

```
504 @permission_classes([IsAdmin | IsStaff | IsClient])
```

图 5.4 DRF 提供的装饰器结合权限类实现鉴权

```
1875 permission_classes = [IsAdmin | IsStaff]
1876 for permission in permission_classes:
1877     if not permission().has_permission(request, None):
1878         return Response(data={'error': "Permission denied"}, status=status.HTTP_403_FORBIDDEN)
```

图 5.5 视图中自行鉴权

5.1.3 中间件层保护

系统通过组合 Django 安全中间件构建防御体系，虽部分配置需随环境变化调整，但已覆盖主流安全威胁：

(1) 会话与认证中间件

SessionMiddleware + AuthenticationMiddleware 提供无状态 HTTP 协议下的身份连续性，满足基础会话管理需求。

(2) CSRF 防护配置

CsrfViewMiddleware 配合 CSRF_TRUSTED_ORIGINS，虽前端需显式携带 CSRF Token，但确保传统表单提交与 AJAX 请求的安全。

(3) 安全与跨域控制

SecurityMiddleware 提供的 HSTS、CSP 等 HTTP 头默认开启，短期内可抵御常规 Web 攻击。

CorsMiddleware 限定 CORS_ALLOWED_ORIGINS 为前端开发地址，虽未实现动态白名单，但符合当前前后端分离开发模式的跨域需求。

5.1.4 密码安全管理

系统通过 Django 原生密码工具保障认证安全：

使用 make_password 的 PBKDF2 算法哈希存储密码，虽未强制升级至 Argon2 等更高强度算法，但默认配置已达到行业基准要求。

check_password 自动处理哈希验证，避免自行实现可能引入的时序攻击风险。

5.2 登录、用户管理和个人资料及通知模块

根据需求，登录模块需要提供注册、忘记密码和登录三个功能；用户管理模块需要为管理员提供注册用户、重置密码和激活或禁用用户三个功能。需要注意的是在登录模块注册时，用户默认被注册为客户；而在用户管理模块中，管理员能够注册任意身份的用户。登录模块各功能和用户管理模块各功能分别如图 5.6 和图 5.7 所示。



图 5.6 登录模块各功能图



图 5.7 用户管理模块各功能图

关于用户个人信息的修改和查看以及登出操作，不为管理员身份的用户可以通过登录成功后页面右上角的个人资料下拉栏中找到入口。对于客户身份的用户还能够在下拉框中找到个人客户资料的入口，同时通知按钮也位于右上角，详情见图 5.8 和 5.9。



图 5.8 个人资料功能图

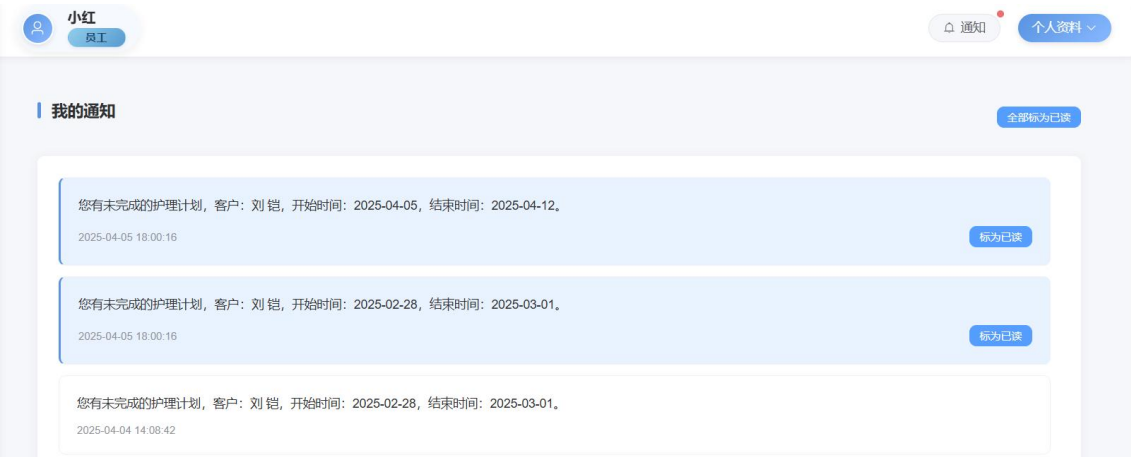


图 5.9 通知按钮及通知界面图

5.3 首页报表展示模块

当用户登录进入主界面之后，三种角色显示的主界面略有不同。不同之处包括左侧扩展栏中显示功能模块的不同、界面上方的身份标识和首页模块中的图表展示（员工和管理员首页图表一致），详情见图 5.10。



图 5.10 主界面差异图

对于客户来说，首页仅展示自己的健康档案中各类身体指标随时间变化的曲线图，如图 5.10 中客户图所示。对于管理员和员工来说，首页中展示了预约数据：预约满意度占比饼图、预约状态占比饼图、服务类型占比饼图；护理计划数据：计划满意度占比饼图、计划状态饼图、计划目标状态饼图；满意度数据：预约平均满意度随时间变化的趋势图、护理计划平均满意度随时间变化的趋势图、各服务的平均满意度柱状图；客户列表，当点击客户列表中某客户后能够查看客户的身体指标曲线图，如图 5.11 和图 5.12 所示。



图 5.11 首页报表（管理员和员工视角）其一



图 5.12 首页报表（管理员和员工视角）其二

5.4 客户档案管理模块

客户档案管理模块的功能为展示客户档案和修改客户档案。客户视角下，个人的客户档案入口在个人资料中，如图 5.8 所示。员工和管理员视角下则通过客户档案模块对客户档案进行访问，该模块入口和选中后展示出来的客户列表如图 5.13 所示。当选中某客户时，其档案详情会通过下拉栏的方式展示出来，展示内容包括客户的基本信息如姓名、性别、出生年月、护理需求、膳食偏好、紧急联系人等，如图 5.14 所示。

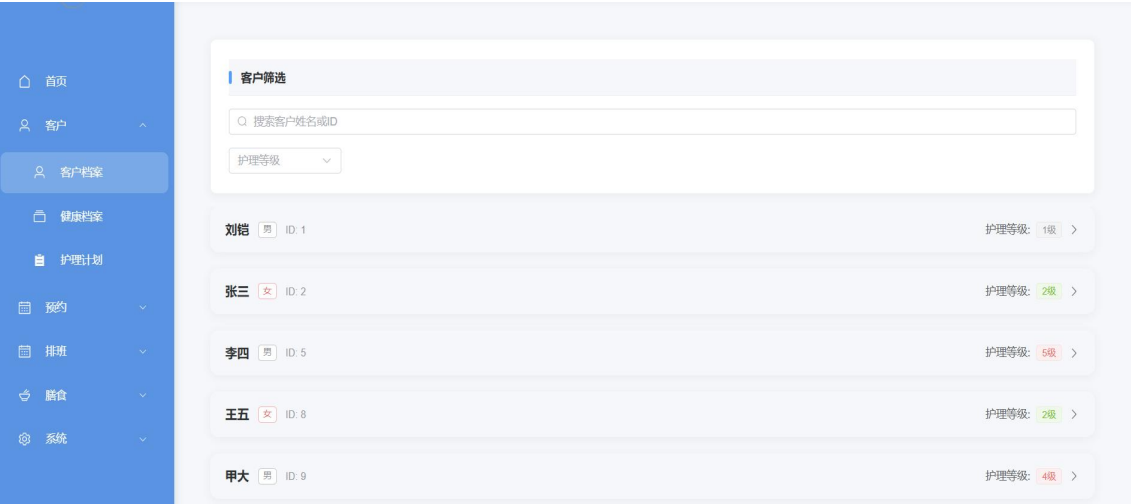


图 5.13 客户档案

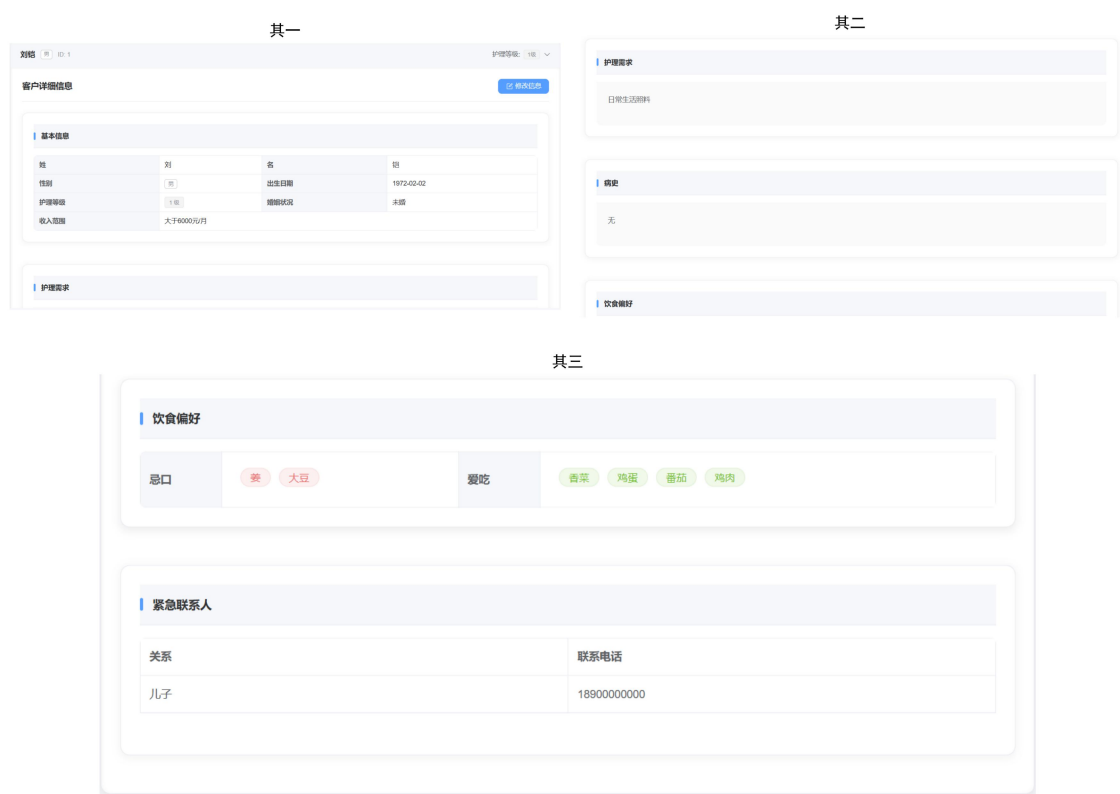


图 5.14 客户档案详情

5.5 健康档案管理模块

健康档案管理模块的主要功能是对健康档案的增删改查，不同视角下权限不一致，详情见章节 2.3。客户视角和管理员与员工视角的健康档案模块内容分别如图 5.15 和图 5.16 所示。

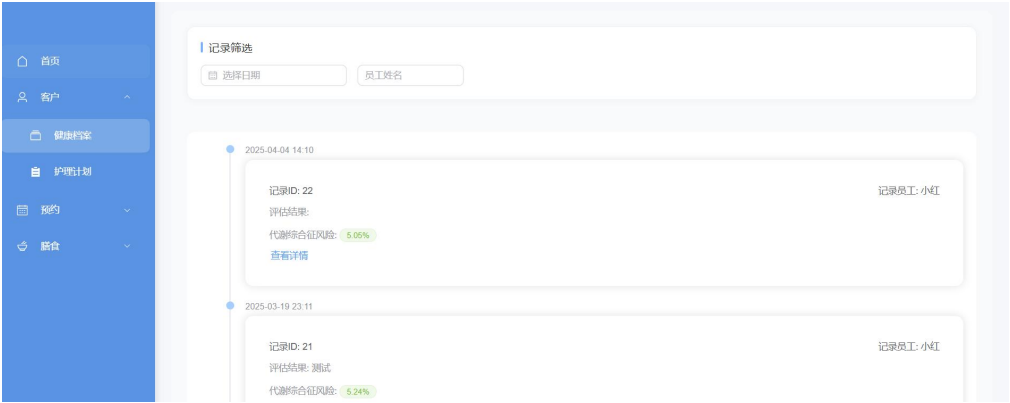


图 5.15 健康档案模块（客户视角）

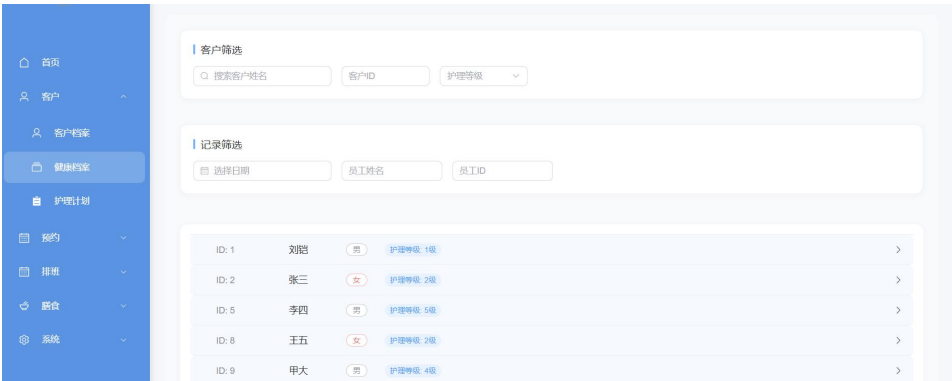


图 5.16 健康档案模块（管理员与员工视角）

两种视角的区别在于，管理员与员工能够先查看客户列表再查看某客户的健康档案并拥有增删改的功能。当管理员与员工点选中某客户时，其展示效果如图 5.17 所示。

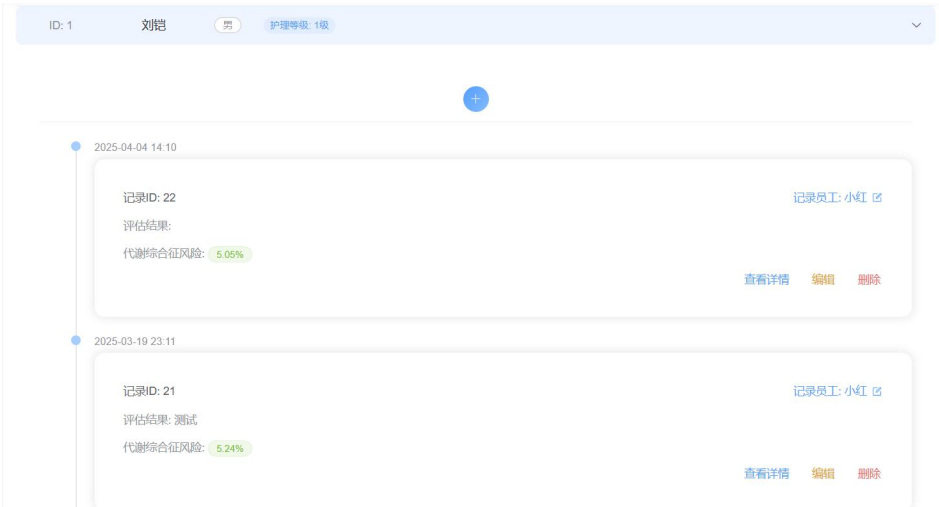


图 5.17 健康档案列表（管理员与员工视角）

编辑窗口与新增窗口类似，这里我们只展示健康档案记录详情窗口和编辑窗口，如图 5.18 所示。

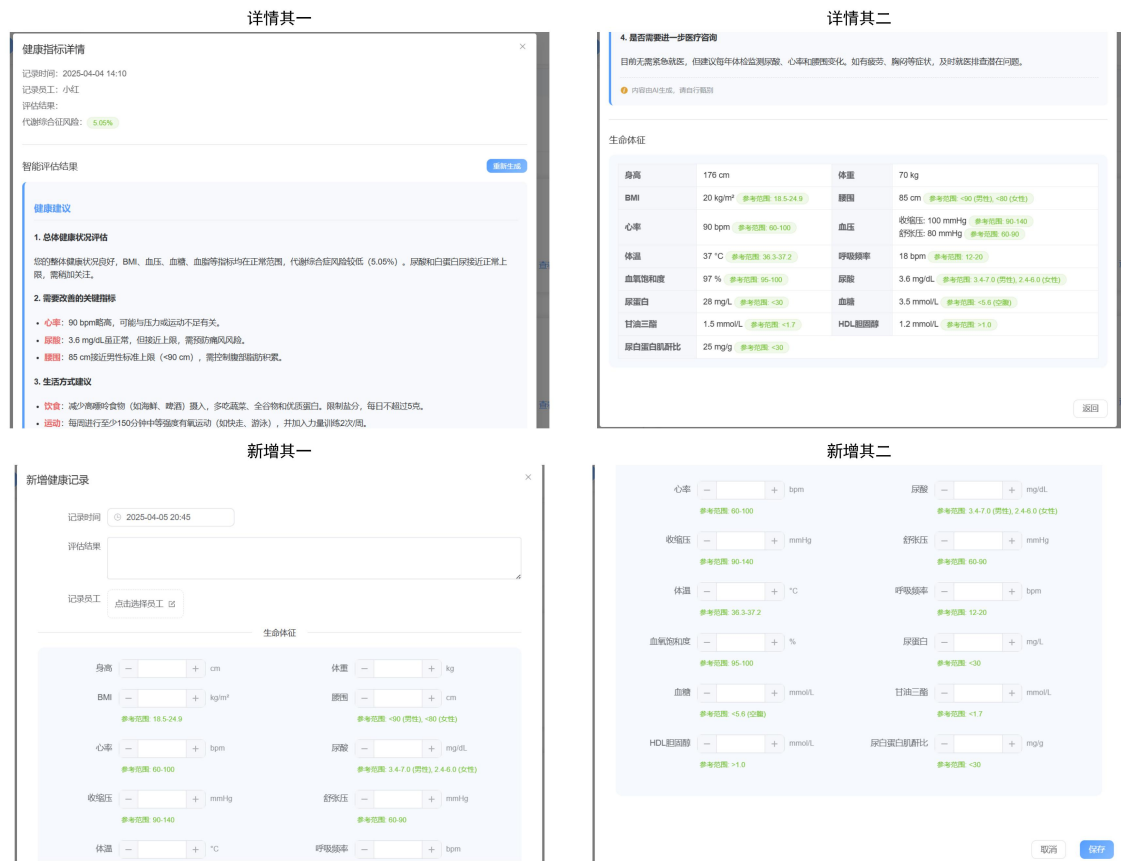


图 5.18 健康档案详情与新增窗口

5.6 护理计划管理模块

护理计划管理模块的主要功能点见章节 2.2。客户视角和管理员与员工视角的护理计划模块内容分别如图 5.19 和图 5.20 所示。



图 5.19 护理计划模块（客户视角）

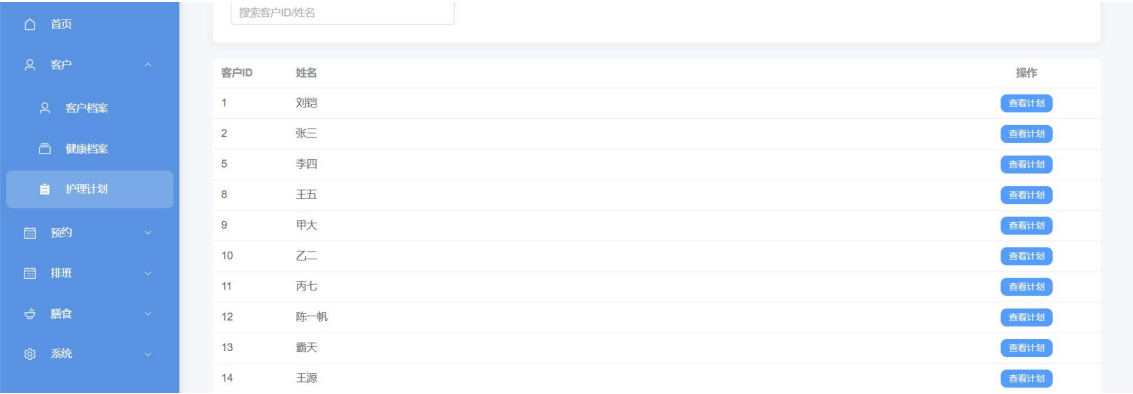


图 5.20 护理计划模块（管理员与员工视角）

两种视角的区别在于：管理员与员工能够先查看客户列表再查看某客户的护理计划并拥有增删改的功能；客户能够对状态为“已完成”的计划进行评分。当管理员与员工点选中某客户时，其展示效果如图 5.21 所示。



图 5.21 护理计划列表（管理员与员工视角）

护理计划拥有详情窗口、进度报告窗口、编辑窗口和新增窗口，由于编辑窗口和新增窗口类似，只讨论编辑窗口，窗口中包含的数据有护理计划的各项信息如责任人、开始日期、结束日期、计划相关的目标、计划绑定的预约等，同时对于该窗口的满意度数据只能由客户修改，各窗口效果如图 5.22 所示。管理员与员工视角下的护理计划详情页中还能够对计划目标和计划相关预约进行编辑，能够更新计划目标的名称和目标状态，能够更新预约的服务、预约时间、服务员工、状态等信息，相关窗口如图 5.23 所示。



图 5.22 护理计划相关窗口

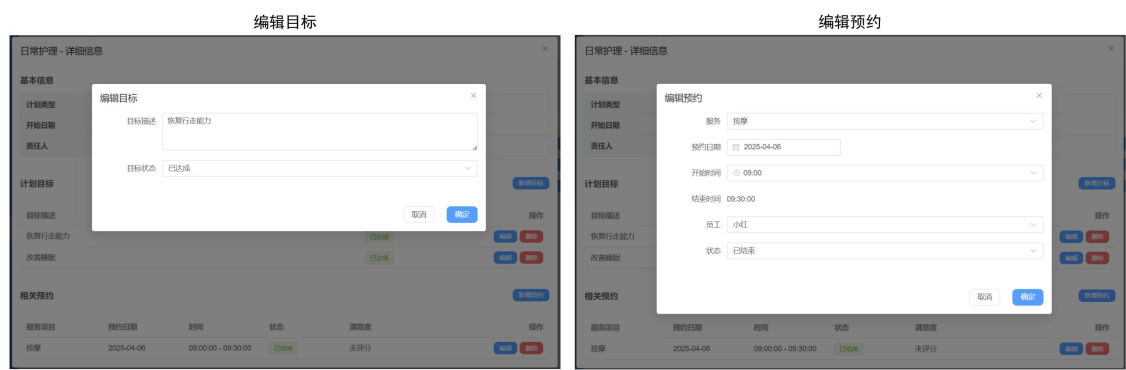


图 5.23 护理计划目标与预约的编辑窗口

5.7 预约服务管理模块

预约服务管理模块的主要功能点见章节 2.6。客户视角和管理员与员工视角的预约模块内容分别如图 5.24 和图 5.25 所示。

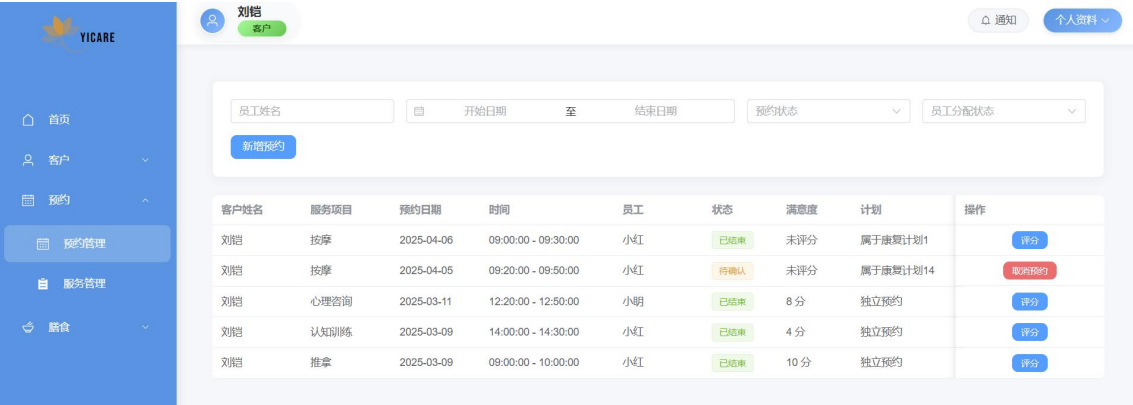


图 5.24 预约模块（客户视角）

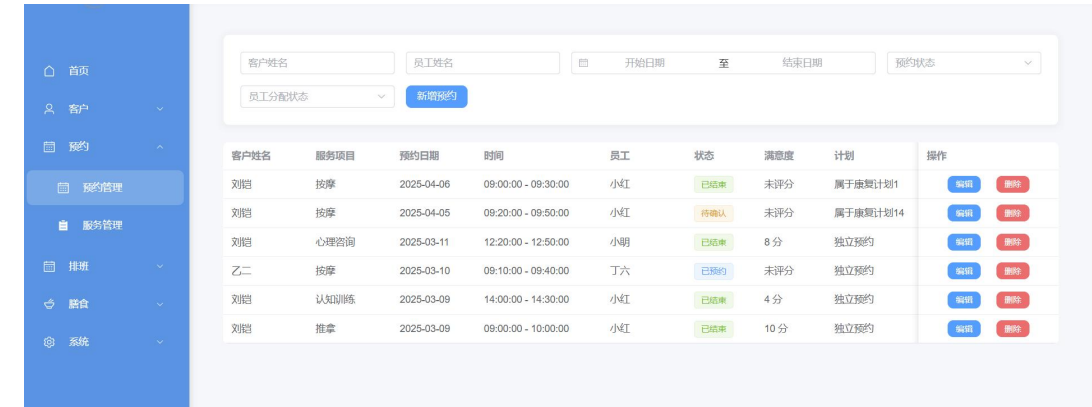


图 5.25 预约模块（管理员和员工视角）

两种视角的区别在于：管理员与员工和客户新增预约时窗口不同；客户能够对“已结束”状态的预约进行评分且能够取消自己的状态为“待确认”和“已预约”的预约，而管理员与员工则拥有所有字段的编辑权限；管理员与员工能够删除预约。由于管理员与员工视角下编辑窗口与新增窗口类似，只展示新增窗口，客户和管理员与员工的新增窗口如图 5.26 所示。



图 5.26 预约模块新增窗口

客户视角和管理员与员工视角的服务模块内容分别如图 5.27 和图 5.28 所示。

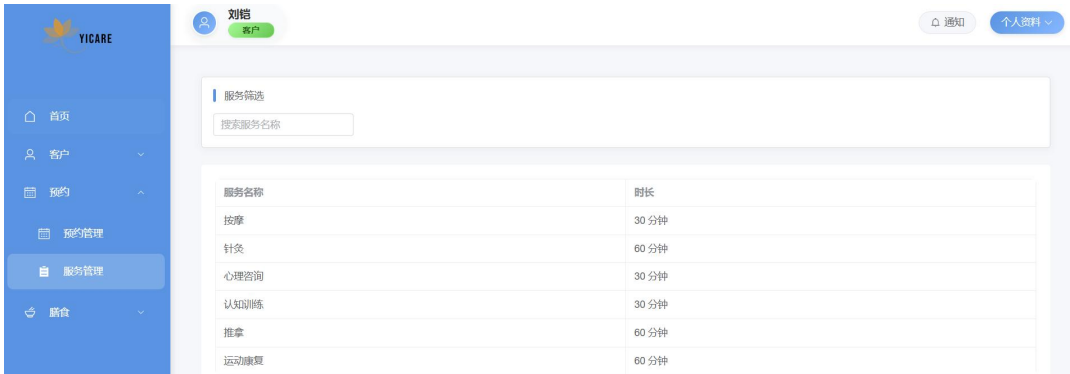


图 5.27 服务模块（客户视角）

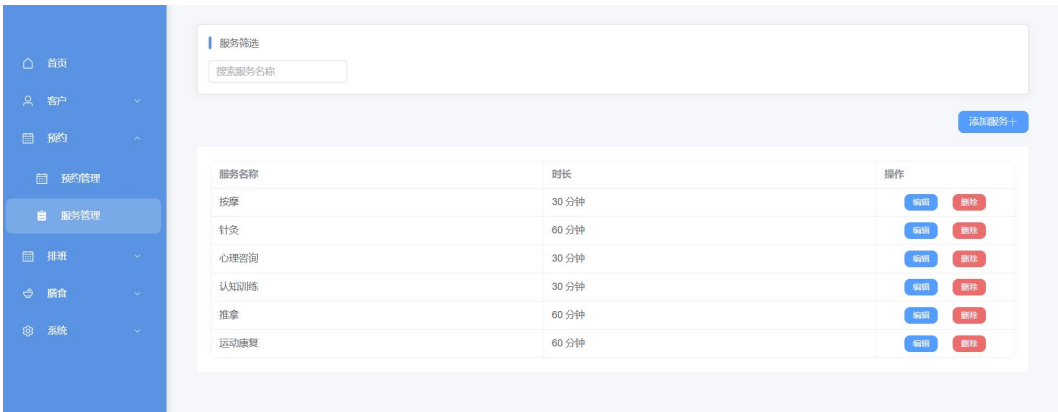


图 5.28 服务模块（管理员与员工视角）

管理员与员工在此模块中拥有增删改的权限，相应功能的窗口如图 5.29 所示。



图 5.29 服务的相关窗口

5.8 排班管理模块

排班管理模块的主要功能点见章节 2.5。客户对该模块没有访问权限。管理员视角和员工视角的排班管理模块内容分别如图 5.30 和图 5.31 所示。

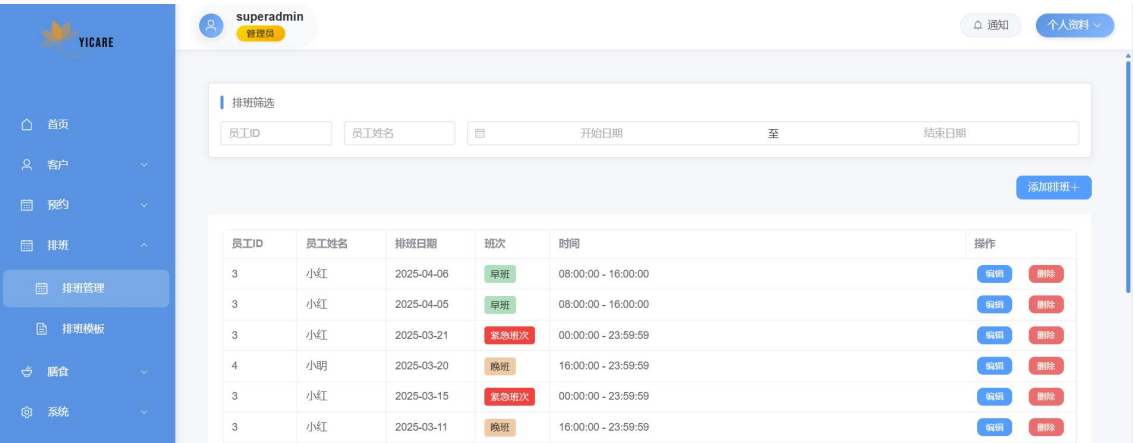


图 5.30 排班管理（管理员视角）

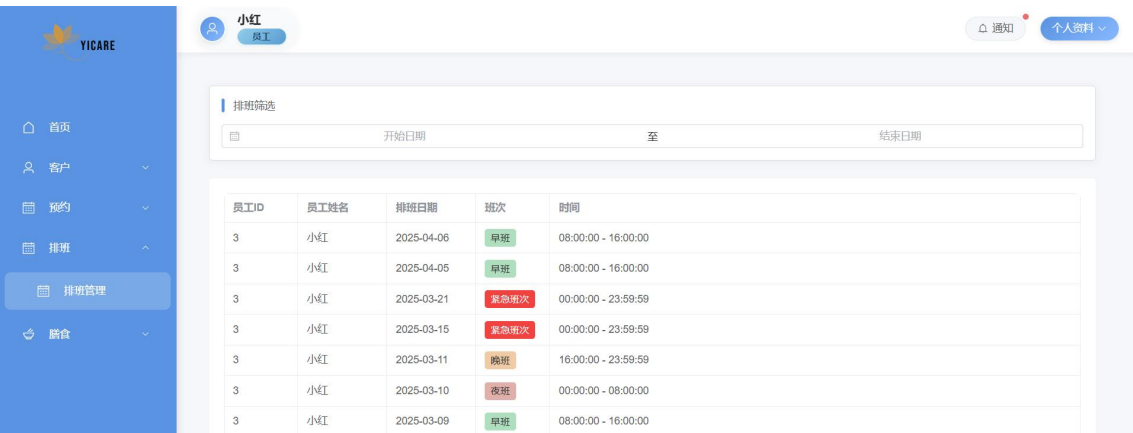


图 5.31 排班管理（员工视角）

管理员在此模块中拥有增删改的权限，在新增和修改时能够选择和编辑员工、排班日期和具体班次等信息，删除时会出现二次确认窗口，相关窗口如图 5.32 所示。管理员还拥有对排班模板模块的增删改查功能，在新增和修改班次时能够编辑班次名称、开始时间、结束时间、颜色编码等信息，在删除班次时会出现二次确认窗口，相关窗口如图 5.33 所示。



图 5.32 排班管理相关窗口

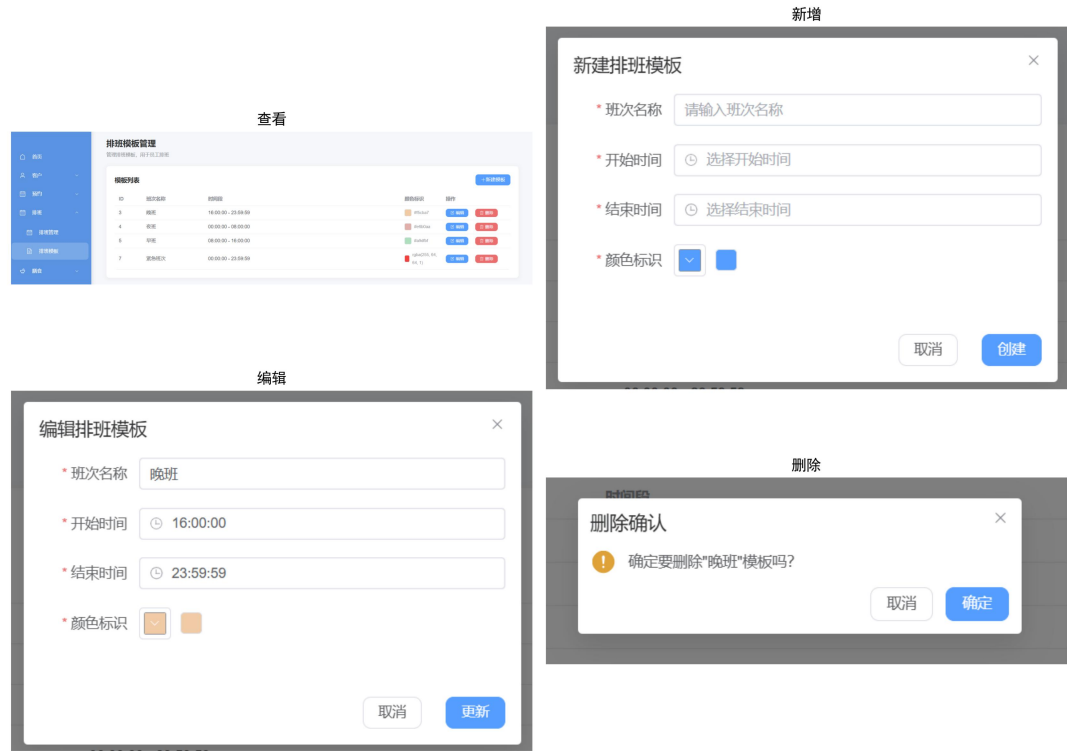


图 5.33 排班模板相关窗口

5.9 膳食计划管理模块

膳食计划管理模块的主要功能点见章节 2.4。客户视角和管理员与员工视角的膳食管理模块内容分别如图 5.34 和图 5.35 所示。

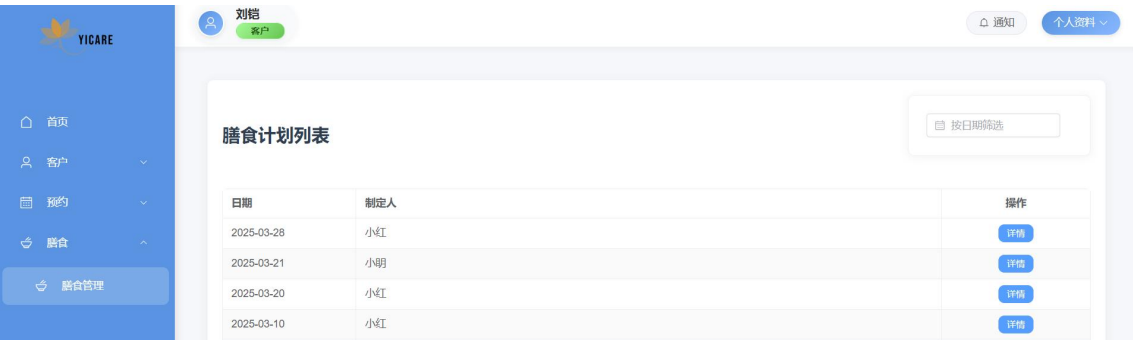


图 5.34 膳食管理（客户）

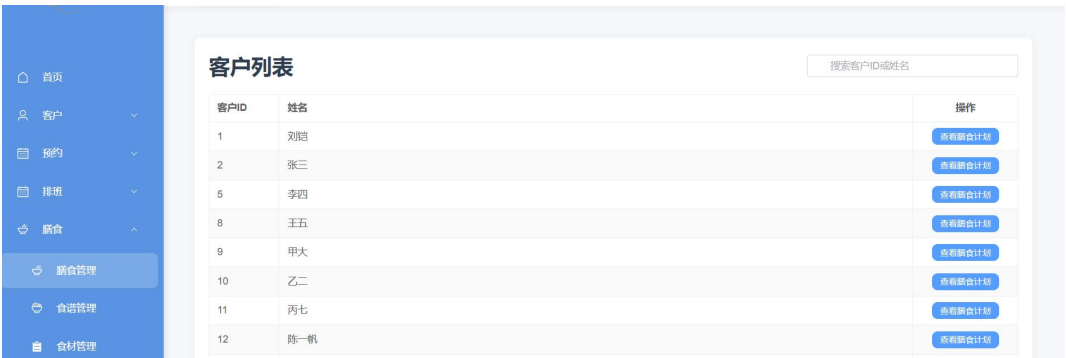


图 5.35 膳食管理（管理员与员工视角）

两种视角的区别在于，管理员与员工能够先查看客户列表再查看某客户的膳食计划并拥有增删改的功能。当管理员与员工点选中某客户时，其展示效果如图 5.36 所示。



图 5.36 膳食计划列表（管理员与员工视角）

客户查看膳食计划详情时与管理员和员工查看内容几乎一致，区别在于客户没有编辑按钮，故在本节中只给出管理员与员工视角下的各相关窗口图，见图 5.37 和图 5.38。



图 5.37 膳食计划详情页



图 5.38 膳食计划编辑与删除窗口

该模块中除了膳食计划外还有两子模块：食谱管理和食材管理，管理员与员工拥有这两个模块的全部权限，分别进入两个模块时其内容如图 5.39 和图 5.40 所示。

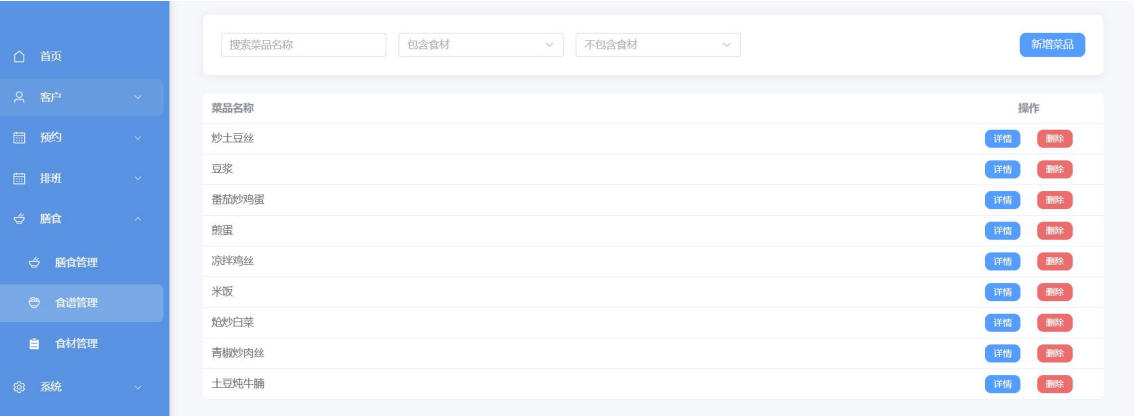


图 5.39 食谱管理

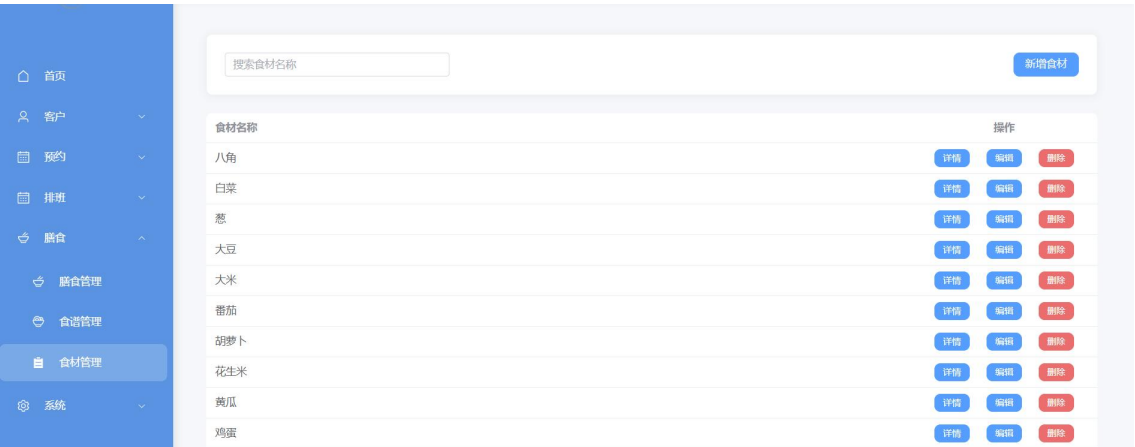


图 5.40 食材管理

这两个子模块中增删改查的相关窗口分别如图 5.41 和图 5.42 所示。在新增和更新食谱时，可以修改食谱名并添加指定重量的食材库中已有食材，在更新和新增保存后，后端会根据食材库中的营养成分信息计算出食谱的营养成分并返回给前端，前端获取数据后会展示在窗口中。在新增和更新食材库时，可以修改食材名称、食材介绍以及某重量的该食材的营养成分列表。在删除某食材或食谱时，都会出现二次确认窗口。如章节 2.4 所述，为简化实现，我们选取的营养素仅包含了热量、脂肪、蛋白质、碳水化合物，在膳食计划模块的营养需求字段、实际营养摄入字段、食谱库的营养成分字段、食材库的营养成分字段中都统一使用了这四种营养成分。



图 5.41 食谱管理相关窗口



图 5.42 食材管理相关窗口

5.10 本章小结

此章依据对需求的剖析以及数据库的设计情况，详尽地讲述了系统各个功能模块具体是怎样设计并得以实现的。其中涵盖了像接口鉴权模块、登录及用户管理模块、首页报表展示模块、客户档案管理模块、健康档案管理模块、护理计划管理模块、预约服务管理模块、排班管理模块还有膳食计划管理模块等诸多模块。对于每一个模块而言，着重对关键功能的设计理念、技术层面的实现方式以及权限的把控情况都做了描述。并且结合 Django 框架所具备的安全机制以及 RBAC 权限模型，让系统在安全性以及功能的完整程度上都有了保障。与此同时，还借助图示的方式，将不同角色所看到的界面以及操作流程展示了出来，进而给中小规模的业务场景送上了一套既高效又可靠的实现办法。

6 智能服务模块研究与实现

本章聚焦于智能服务模块的研究与实现，结合大语言模型与机器学习技术，探索了个性化健康管理及疾病预测的智能化应用。通过集成阿里云百炼平台的通义千问-Max-Latest 模型与基于健康数据的代谢综合征预测模型，系统实现了高度个性化的饮食推荐、健康指标评估以及疾病风险预测功能，为康养中心的中老年用户提供了科学、高效的健康管理服务。

6.1 大模型的应用

本节运用阿里云百炼平台所给予的先进大语言模型——通义千问-Max-Latest（qwen-max-latest），来为用户给予高度贴合个人情况的健康管理服务。凭借它强劲的自然语言理解以及生成能力，系统具备如下能力：其一，能够对复杂且具个性化的需求加以处理，切实将用户诸多方面的信息（像健康数据、个人偏好、过往历史记录等等）进行有效整合；其二，可自动化地给出专业建议，自动生成在营养和健康层面的专业级别的建议，由此缓解了人工咨询所带来的压力；其三，能够提升用户的体验，以自然且人性化的语言去和用户展开交互，给予易于理解的反馈与指导。通义千问-Max-Latest 这一大模型在本项目当中的应用，着实让服务的智能化以及个性化水平有了明显的提升。

模型的集成靠的是调用阿里云百炼平台所给出的兼容 OpenAI 规范的 API 接口来达成的，这就保障了技术对接起来是比较便捷的。具体的代码实现可见附录 A 中图 A.19~A.23。

模型的集成重点体现在两大核心功能方面，其一是个性化饮食推荐功能，其二则是健康指标评估以及给出相关建议的功能。这些应用实实在在地展现出了大语言模型在智慧康养领域以及个性化健康管理领域当中所具备的极大潜力。

6.1.1 个性化饮食推荐

个性化饮食推荐的目标是为康养中心的中老年用户生成符合其个人健康状况、偏好和饮食禁忌的每日膳食计划。

(1) 实现方式

角色设定：通过系统提示词（System Prompt）将模型设定为“营养膳食专家”，使其以专业的口吻和知识库进行回应。

信息整合：函数动态构建用户提示词（User Prompt），整合了用户的多维度信息，包括：a) 近期（1-3 天）的饮食记录，用于避免短期内食物重复；b) 明确的

忌口食材列表；c) 偏好的食材列表；d) 系统提供的可选菜谱库（包含食材与营养成分）；e) 用户的特定营养成分需求比例；f) 用户的基本信息（性别、年龄）和病史。

智能生成：指示模型基于以上信息，生成包含早、中、晚三餐的具体食谱建议。

要求模型：a) 严格遵守用户的饮食禁忌；b) 尽量选用用户偏好的食材和菜谱库中的菜品；c) 避免单日内菜品重复；d) 考虑用户的营养需求（在合理范围内）；e) 对每餐提供人性化的解释；e) 使用中文回答。

(2) 模型作用

在此功能中，大模型扮演了信息处理、约束满足和自然语言生成的角色。它能够理解复杂的输入信息，根据多重约束（如禁忌、偏好、营养需求、避免重复）进行推理和规划，并最终生成一份结构清晰、易于理解的个性化饮食方案。最终生成结果示例如图 6.1 所示。



图 6.1 个性化饮食推荐

6.1.2 健康指标评估与建议

健康指标评估与建议的目标是基于用户的身体指标和基本信息，提供专业的健康状况评估和易于理解的健康建议。

(1) 实现方式

角色设定：通过系统提示词将模型设定为“专精中老年慢性病的专业医生”。

数据输入：函数构建的用户提示词包含了用户的：a) 基本人口学信息（年龄、性别、婚姻状况、收入、种族）；b) 详细的身体检查指标（如 BMI、身高、体重、尿酸、心率、血糖、血压、血脂、腰围等），并附带了正常值范围供模型参考；c) 系统预先计算的代谢综合征患病概率。

智能分析与建议：指示模型基于这些数据，完成以下任务：a) 评估用户的总体健康状况；b) 识别出需要关注或改善的关键指标；c) 提供具体的生活方式建议（如饮食调整、运动建议）；d) 判断是否需要进一步的医疗咨询；e) 要求使

用简洁、易懂的中文进行回答。

(2) 模型作用

在此功能中，大模型的核心作用是解读相对复杂的健康数据，并将其转化为普通用户能够理解的评估和建议。它模拟了医生根据检查报告进行分析和给出初步指导的过程，提高了健康信息的可及性和客户对自身健康状况的认知。最终生成结果示例如图 6.2 所示。



图 6.2 健康指标评估与建议

6.2 代谢综合征预测

康养中心中会定期对客户的身体各项指标进行记录，本节的目的是将这些数据利用起来并提升系统的智能化程度。有研究指出，代谢综合征（MetS）在中国正成为一个日益重要的健康问题，且年龄大于等于 60 岁的人群患病率要显著高于小于 60 岁的人群^[26]。因此本节选取了 MetS 作为疾病预测的对象，尝试将记录在健康档案中的客户数据利用起来，为康养中心智能化提供新的思路。代码可见附录 C。

6.2.1 方法与材料

(1) 数据集

本节使用了来自 Kaggle 的代谢综合征数据集^[27]，该数据集包含 2401 个样本，每个样本有 15 个特征。数据集涵盖了年龄、性别、婚姻状况、收入和种族等人口

统计学特征，以及腰围、体重指数（BMI）、白蛋白尿、尿白蛋白肌酐比、尿酸、血糖、高密度脂蛋白胆固醇（HDL-C）和甘油三酯等临床指标。

类别分布数据集中代谢综合征阴性（0）和阳性（1）样本分布不均衡，阴性样本明显多于阳性样本，如图 6.3 所示。这种类别不平衡在训练过程中通过分层抽样技术得到了有效处理。

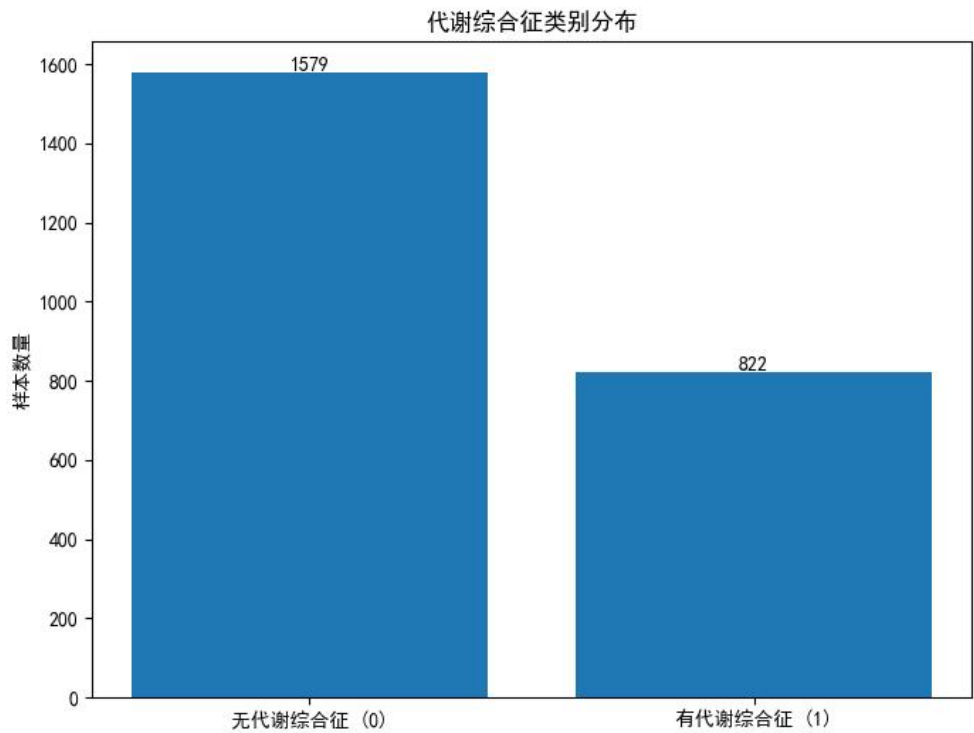


图 6.3 代谢综合征类别分布

(2) 数据预处理

数据集中存在部分缺失值，包括婚姻状况（208 个）、收入（117 个）、腰围（85 个）和 BMI（26 个）。针对数值型特征（腰围、BMI）的缺失，采用中位数进行填充，因为中位数相对于均值对数据中的极端值（异常值）更为稳健，可以提供更可靠的中心趋势估计。考虑到数据集中未注明收入单位且其与代谢综合征的关系可能并非简单的线性关系，为避免引入潜在误差并使模型能更好地捕捉不同收入水平可能存在的模式，将收入特征离散化为低收入（Low）、中等收入（Mid）和高收入（High）三类序数类别。统计显示低收入样本 1281 个，中等收入 467 个，高收入 653 个。对于分类特征如性别、婚姻状况、收入类别和种族，则采用标签编码（Label Encoding）将其转换为数值形式，这是因为大多数机器学习模型要求

输入数据为数值类型。

(3) 特征工程

为探索并可能捕捉到比单一变量更具预测能力的潜在关系或生理指标，本节基于现有特征创建了几个复合特征：a) 腰围与 BMI 的比率（WaistBMIRatio）；b) 血糖与 HDL-C 的比率（GlucoseHDLRatio）；c) 甘油三酯与 HDL-C 的比率（TrigHDLRatio），该比率在临床上被认为是评估心血管风险的一个有用指标^[28]；d) 年龄分组（AgeGroup）。这些工程化的特征旨在增强模型对复杂生物模式的识别能力。

(4) 特征选择

为了降低模型复杂度、减少过拟合风险并提高模型的可解释性，需要进行特征选择。本研究使用随机森林算法来评估各特征（包括原始特征和工程特征）对预测代谢综合征的重要性，因为随机森林在计算特征重要性时能够较好地处理特征间的非线性关系和交互作用。基于算法输出的重要性得分进行排序，并结合领域知识，最终选择了 10 个关键特征进行后续建模：年龄、腰围、BMI、尿白蛋白肌酐比、血糖、HDL-C、甘油三酯以及三个复合特征（WaistBMIRatio、GlucoseHDLRatio、TrigHDLRatio）。选择这些特征旨在保留最具有信息量的预测因子，同时剔除冗余或贡献度较低的特征，以期达到较好的预测性能与模型简洁性之间的平衡。结果如图 6.4 所示。

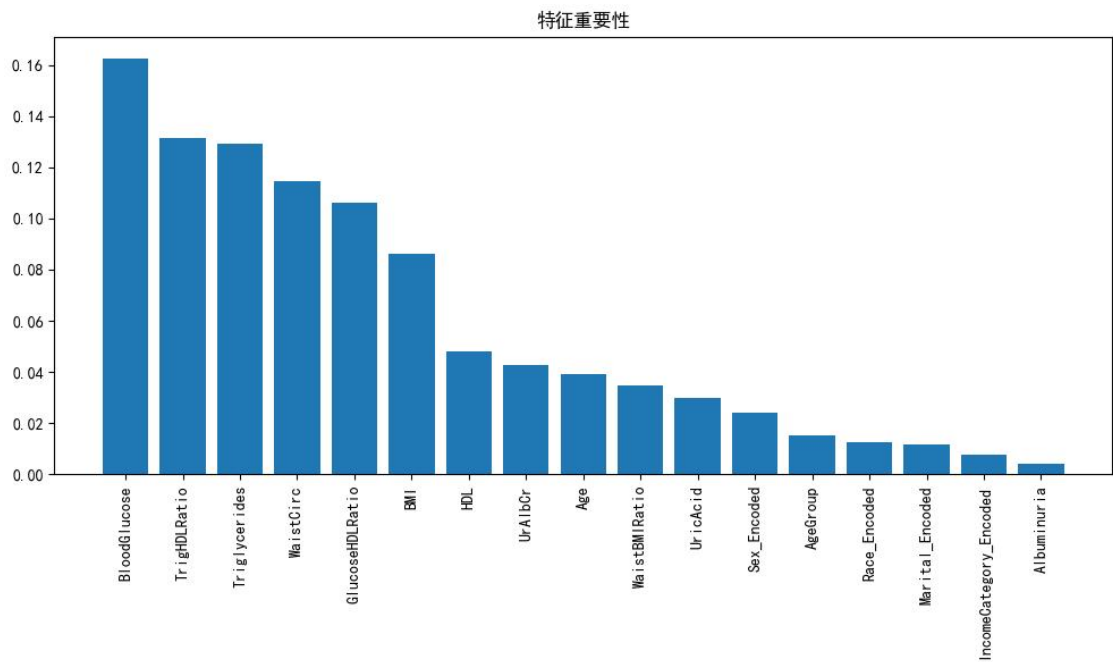


图 6.4 特征重要性

6.2.2 实验结果

(1) 模型训练与评估

数据集以 8:2 的比例随机划分为训练集（1920 样本）和测试集（481 样本），其中训练集用于模型学习，测试集用于评估模型在未见过数据上的泛化能力。为选出最优的基础预测模型，本研究基于 `scikit-learn` 库比较了多种常用的机器学习模型：随机森林、梯度提升、逻辑回归、支持向量机、k 近邻、决策树、朴素贝叶斯、AdaBoost 和 ExtraTrees。采用 5 折交叉验证方法在训练集上对各模型进行评估，目的是获得更稳健、可靠的模型性能估计，减少单次划分可能带来的偏差。交叉验证结果（平均准确率及标准差）如表 6.1 所示。

表 6.1 单一模型的交叉验证评估结果

模型名称	准确率
随机森林	0.8620 ± 0.0260
梯度提升	0.8698 ± 0.0200
逻辑回归	0.8328 ± 0.0230
支持向量机	0.8490 ± 0.0166
k 近邻	0.8458 ± 0.0221
决策树	0.8234 ± 0.0113
朴素贝叶斯	0.8036 ± 0.0194
AdaBoost	0.8667 ± 0.0153
ExtraTrees	0.8536 ± 0.0214

根据交叉验证结果，梯度提升（Gradient Boosting）模型展现出最高的平均准确率（0.8698）和相对较小的波动性（±0.0200），因此被选为最佳单一模型进行后续优化。为了进一步提升模型的预测性能，采用网格搜索（Grid Search）结合 5 折交叉验证的方法对梯度提升模型的关键超参数（如学习率、树的最大深度、叶节点最小样本数、估计器数量等）进行系统性的调优。网格搜索能够穷举指定的参数组合，找到在交叉验证中表现最优的配置，最终获得最佳参数配置为：学习率 0.01，最大深度 3，最小分裂样本数 2，估计器数量 300。优化后的梯度提升模型在交叉验证中的得分提升至 0.8734。

(2) 模型性能

在独立的测试集上，评估了经过超参数优化的最佳单一模型（梯度提升）以及一个简单的集成模型（该集成模型整合了优化后的梯度提升模型和使用默认参数的其它 8 种模型）的性能。评估结果显示，最佳单一模型（梯度提升）的表现优

于本次实验配置的集成模型，结果如表 6.2 所示。这可能是因为优化后的单一梯度提升模型本身性能已足够强大，而简单的集成策略未能带来额外的显著提升，或者默认参数的其他模型表现相对较弱。因此，最终选择优化后的梯度提升模型作为部署模型。

表 6.2 最佳单一模型（梯度提升）与集成模型在测试集的预测准确率

模型名称	准确率
梯度提升	86.90%
集成模型	83.99%

梯度提升模型的混淆矩阵如图 6.5 所示，详细性能指标如表 6.3 所示。

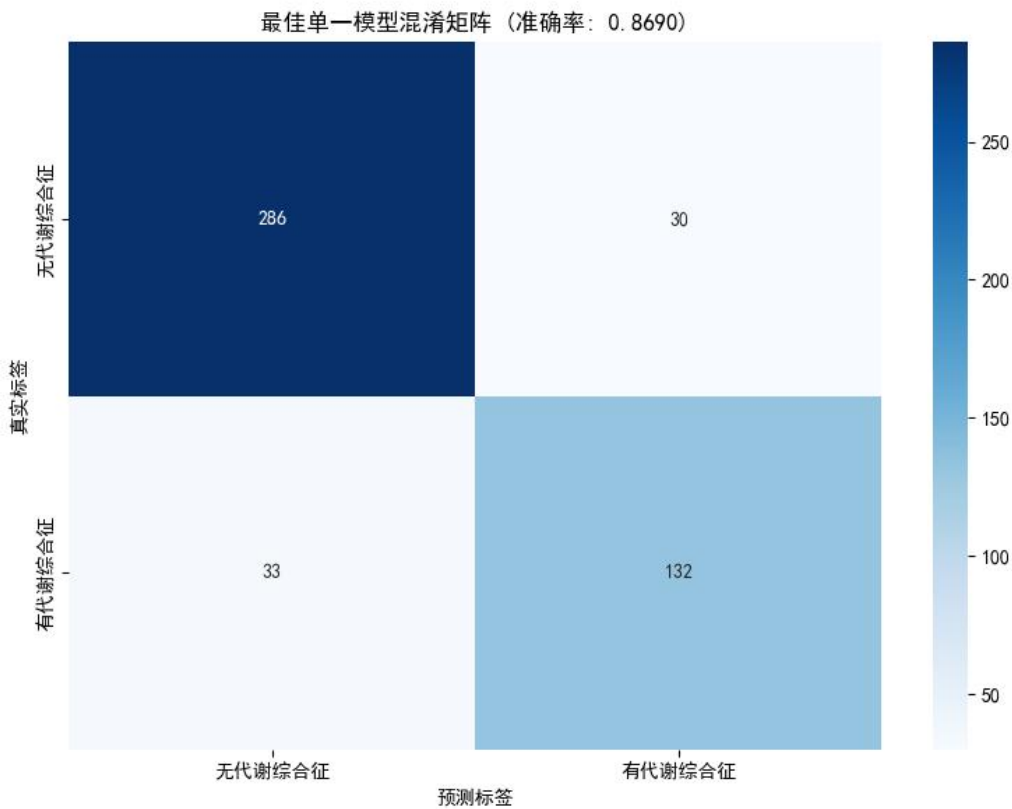


图 6.5 最佳单一模型（梯度提升）混淆矩阵

表 6.3 梯度提升模型的详细性能指标

有无代谢综合征	精确率	召回率	F1 分数
无	0.90	0.91	0.90
有	0.81	0.80	0.81

该模型对无代谢综合征人群的准确识别率（特异性）达 90.5%，对有代谢综合征人群的准确识别率（敏感性）为 80.0%。ROC 曲线分析显示模型具有良好的区分能力，曲线远离对角线，AUC 值较高，证实了模型在不同阈值下均保持较好的分类性能，如图 6.6 所示。曲线精确率-召回率曲线进一步验证了模型在平衡假阳性和假阴性方面的能力，对于临床决策支持系统尤为重要，如图 6.7 所示。

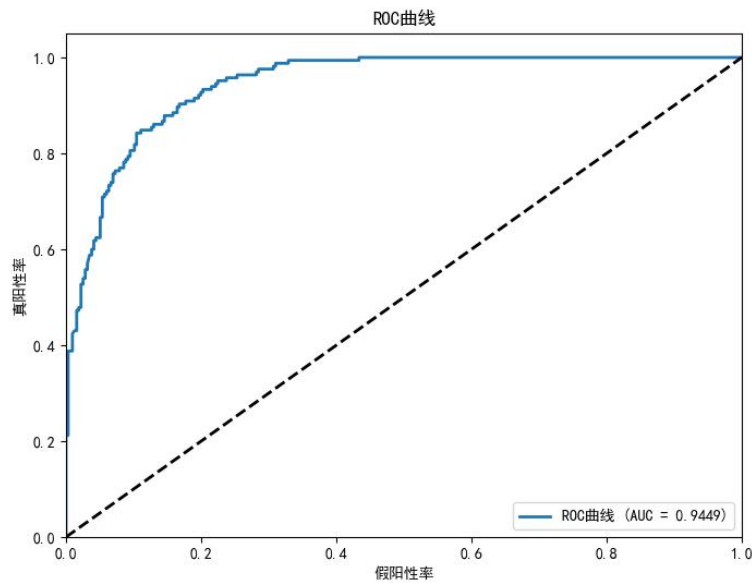


图 6.6 梯度提示模型 ROC 曲线

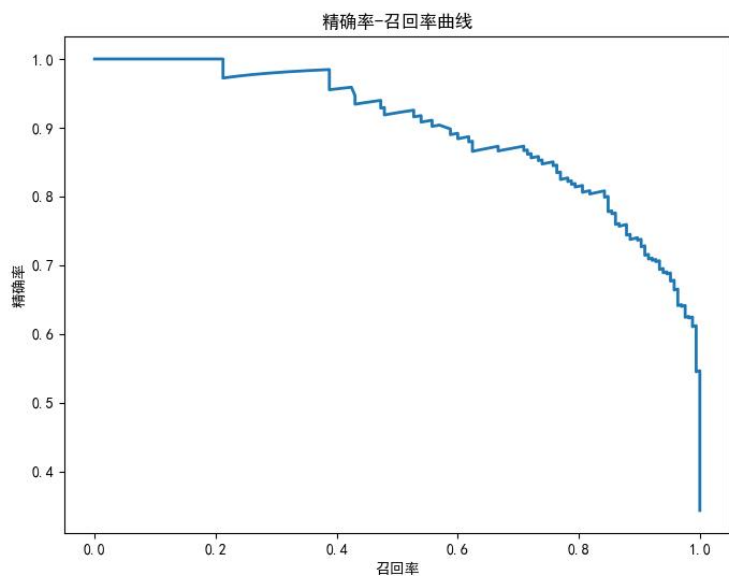


图 6.7 梯度提示模型精确率-召回率曲线

(3) 临床特征分析

图 6.8（特征与目标变量关系箱线图）分析揭示了各临床指标与代谢综合征的关系：a) 腰围：代谢综合征患者腰围显著高于非患者；b) BMI：代谢综合征患者 BMI 明显升高；c) 血糖：代谢综合征患者血糖水平较高；d) HDL-C：代谢综合征患者 HDL-C 水平较低；e) 甘油三酯：代谢综合征患者甘油三酯水平明显高于非患者；f) 年龄：代谢综合征在中老年人群中更为常见。这些发现与代谢综合征的临床诊断标准高度吻合^[29]，证实了现有诊断指标的合理性。

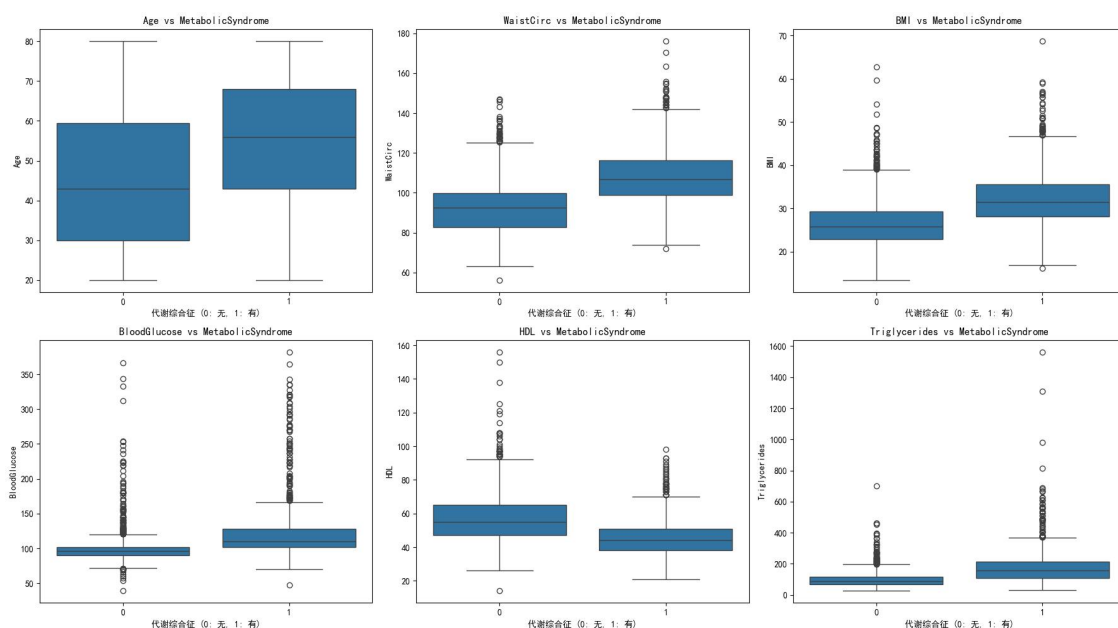


图 6.8 特征与目标变量关系箱线图

(4) 临床意义

基于混淆矩阵（图 6.5）分析，该模型在诊断代谢综合征方面表现出色，假阳性率低（ $30/316=9.5\%$ ），假阴性率也较低（ $33/165=20\%$ ）。这对临床筛查具有重要价值，尤其是模型对无代谢综合征患者的高准确识别能力（91%）可减少不必要的进一步检查，而对有代谢综合征患者的准确识别（80%）也能确保大多数患者得到适当治疗。选定的 10 个重要特征证实了目前临床上用于代谢综合征诊断的指标（如腰围、BMI、血糖、HDL 和甘油三酯）的科学性，同时也表明新创建的复合特征（如 TrigHDLRatio）在疾病风险评估中可能具有额外价值。

(5) 模型部署

模型最终部署在健康档案模块中，当管理员或员工添加或更新客户的健康档案后，后端会将档案中被用于预测的各项身体指标和客户本人的相关数据传入训练好的最终模型（梯度提升）中，预测结果（患 MetS 概率）被填入健康档案表中的

mets_probability。前端效果如图 6.9 所示。

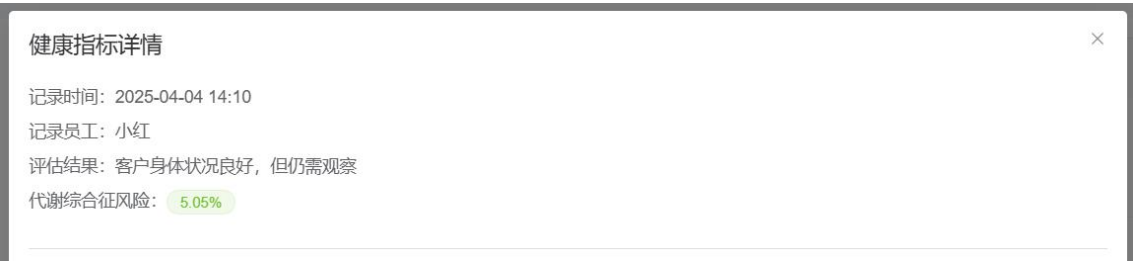


图 6.9 MetS 预测结果前端效果

6.3 本章小结

本章借助通义千问-Max-Latest 大模型达成了个性化饮食推荐以及健康指标评估的目标，凭借其自然语言处理的本事生成专业且易懂的健康方面的建议，使得用户体验以及服务的智能化程度都得到了提升。与此同时，依据代谢综合征数据集来构建梯度提升模型，并且对该模型予以优化，从而实现了具备高准确率的疾病预测功能，此功能顺利部署在了健康档案模块当中，给临床筛查以及健康管理给予了强有力的支撑，进而将智能技术在康养领域所拥有的广阔应用前景展现了出来。

7 系统测试

系统测试在软件开发生命周期里属于极为关键的一个环节，其主要作用在于对已经集成好的系统加以验证，看其是否能够符合所有明确指定的各项需求，这里面既涉及功能方面的需求，也涵盖非功能方面的需求。测试所期望达成的目标是，在软件正式被部署到实际生产环境之前，要能够找出并且修复尽可能多存在的缺陷，以此来切实保障软件具备应有的质量，拥有可靠的性能，进而让用户能够获得较高的满意度^[30]。测试相关的活动一般来说会被划分成不同的阶段，像是在开发流程当中会有单元测试以及集成测试，而等到系统全部完成之后，还会开展系统测试以及验收测试。

在本项目开展开发工作期间，各个功能模块均已经完成了单元测试这一环节。这些测试着重于对模块内部逻辑是不是正确加以关注，至于其测试用例的设计方面，则能够参照第二章需求分析里所涉及的那些功能点。基于此情况，在本章当中，测试的重点会放置在系统层面展开相关测试，尤其是要针对涉及到多个模块相互之间进行交互、复杂业务流程以及非功能性需求的验证这些方面来开展。

应当说明的是，开发以及测试工作全都由同一人来完成，这就很可能存在着一些固有的局限情况，比如说存在着‘确认偏见’这种情况，也就是会倾向按照事先预期好的代码执行路径去开展测试，另外还存在着对自身代码问题方面的‘盲点’。即便如此，本测试章节在设计的时候也会尽量去做到客观且周全，要尽可能把系统的核心功能以及关键业务场景都包含进去。测试工作重点就在于去验证核心功能是不是正确以及是不是稳定，毕竟这些功能是用用户最为常用的部分，而且对系统价值的贡献也是最大的那部分。通过保证核心用例能够得以通过，这样就能在很大程度上躲开那些会影响到用户主要体验的严重问题。对于边缘情况或者非核心功能可能存在的潜在缺陷，虽说也应该给予关注，不过在资源比较有限的情形下，还是要优先确保核心功能的质量才行。

7.1 测试环境

系统采用前后端分离架构，所有组件均在本地开发环境运行，通过环回地址（127.0.0.1）进行跨服务通信。测试也在本地开发环境下进行。

(1) 硬件设备参数

操作系统：Windows 11 64 位

CPU 型号：Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz 2.59 GHz

内存：16.0 GB (15.9 GB 可用)

(2) 各服务组件配置

前端服务：

- 开发框架：Vue 3.5.13 + Vite 6.2.0
- 运行模式：开发调试模式（vite dev server）
- 服务地址：127.0.0.1:5173 (HTTP 协议)
- 浏览器支持：Microsoft Edge 135.0.3179.54 (正式版本) (64 位)

后端服务：

- Web 框架：Django 5.1.6（运行于开发服务器）
- 协议接口：RESTful API @ 127.0.0.1:8000
- Python 环境：3.10

数据库服务：

- 主存储：MySQL Community Server 8.1
- 连接参数：127.0.0.1:3306（TCP/IP 协议）

缓存服务：Redis 5.0.14.1（单节点部署）

- 端口：127.0.0.1:6379
- 用途：缓存常用视图；作为 Celery 的 broker

7.2 功能测试

测试过程中，选择某用例并执行其步骤，每一步都应有其对应的预期结果，将实际结果与预期结果相比较，判断用例实际结果是否正确。系统部分测试用例及测试结果如表 7.1 所示。

表 7.1 系统部分测试用例及测试结果

序号	测试涉及模块	测试用例	预期结果	测试结果
1	排班模块 + 预约模块	1、管理员登录，为员工 a 添加排班 2、为客户 b 添加员工 a 排班期间的预约，点击员工列表 3、选择员工 a 并确认	1、登录成功；添加成功 2、能够查看到员工 a 出现在可选员工列表 3、预约添加成功	通过

续表 7.1 系统部分测试用例及测试结果

序号	测试涉及模块	测试用例	预期结果	测试结果
2	排班模块 + 预约模块	1、管理员登录，为员工 a 添加排班 2、给员工 a 添加一个排班期间的预约 3、给客户 b 添加预约，时段为员工 a 已有预约的时段，点击员工列表	1、登录成功；添加成功 2、添加成功 3、无法查看到该员工 a	通过
3	预约模块	1、客户登录，为自己添加某时段预约 2、再次添加相同时段预约	1、登录成功，添加成功 2、添加失败，提示“客户在该时间段已有预约”	通过
4	食材管理 + 食谱管理	1、管理员登录，为食材库添加某食材 2、在食谱管理中添加某食谱，查看该食材是否出现在食材选择菜单中 3、选择该食材，并确认 4、查看该食谱	1、登录成功，添加成功 2、能够查看到该食材 3、添加成功 4、食谱中关于食材的信息正确显示，营养素计算正确	通过
5	食谱管理 + 膳食管理	1、管理员登录，为食谱库添加任意食谱 2、在膳食管理中为任一客户添加膳食计划 3、在膳食计划详情中往某餐中添加该食谱，点击确认 4、查看计划详情	1、登录成功，添加成功 2、添加成功 3、该食谱出现在可选项中，添加成功 4、膳食计划详情页显示正确，营养素计算正确。	通过

续表 7.1 系统部分测试用例及测试结果

序号	测试涉及模块	测试用例	预期结果	测试结果
6	客户档案 + 膳食管理	1、管理员登录，在客户档案中为任一客户添加其忌口 2、在膳食管理中为该客户添加膳食计划 3、在膳食计划详情页中点击生成智能推荐按钮	1、登录成功，添加成功 2、添加成功 3、生成成功，并且生成的食谱都避免了客户忌口，且选用了食谱库中食谱	通过
7	预约管理 + 排班模块	1、管理员登录，为员工 a 添加排班 2、为客户 b 添加员工 a 排班时段中预约并指定员工 a 3、删除员工 a 排班 4、查看客户 b 预约	1、登录成功，添加成功 2、添加成功 3、删除成功 4、客户 b 的预约中指定的员工显示为空	通过
8	首页报表 + 健康档案	1、管理员登录，进入健康档案模块，选择客户 a 2、新增客户 a 的健康档案 3、在新增健康档案的窗口中填写身体指标等字段并保存 4、进入首页，在客户列表中找到客户 a 并点击查看数据 5、查看客户 a 的身体数据曲线图	1、登录成功，成功进入健康档案模块，能够查看客户 a 的健康档案 2、弹出新增健康档案窗口 3、填写成功，保存成功 4、成功进入首页，找到客户 a 并成功进入客户 a 的身体指标数据界面 5、能够查看到刚新增的身体指标数据	通过

7.3 性能测试

性能测试的目的在于对系统在不同负载情形下的响应能力、稳定性以及资源利用率展开评估，以此来查验其是否能够契合预定好的性能方面的需求（可参看 2.9.1 节内容），同时还能从中找出潜在存在的性能瓶颈之处。在本研究当中，选取了 Apache JMeter 当作性能测试所运用的工具，原因在于它属于一款具备开源性质、功能颇为强大并且被广泛加以运用的负载测试工具。此次所开展的性能测试大体上涵盖了负载测试（Load Testing）以及压力测试（Stress Testing）这两个不同的部分。

7.3.1 用户数与并发量估算

要设计出合理的性能测试场景，首先就得对系统预期的用户量以及并发访问量予以估算。该系统主要是为康养中心提供服务的，其用户群体涵盖了内部员工以及外部客户。按照一个中型康养中心的规模来做合理的假设：

员工用户的数量情况是这样的，大致估算下来，约有 50 名员工存在使用系统的需求，他们会借助系统来开展日常管理方面的工作，同时也会利用系统完成服务安排之类的相关操作。

客户方面的用户数量情况是这样的：假定中心所提供的服务面向大约 200 名客户，这些客户有可能借助系统来查询相关的信息，也有可能通过系统去预约各类服务，或者还能在系统里查看自身的健康报告。

其注册用户的总数达到了 250 名，这其中包含了 50 名员工以及 200 名客户。

预估峰值并发用户的数量时，要清楚并非所有注册用户都会在同一时间处于在线且活跃的状态。依据过往经验以及相关推测来看，在业务高峰时段，像是日间工作时间、探视高峰期这类时间段，大概会有百分之二十的注册用户能够同时活跃在线上。基于这样的情况，预估系统在高峰时段得要能够支持差不多 50 个并发用户进行访问才行。负载测试将基于此并发量进行设计。

7.3.2 负载测试

(1) 测试目标

模拟系统在预估的正常高峰负载（50 并发用户）下的表现，检查系统的响应时间、吞吐量和资源利用率是否达到预期标准，验证系统能否稳定处理正常业务压力。

(2) 测试配置

线程组（模拟用户数）：50 个线程（虚拟用户）。

Ramp-up Period（启动时长）：300 秒（5 分钟）。设置为较长的 Ramp-up 时间是为了模拟用户逐步登录系统的过程，避免瞬时压力过大导致结果失真，并观察系统在负载逐渐增加过程中的表现。

Loop Count / Duration（持续时间）：测试持续运行 1 小时。选择足够长的持续时间是為了观察系统在稳定负载下的长期运行表现，检测是否存在内存泄漏、资源耗尽等随时间累积的问题。

测试场景（业务流程模拟）：测试脚本模拟典型的用戶交互流程，主要包含以下步骤：a) 用戶登录：每个虚拟用戶首先执行一次登录操作，获取认证凭证（cookie），此操作仅在线程启动时执行一次；b) 查询客戶信息：模拟用戶查看自己的基本资料；c) 查询服务：模拟用戶浏览或搜索可用的康养服务项目。

思考时间（Think Time）：在每个请求之间添加 2 到 5 秒的随机暂停时间。这用于模拟真实用戶在操作之间的思考或阅读时间，使测试负载更接近实际情况。JMeter 中可以通过统一随机定时器实现。

(3) 预期结果与监控指标

平均响应时间（Average Response Time）：针对核心业务请求（登录、查询信息、查询服务），平均响应时间应低于 2 秒。

吞吐量（Throughput）：系统应能维持一个相对稳定的事务处理速率（TPS），无明显下降趋势。具体数值将在测试中记录。

错误率（Error Rate）：测试期间不应出现或仅出现极少量的请求错误。

服务器资源利用率：CPU、内存等关键资源的使用率应保持在健康水平，即 CPU < 70%，内存 < 70%。

测试结果记录：

表 7.2 负载测试汇总报告										
标签	样本	平均值	最小值	最大值	标准偏差	异常%	吞吐量	接收速率 KB/sec	发送速率 KB/sec	平均字节数
登录	50	474	455	499	10.92	0.000%	0.16967/s ec	0.12	0.04	734.0
查询 客戶	245 10	7	4	249	3.57	0.000%	6.82253/s ec	4.79	1.73	719.0
查询 服务	244 83	6	4	133	3.04	0.000%	6.82264/s ec	4.51	1.72	677.0
总体	490 43	7	4	499	15.28	0.000%	13.63363/ sec	9.29	3.44	698.0

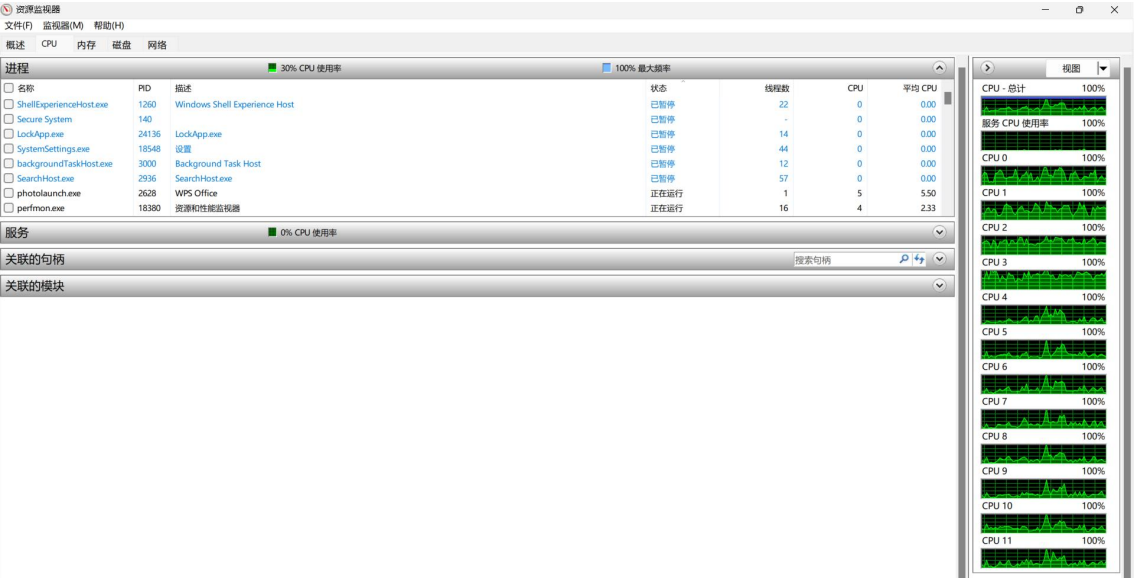


图 7.1 负载测试 CPU 占用率

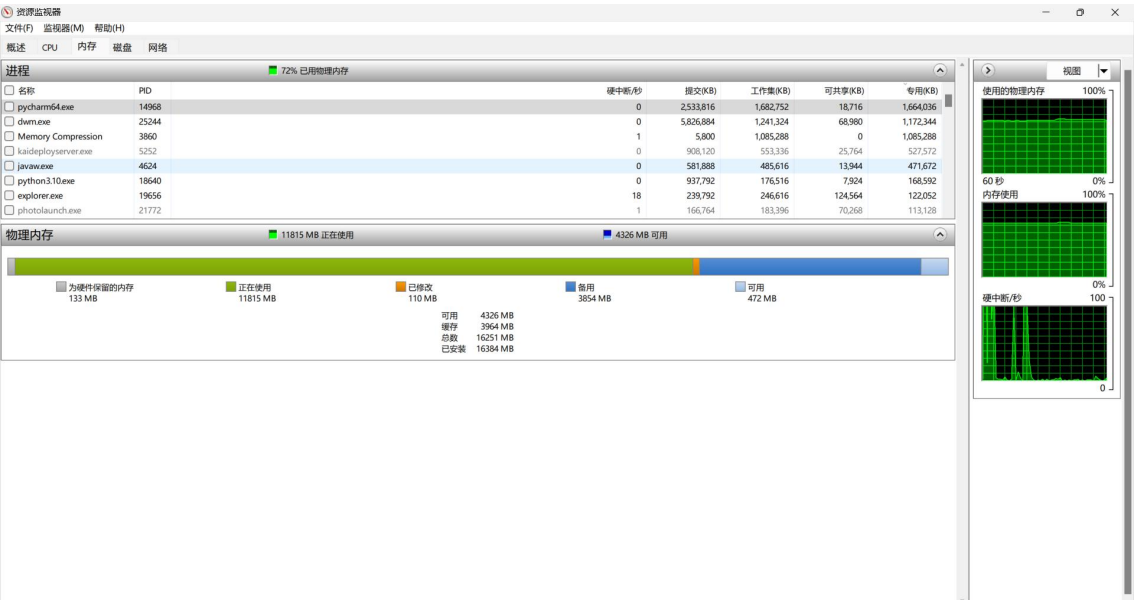


图 7.2 负载测试内存使用率

由表 7.2 可得在并发用户数为 50，运行 1 小时的情况下，总体平均响应时间为 7ms，未发生错误，平均吞吐率约为 13 个请求每秒，符合预期的性能指标要求。

图 7.1 为服务器在负载测试中某刻的 CPU 占用率（30%）。负载测试全程的服务器 CPU 占用率始终维持在一个较低的水平，未超过 70%，符合预期。

图 7.2 为服务器在负载测试中某刻的内存使用率（72%）。虽超过了 70%，但在负载测试开始前内存使用率就已经达到了 70%。在负载测试过程中，服务器内存使用率未发生较大波动，表明并未消耗过多内存，不存在内存泄漏的情况，符

合预期。

7.3.3 压力测试

(1) 测试目标

通过逐步增加并发用户数，超出预期的正常负载，目的是找到系统的性能拐点（即性能开始显著下降的点）和最终的崩溃点（系统无法响应或错误率急剧上升的点），识别系统处理能力的上限和潜在瓶颈所在。

(2) 测试配置

线程组：采用逐步加压的方式，从 50 个并发用户开始，每 30 秒增加 50 个用户，直至达到 500 个用户。

Ramp-up Period：每个增加 50 用户的阶段内，Ramp-up 时间设为 30 秒，使得用户平稳增加。

Duration：每个并发用户数级别（50~450）稳定运行 30 秒，以观察该负载下的性能表现。

峰值持续与卸载：当达到 500 并发用户数的峰值后，持续运行 60 秒，以观察系统在极限负载下的稳定性。之后，模拟负载下降，每 1 秒停止 5 个线程，直至所有线程结束。

测试场景：与负载测试场景类似，模拟用户登录、查询信息、查询服务等操作，并包含思考时间。

(3) 预期结果与监控指标

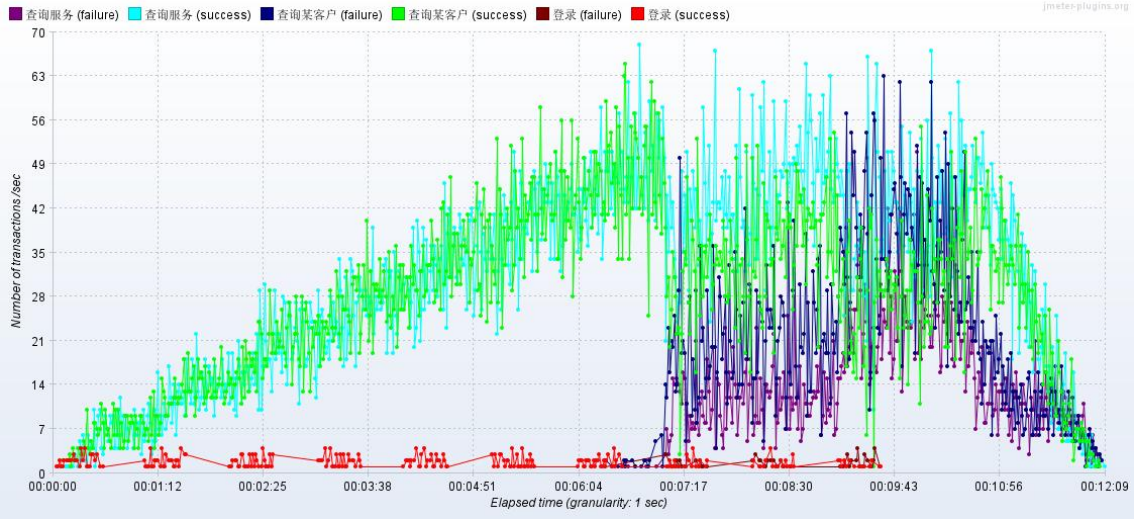
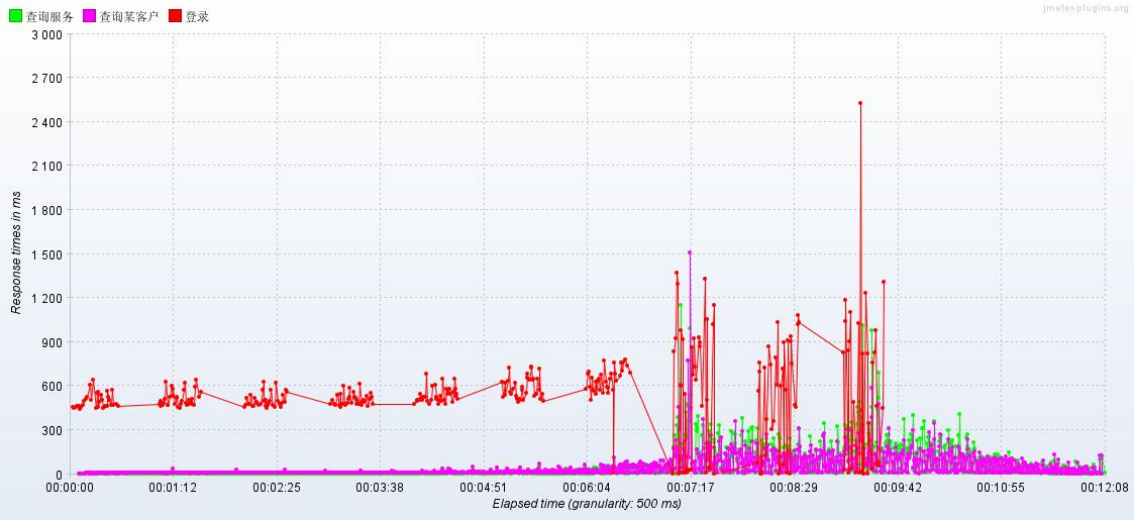
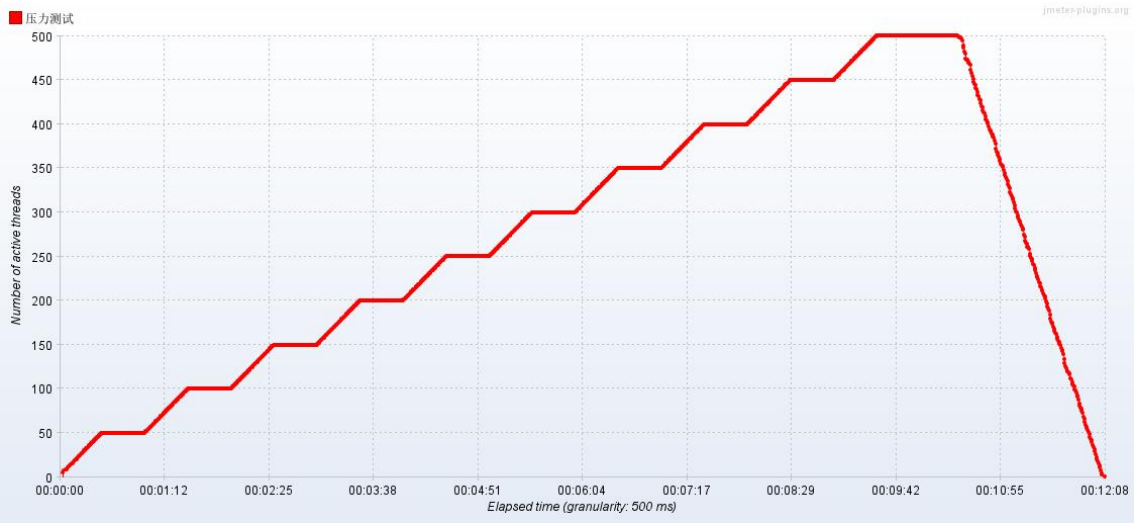
性能拐点：响应时间开始急剧增加或吞吐量开始显著下降时的并发用户数。

系统极限：系统开始出现大量错误、响应时间普遍超过可接受阈值或服务器资源达到饱和时的并发用户数。

瓶颈分析：结合测试过程中的响应时间、吞吐量、错误率以及服务器资源监控数据，分析导致性能下降或系统崩溃的瓶颈。

测试结果记录：结合图 7.3~图 7.9 可知，当并发用户数超过 300 时开始出现请求错误；当并发用户数达到 400 时，系统性能急剧下降，出现大量请求错误，响应时间快速增长；当并发用户数达到 500 时，系统的 CPU 占用率达到了 87%，远超 70%，近乎达到上限。由此可知 300 并发用户数为当前系统的性能拐点，500 并发用户数已达到系统极限。

通过分析服务器资源使用率、出现错误的请求响应和服务器的错误日志，发现本系统性能瓶颈的核心在于 MySQL 连接池耗尽 + Django 同步处理模型限制，高并发下两者的叠加导致请求拒绝和 HTTP 状态码 500 错误。通过增加 MySQL 连接数、启用连接池、优化并发配置及缓存高频查询，可显著提升系统吞吐量。长期来看可以向异步架构演进，并引入负载均衡实现水平扩展。



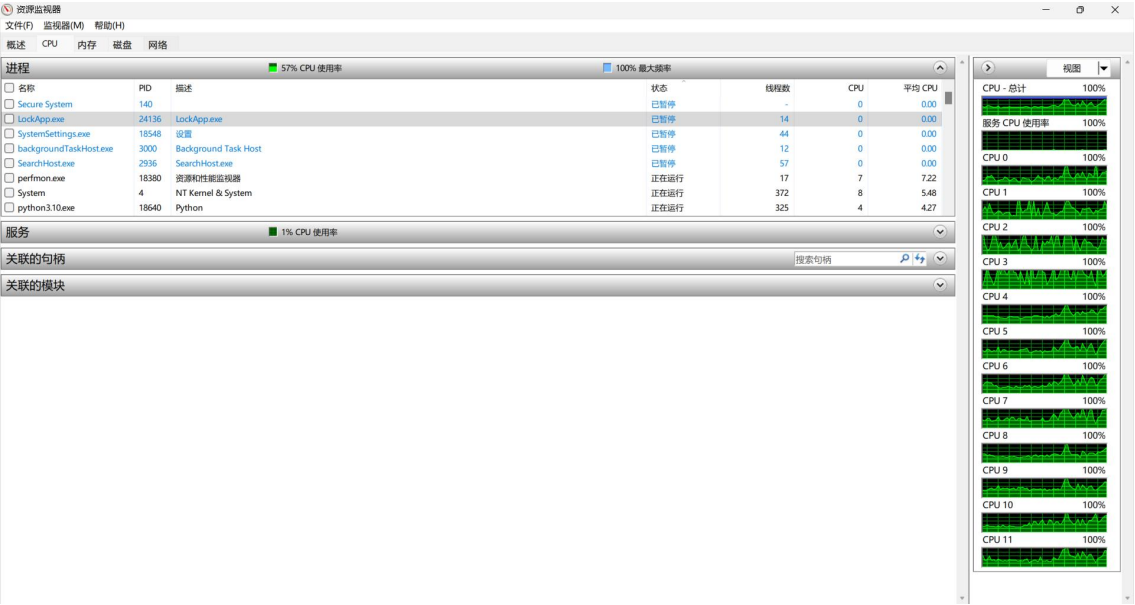


图 7.6 并发用户数为 300 时的 CPU 占用率

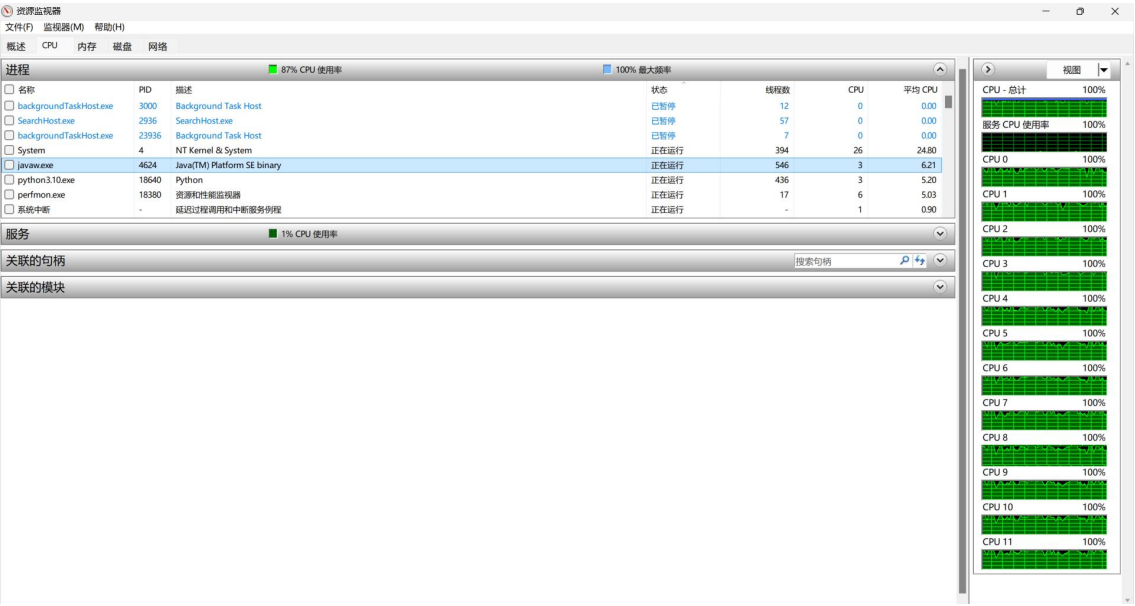


图 7.7 并发用户数为 500 时的 CPU 占用率

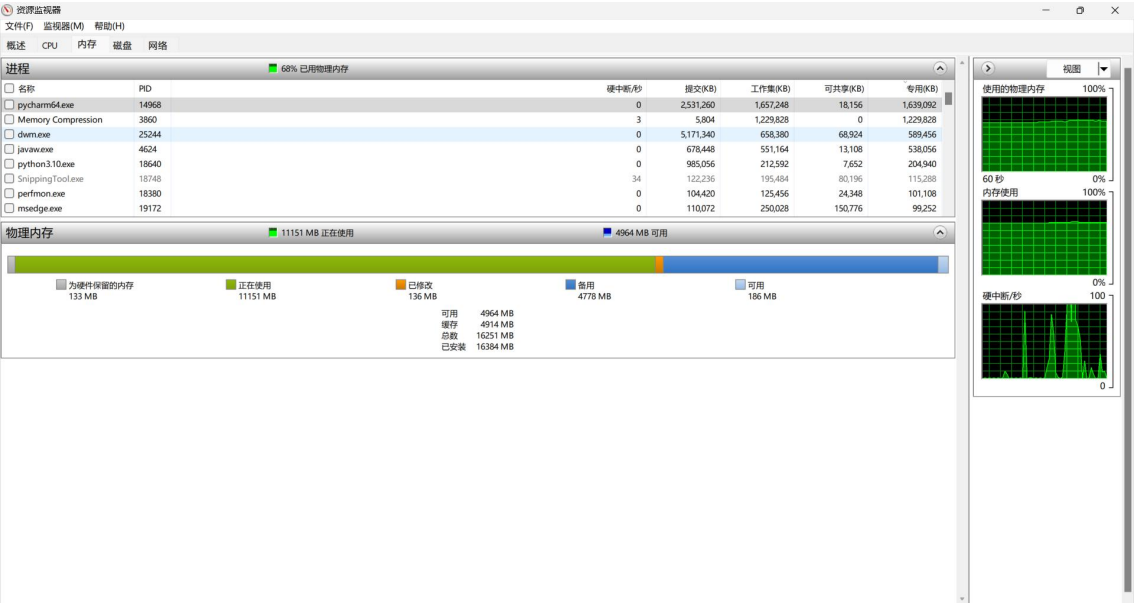


图 7.8 并发用户数为 300 时的内存使用率

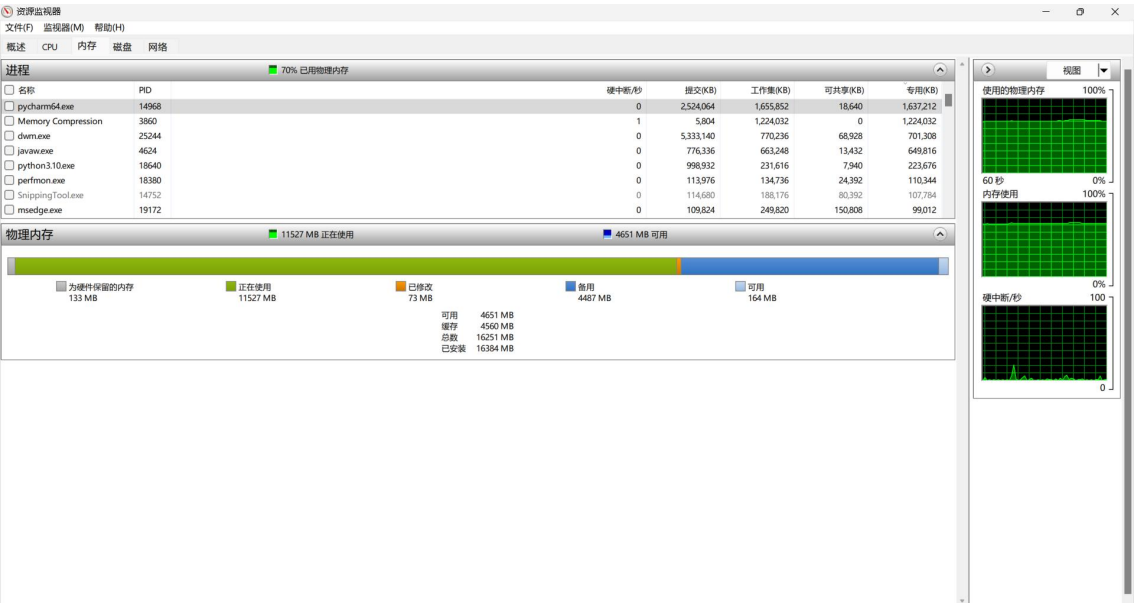


图 7.9 并发用户数为 500 时的内存使用率

7.4 本章小结

此章对系统测试的整个过程以及最终结果都作了详尽阐述，其中涉及到测试环境的搭建工作，还有功能测试以及性能测试等方面内容。测试环境是依照本地前后端分离架构来设定的，在这当中，对于硬件配置以及服务组件配置都予以了明确。功能测试借助多模块交互用例对核心功能的正确与否进行了验证，而且所有的测试用例都顺利通过了，这也就为系统的稳定性提供了有力保障。性能测试方

面是运用 JMeter 来开展负载测试以及压力测试的，测试所得结果显示，在有 50 个并发用户的情况下，系统的平均响应时间为 7ms，其运行状态十分稳定；而通过压力测试能够看出，当并发用户达到 300 个的时候就成了性能的拐点，之所以出现这种情况，其瓶颈在于 MySQL 连接池被耗尽以及 Django 同步处理存在限制。至于优化方面的相关建议，包含了增加连接数、对缓存进行优化以及去探索异步架构等内容，这些都为系统在后续的改进工作指明了方向。

8 总结与展望

8.1 工作总结

本文聚焦于当前我国人口老龄化背景下康养服务面临的实际挑战，设计并实现了一套基于现代 Web 技术的智慧康养信息管理系统。主要工作总结如下：

(1) 系统化需求分析：深入分析了康养机构的核心业务流程，包括客户管理、健康监测、膳食营养、护理计划、服务预约等，并结合管理员、员工、客户三类用户的视角，梳理了详细的功能需求和非功能需求，为系统设计奠定了坚实基础。

(2) 现代化技术架构选型与设计：采用了前后端分离的 B/S 架构模式，前端选用 Vue 3 生态，后端基于 Python 的 Django 框架，数据库采用 MySQL 结合 Redis 缓存，构建了一个技术先进、易于维护和扩展的系统平台。详细阐述了技术选型的依据和各层级的设计思路。

(3) 精细化数据库设计：遵循数据库设计范式，结合康养业务数据的特点，设计了涵盖用户、客户、员工、健康档案、护理计划、膳食、预约、排班等核心实体的数据库模型，并通过 Django ORM 实现了数据库的物理结构。

(4) 核心功能模块实现：重点实现了客户信息管理、护理计划、健康档案、膳食计划、排班、预约、通知等七大核心模块的关键功能，并利用 ECharts 实现了健康数据的可视化展示，提升了信息的可读性和决策支持能力。

(5) 智能化服务集成与探索：

大语言模型应用：创新性地接入了阿里云通义千问大模型，实现了个性化饮食推荐和健康指标智能解读两大功能，探索了 LLM 在提升康养服务智能化、个性化和人性化方面的潜力。

疾病风险预测：利用公开数据集训练并评估了多种机器学习模型，最终选用优化后的梯度提升模型实现了代谢综合征的预测功能（准确率 86.90%），为客户提供了主动健康风险预警。

(6) 系统化测试与评估：设计并执行了功能测试和性能测试。功能测试验证了核心业务流程的正确性。性能测试评估了系统在不同负载下的表现，结果显示系统在预期并发下性能良好，并识别出约 300 并发用户为性能拐点，分析了潜在瓶颈。

总体而言，本系统成功整合了康养机构的核心业务管理功能，并通过引入 AI 技术提升了服务的智能化水平，为解决当前康养服务面临的痛点问题提供了一套可行的技术方案。

8.2 不足与展望

尽管本系统已基本实现预定目标，但仍存在一些局限性和未来可改进的方向：

(1) 关于数据来源与模型的泛化能力：代谢综合征预测模型是依靠公开数据集来开展训练工作的。不过该模型在针对真实康养中心客户数据进行应用的时候，其表现情况还需要再进一步地去加以验证，同时也需要做出相应的调优处理。在后续的发展进程中，可以着手去收集相关机构内部经过脱敏处理的数据，以此来构建出一个能够更加契合实际业务场景的预测模型，而且还可以将其进一步拓展延伸，使其能够应用到对其他一些老年常见疾病的风险预测工作中去。

(2) 智能化服务所达到的程度：当下，大模型的应用大多聚焦在推荐以及解读这两块内容上。往后可以去进一步探寻更深层次的集成化应用方式。比如说，可以把客户的健康数据同护理计划相互结合起来，从而自动生成更为精细、精准的康复方面的建议或者护理过程中的要点内容；也可以借助 LLM 来辅助员工完成文档记录以及报告撰写的相关工作；还能够开发那种基于自然语言展开交互的智能客服或者健康助手。

(3) 物联网（IoT）设备的集成：当下系统在很大程度上主要依靠手动录入健康数据。往后可以去斟酌怎样集成诸如智能手环、血压计、血糖仪这类可穿戴或者家用的健康监测设备，以此达成健康数据能够自动且实时地被采集以及上传的效果，进而构建起更为完备的动态健康监测体系。

(4) 移动端支持：当下的系统仅仅能够提供 Web 端的访问途径。鉴于员工存在移动办公的需求，同时客户也有随时查看信息的需要，在未来可以着手去开发移动应用程序，像 APP 或者小程序都可以，以此来给予更为便利的访问感受。

(5) 系统性能优化及部署：性能测试显示，当下系统在面临较高并发状况时存在着瓶颈问题。要是未来想要能够支持规模更大的用户群体，那就得针对这一情况开展相应的性能优化工作（比如对数据库加以优化、对异步处理进行改造、引入像 Nginx + uWSGI/Gunicorn 这类更为专业的 Web 服务器等），与此同时，还需要考虑运用负载均衡、将数据库读写分离等分布式部署的方案。

(6) 安全性增强：虽说已经落实了基础的安全举措，不过考虑到医疗健康数据极为敏感这一特性，在未来，于数据加密、访问审计以及安全防护等多个方面，还能够进一步去做更为稳固的强化处理。

展望未来，智慧康养信息系统将朝着更智能、更互联、更个性化的方向发展。通过持续的技术迭代和功能完善，本系统有望为提升我国康养服务水平、应对人口老龄化挑战贡献一份力量。

参 考 文 献

- [1] 乔晓春. 中国人口老龄化的过去、现在和未来[J]. 社会政策研究, 2024(01): 47-63. DOI: 10.19506/j.cnki.cn10-1428/d.2024.01.001.
- [2] 清华大学互联网产业研究院. 智慧养老产业白皮书（2019）[R]. 北京: 清华大学互联网产业研究院, 2019.
- [3] 潘晔, 刘志辉, 胡倩倩, 等. 中国老年人慢性病多病共存模式的研究 [J]. 中国全科医学, 2023, 26(29): 3608-3615. DOI: 10.12114/j.issn.1007-9572.2023.0186.
- [4] 工业和信息化部, 民政部, 国家卫生健康委. 智慧健康养老产业发展行动计划（2021-2025年）[R]. 工信部联电子（2021）154号, 2021-10-20.
- [5] 雷晓康, 汪静. 健康中国背景下的智慧健康养老: 战略目标、体系构建与实现路径[J]. 西 北 大 学 学 报 （ 哲 学 社 会 科 学 版 ） , 2020, 50(1):131-139.DOI:10.16152/j.cnki.xdxbsk.2020-01-013.
- [6] 郭佳丽. H 养老院医养信息系统的设计与实现[D]. 北京: 首都经济贸易大学, 2022. DOI: 10.27338/d.cnki.gsjmu.2022.001429.
- [7] 李大伟. 老年人康养庄园智慧管理系统研究[J]. 现代信息科技, 2019, 3(1): 179-181.
- [8] 张俊萌, 梁志达. 基于 ECharts 的康养数据可视化系统的设计与实现[J]. 工业控制计算机, 2024, 37(11): 155-156.
- [9] 罗东霞. 面向老年痴呆的智慧康养平台的设计与实现[D]. 成都: 四川大学, 2021.DOI: 10.27342/d.cnki.gscdu.2021.002906.
- [10] 吴金容. 某养老院管理信息系统的设计与实现[D]. 南昌: 江西财经大学, 2019. DOI: 10.27175/d.cnki.gjxcu.2019.000132.
- [11] LIU X Y, CHAU K Y, LIU X X, et al. The Progress of Smart Elderly Care Research: A Scientometric Analysis Based on CNKI and WOS[J]. International Journal of Environmental Research and Public Health, 2023, 20(2): 1086. DOI:10.3390/ijerph20021086.
- [12] DAHARI Z, CHOONG C J, POH J Z, et al. Development of Smart Elderly Care Mobile Application for Health Management System[C]//Proceedings of the 2022 IEEE International Conference on Artificial Intelligence in Engineering and Technology (IICAET). Kota Kinabalu, Malaysia: IEEE, 2022: 1-6. DOI: 10.1109/IICAET55139.2022.9936853.
- [13] LADKOOM K, KHOPLOYKLANG T. The Development of the Application for Public Health Support for the Elderly[C]//2024 8th International Conference on Information Technology (InCIT). Chonburi, Thailand: IEEE, 2024: 714-719. DOI: 10.1109/InCIT63192.2024.10810657.
- [14] MAHAJAN A, JINDAL P, SHARMA A K, et al. Essence of Internet of Medical Things in

- Geriatric Healthcare[C]//2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT). Kamand, India, 2024: 1-6. DOI: 10.1109/ICCCNT61001.2024.10723902.
- [15] GLENN J, SARMA P, CRISTMAN P, et al. Using the TrueLoo Smart Device to Record Toileting Sessions in Older Adults: Retrospective Validation and Acceptance Study[J]. JMIR aging, 2024, 7: e50856. DOI: 10.2196/50856.
- [16] KOUROUBALI A, KONDYLAHIS H, LOGOTHETIDIS F, et al. Developing an AI-Enabled Integrated Care Platform for Frailty[J]. Healthcare (Basel, Switzerland), 2022, 10(3): 443. DOI: 10.3390/healthcare10030443.
- [17] ZHANG L, HU X. Analysis of the Smart Medical Service Model in Super-Aged Society-UR Agency as an Example[J]. Journal of healthcare engineering, 2022, 2022: 8368057. DOI: 10.1155/2022/8368057.
- [18] MAEDER A, MORGAN G. Smart Ageing: Digital Solutions for Future Care[J]. Studies in health technology and informatics, 2020, 270: 678–682. DOI: 10.3233/SHTI200246.
- [19] 李云云. 浅析 B/S 和 C/S 体系结构[J]. 科学之友, 2011(2): 6-8.
- [20] 王冉阳. 基于 Django 和 Python 的 Web 开发[J]. 电脑编程技巧与维护, 2009, (02): 56-58.
- [21] 曾超宇, 李金香. Redis 在高速缓存系统中的应用[J]. 微型机与应用, 2013, 32(12): 11-13. DOI: 10.19358/j.issn.1674-7720.2013.12.004.
- [22] 朱二华. 基于 Vue.js 的 Web 前端应用研究 [J]. 科技与创新, 2017(20): 119-121. DOI: 10.15913/j.cnki.kjycx.2017.20.119.
- [23] 兰旭辉, 熊家军, 邓刚. 基于 MySQL 的应用程序设计 [J]. 计算机工程与设计, 2004, (03): 442-443+468. DOI: 10.16208/j.issn1000-7024.2004.03.037.
- [24] 惠燕, 潘煜, 徐光辉. 数据库设计范式 [J]. 现代电子技术, 2003(24): 107-108.
- [25] I. Puja, P. Posic, and D. Jaksic. Overview and comparison of several relational database modelling methodologies and notations [C]//2019 42nd International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO). Opatija, Croatia: IEEE, 2019: 1641-1646. DOI: 10.23919/MIPRO.2019.8756667.
- [26] NEELAND I J, LIM S, TCHERNOF A, et al. Metabolic syndrome [J]. Nature Reviews Disease Primers, 2024, 10: 77. DOI: 10.1038/s41572-024-00563-5.
- [27] ANTONY A. Metabolic syndrome: A comprehensive dataset on risk factors and health indicators [DB/OL]. Kaggle, 2023 [2025-03-10]. <https://www.kaggle.com/datasets/antimoni/metabolic-syndrome>.
- [28] 常妍, 李伟. 甘油三酯/高密度脂蛋白胆固醇比值与 2 型糖尿病及其并发症的关系研究进展[J]. 临床医学进展, 2024, 14(7): 1248-1254. DOI: 10.12677/acm.2024.1472139.

- [29] 薛建强, 刘玉萍, 倪国华, 等. 60 岁以上人群身体测量指标对代谢综合征的预测价值[J]. 心血管病学进展, 2022, 43(10): 749-756. DOI:10.16806/j.cnki.issn.1004-3934.2022.10.019.
- [30] 张新华, 何永前. 软件测试方法概述[J]. 科技视界, 2012(04): 35-37.

附录 A：后端关键代码

后端代码框架如图 A.1 所示：

```

smart_care/
|
├── .idea/           # IDE配置文件
├── .venv/           # 虚拟环境
├── api/             # API应用目录
|   ├── pycache/     # Python缓存文件
|   ├── migrations/  # 数据库迁移文件
|   ├── init.py      # 初始化文件
|   ├── admin.py     # 管理界面配置
|   ├── apps.py      # 应用配置
|   ├── cache_keys.py # 缓存键定义
|   ├── models.py    # 数据模型定义
|   ├── permissions.py # 权限控制
|   ├── serializers.py # 序列化器
|   ├── tasks.py     # 异步任务
|   ├── tests.py     # 测试文件
|   ├── urls.py      # URL路由配置
|   ├── utils.py     # 工具函数
|   └── views.py     # 视图函数
|
├── models/          # 模型文件目录
├── smart_care/       # 项目配置目录
├── templates/        # 模板文件目录
├── celerybeat-schedule.bak # Celery调度备份
├── celerybeat-schedule.dat # Celery调度数据
├── celerybeat-schedule.dir # Celery调度目录
├── dump.rdb          # Redis数据库文件
├── manage.py         # Django管理脚本
└── requirements.txt  # 项目依赖文件
  
```

图 A.1 后端代码框架

其中 `api` 文件夹为后端的唯一应用，后端中的所有视图都编写在该文件夹下。数据模型的定义在 `models.py` 文件中，视图的编写在 `views.py` 文件中，URL 路由配置在 `urls.py` 文件中，定时任务配置在 `tasks.py` 文件中，权限控制相关函数在 `permissions.py` 文件中，工具函数在 `utils.py` 文件中。由于 `permissions.py` 中代码在章节 5.1 中展示过，故本章将依次展示上述文件中除了 `permissions.py` 文件的代码。

`models.py` 文件中代码如图 A.2~A.10 所示。

```
1  from django.db import models
2  from enum import Enum
3  from django.contrib.auth.models import AbstractUser
4
5
6  9 个用法  ⬆️ Viittt0000
7  class IncomeRange(Enum):
8      LOW = '小于3000元/月'
9      MID = '3000-6000元/月'
10     HIGH = '大于6000元/月'
11
12  13 个用法  ⬆️ Viittt0000
13  class MaritalStatus(Enum):
14      MARRIED = '已婚'
15      SINGLE = '未婚'
16      DIVORCED = '离婚'
17      WIDOWED = '丧偶'
18      SEPARATED = '分居'
19
20  2 个用法  ⬆️ Viittt0000
21  class Status(Enum):
22      PENDING = "待确认"
23      SHIPPED = "已预约"
24      DELIVERED = "已结束"
25      CANCELLED = "已取消"
26
```

图 A.2 models.py 代码其 1

```
27  class PlanStatus(Enum):
28      PENDING = "待开始"
29      SHIPPED = "进行中"
30      DELIVERED = "已完成"
31
32  1 个用法  ⬆️ Viittt0000
33  class PlanType(Enum):
34      RECOVER = "康复护理"
35      DAILY = "日常护理"
36
37  2 个用法  ⬆️ Viittt0000
38  class GoalStatus(Enum):
39      UNDONE = "未达成"
40      DONE = "已达成"
41
42  6 个用法  ⬆️ Viittt0000
43  class Type(Enum):
44      BREAKFAST = "早餐"
45      LUNCH = "午餐"
46      DINNER = "晚餐"
47
48  5 个用法  ⬆️ Viittt0000
49  class GENDER(Enum):
50      MALE = "男"
51      FEMALE = "女"
52
```

图 A.3 models.py 代码其 2

```

54  class Appointments(models.Model):
55      appointment_id = models.AutoField(primary_key=True)
56      client = models.ForeignKey(to: 'Clients', models.CASCADE)
57      service = models.ForeignKey(to: 'Services', models.RESTRICT)
58      schedule_date = models.DateField()
59      schedule_time = models.TimeField()
60      end_time = models.TimeField(blank=True, null=True)
61      staff = models.ForeignKey(to: 'Staff', models.SET_NULL, blank=True, null=True)
62      state = models.CharField(
63          max_length=3,
64          choices=[(tag.value, tag.name) for tag in Status],
65          default=Status.PENDING.value,
66      )
67      satisfaction = models.IntegerField(blank=True, null=True)
68      plan = models.ForeignKey(to: 'CarePlans', models.CASCADE, blank=True, null=True)
69
70
71  24 个用法  📄 Viitttooooo
72  class CarePlans(models.Model):
73      plan_id = models.AutoField(primary_key=True)
74      client = models.ForeignKey(to: 'Clients', models.CASCADE)
75      plan_state = models.CharField(
76          max_length=3,
77          choices=[(tag.value, tag.name) for tag in PlanStatus],
78          default=PlanStatus.PENDING.value,
79      )
80      plan_type = models.CharField(
81          max_length=4,
82          choices=[(tag.value, tag.name) for tag in PlanType],
83          blank=True,
84          null=True
85      )

```

图 A.4 models.py 代码其 3

```

85      start_date = models.DateField()
86      end_date = models.DateField()
87      staff = models.ForeignKey(to: 'Staff', models.SET_NULL, blank=True, null=True)
88      plan_satisfaction = models.IntegerField(blank=True, null=True)
89
90
91  20 个用法  📄 Viitttooooo
92  class Clients(models.Model):
93      client_id = models.AutoField(primary_key=True, db_comment='客户id')
94      user = models.ForeignKey(to: 'Users', models.CASCADE)
95      care_level = models.IntegerField(blank=True, null=True)
96      care_demand = models.CharField(max_length=255, blank=True, null=True)
97      medical_history = models.CharField(max_length=255, blank=True, null=True)
98      dietary_preference = models.JSONField(blank=True, null=True)
99      gender = models.CharField(
100          max_length=10,
101          choices=[(tag.value, tag.name) for tag in GENDER],
102          default=GENDER.MALE.value,
103      )
104      birth_date = models.DateField(blank=True, null=True)
105      emergency_contact = models.JSONField(blank=True, null=True)
106      marital = models.CharField(
107          max_length=10,
108          choices=[(tag.value, tag.name) for tag in MaritalStatus],
109          default=MaritalStatus.SINGLE.value,
110      )
111      income_range = models.CharField(
112          max_length=20,
113          choices=[(tag.value, tag.name) for tag in IncomeRange],
114          default=IncomeRange.MID.value,
115      )

```

图 A.5 models.py 代码其 4

```

117 class DietPlans(models.Model):
118     diet_plan_id = models.AutoField(primary_key=True)
119     client = models.ForeignKey(Clients, models.CASCADE)
120     diet_date = models.DateField()
121     staff = models.ForeignKey(to: 'Staff', models.SET_NULL, blank=True, null=True)
122     nutrition_requirements = models.JSONField(blank=True, null=True)
123     nutrition_taken = models.JSONField(blank=True, null=True)
124     smart_recommendation = models.TextField(blank=True, null=True)
125
126     class Meta:
127         unique_together = ('diet_date', 'client')
128
129
130 17 个用法  Viittotooo
131 class FoodRecipes(models.Model):
132     recipe_id = models.AutoField(primary_key=True)
133     recipe_name = models.CharField(max_length=255)
134     nutrition_info = models.JSONField(blank=True, null=True)
135
136 10 个用法  Viittotooo
137 class HealthMetrics(models.Model):
138     metric_id = models.AutoField(primary_key=True)
139     client = models.ForeignKey(Clients, models.CASCADE)
140     vital_signs = models.JSONField(blank=True, null=True)
141     record_date = models.DateTimeField(blank=True, null=True)
142     assessment = models.CharField(max_length=1024, blank=True, null=True)
143     staff = models.ForeignKey(to: 'Staff', models.CASCADE, blank=True, null=True)
144     mets_probability = models.FloatField(blank=True, null=True)
145     smart_assessment = models.CharField(max_length=1024, blank=True, null=True)

```

图 A.6 models.py 代码其 5

```

147 class Ingredients(models.Model):
148     ingredient_id = models.AutoField(primary_key=True)
149     ingredient_name = models.CharField(max_length=255, unique=True)
150     ingredient_description = models.CharField(max_length=255)
151     ingredient_amount = models.IntegerField()
152     unit = models.CharField(max_length=255)
153     nutrition = models.JSONField()
154
155
156 8 个用法  Viittotooo
157 class PlanGoals(models.Model):
158     goal_id = models.AutoField(primary_key=True)
159     plan = models.ForeignKey(CarePlans, models.CASCADE)
160     goal_state = models.CharField(
161         max_length=3,
162         choices=[(tag.value, tag.name) for tag in GoalStatus],
163         default=GoalStatus.UNDONE.value,
164     )
165     description = models.CharField(max_length=1024, blank=True, null=True)
166
167 11 个用法  Viittotooo
168 class PlanRecipe(models.Model):
169     diet_plan = models.ForeignKey(DietPlans, models.CASCADE)
170     recipe = models.ForeignKey(FoodRecipes, models.CASCADE)
171     type = models.CharField(
172         max_length=2,
173         choices=[(tag.value, tag.name) for tag in Type],
174         blank=True,
175         null=True
176     )

```

图 A.7 models.py 代码其 6

```

178  class RecipeIngredient(models.Model):
179      id = models.AutoField(primary_key=True)
180      recipe = models.ForeignKey(FoodRecipes, models.CASCADE)
181      ingredient = models.ForeignKey(Ingredients, models.CASCADE)
182      quantity = models.DecimalField(max_digits=10, decimal_places=2)
183
184
185  class Roles(models.Model):
186      role_id = models.IntegerField(primary_key=True)
187      role_name = models.CharField(max_length=255)
188
189
190  class Services(models.Model):
191      service_id = models.AutoField(primary_key=True)
192      service_name = models.CharField(max_length=255)
193      duration = models.IntegerField(blank=True, null=True)
194
195
196  class ShiftTemplates(models.Model):
197      template_id = models.AutoField(primary_key=True)
198      shift_name = models.CharField(max_length=255)
199      start_time = models.TimeField()
200      end_time = models.TimeField()
201      color_code = models.CharField(max_length=255, blank=True, null=True)
202

```

图 A.8 models.py 代码其 7

```

204  class Staff(models.Model):
205      staff_id = models.AutoField(primary_key=True)
206      user = models.ForeignKey(to='Users', models.CASCADE)
207
208
209  class StaffSchedules(models.Model):
210      shift_id = models.AutoField(primary_key=True)
211      staff = models.ForeignKey(Staff, models.CASCADE)
212      template = models.ForeignKey(ShiftTemplates, models.RESTRICT)
213      assigned_date = models.DateField()
214
215  class Meta:
216      unique_together = ('staff', 'assigned_date')
217
218
219  class Users(AbstractUser):
220      role = models.ForeignKey(Roles, models.RESTRICT)
221      is_reset = models.BooleanField(default=False)
222
223  def __str__(self):
224      return self.username

```

图 A.9 models.py 代码其 8


```

12 个用法  👤 Viitttoooo
227  class Notification(models.Model):
228     user = models.ForeignKey(to: 'Users', on_delete=models.CASCADE)
229     message = models.TextField()
230     created_at = models.DateTimeField(auto_now_add=True)
231     is_read = models.BooleanField(default=False)
232
233     👤 Viitttoooo
234     class Meta:
        ordering = ['-created_at'] # 按创建时间降序排列

```

图 A.10 models.py 代码其 9

views.py 文件代码量过大，总长度超过 2000 行，出于篇幅考虑这里只展示其中典型的几个视图，如图 A.11~A.14 所示。

```

71  # 管理员更新用户状态
72  1 个用法  👤 Viitttoooo
73  @swagger_auto_schema(
74      method='PATCH'
75  )
76  @api_view(['PATCH'])
77  @permission_classes([IsAdmin])
78  def update_user_active(request, pk):
79      user = Users.objects.get(pk=pk)
80      if user.role.role_name == "管理员":
81          return Response(data={'error': '不能更改管理员账户状态'}, status=status.HTTP_403_FORBIDDEN)
82      user.is_active = not user.is_active
83      user.save()
84      cache.delete(USERS_DATA)
85      serializer = UserSerializer(user)
86      return Response(serializer.data)

```

图 A.11 views.py 代码其 1

```

88  # 获取所有用户信息
89  1 个用法  👤 Viitttoooo
90  @swagger_auto_schema(
91      method='GET',
92      operation_description="获取所有用户信息", # 对接口的描述
93      responses={ # 响应描述
94          200: '返回所有用户信息'
95      }
96  )
97  @api_view(['GET'])
98  @permission_classes([IsAdmin])
99  def user_list(request):
100      users_data = cache.get(USERS_DATA)
101      if not users_data:
102          users = Users.objects.all()
103          serializer = UserSerializer(users, many=True)
104          users_data = serializer.data
105          cache.set(USERS_DATA, users_data)
106      return Response(users_data, status=status.HTTP_200_OK)

```

图 A.12 views.py 代码其 2

```

2522 @swagger_auto_schema(
2523     method="PATCH",
2524 )
2525 @api_view(["PATCH"])
2526 @permission_classes([IsAdmin | IsStaff])
2527 def generate_metrics_assessment(request, metric_id):
2528     try:
2529         metric = HealthMetrics.objects.get(pk=metric_id)
2530     except HealthMetrics.DoesNotExist:
2531         return Response( data: {'error': 'metric does not exist'}, status=status.HTTP_404_NOT_FOUND)
2532
2533     # 获取用户信息
2534     client = metric.client
2535     age = calculate_age(client.birth_date)
2536     sex = client.gender
2537     marital = client.marital
2538     income_category = client.income_range
2539     race = 'Asian'
2540     vital_signs = metric.vital_signs
2541     mets_probability = metric.mets_probability
2542
2543     assessment = get_metrics_assessment(age, sex, marital, income_category, race, vital_signs, mets_probability)
2544
2545     metric.smart_assessment = assessment
2546     metric.save()
2547     cache.delete(PER_METRICS.format(client.client_id))
2548     return Response( data: {'smart_assessment': assessment}, status=status.HTTP_200_OK)

```

图 A.13 views.py 代码其 3

```

740 # 查询某客户所有健康档案
741 # 1 个用法  Viitttoooo
742 @swagger_auto_schema(
743     method='GET',
744     responses={
745         200: "成功",
746         400: "失败",
747     }
748 )
749 @api_view(['GET'])
750 @permission_classes([IsStaff | IsAdmin | IsClient])
751 def get_client_health_metrics(request, client_id):
752     cache_key = PER_METRICS.format(client_id)
753     metrics_data = cache.get(cache_key)
754     if not metrics_data:
755         try:
756             metrics = HealthMetrics.objects.filter(client_id=client_id)
757             serializer = HealthMetricSerializer(metrics, many=True)
758             metrics_data = serializer.data
759             cache.set(cache_key, metrics_data)
760         except Exception as e:
761             return Response( data: {"error": str(e)}, status=status.HTTP_400_BAD_REQUEST)
762     return Response(metrics_data, status=status.HTTP_200_OK)

```

图 A.14 views.py 代码其 4

urls.py 文件中接口路由共计 80 余个，这里仅展示部分接口路由，如图 A.15 所示。


```

1  from . import views
2  from django.urls import path
3
4  urlpatterns = [
5      path('users/', views.user_list, name='user-list'),
6      path('users/<int:pk>/', views.user_detail, name='user-detail'),
7      path('users/info/', views.get_user_info, name='user-info'),
8      path('users/login/', views.login, name='login'),
9      path('users/register/', views.register, name='register'),
10     path('users/logout/', views.user_logout, name='user_logout'), # 登出接口路由
11     path('users/update/', views.edit_user, name='edit_user'), # 用户个人更新用户数据
12     path('users/password_reset/<int:pk>/', views.reset_password, name='reset_password'), # 管理员重置用户密码
13     path('users/deny_reset/<int:pk>', views.reset_deney, name='reset_deney'), # 管理员拒绝重置密码申请
14     path('users/password_update/', views.update_password, name='update_password'), # 用户个人更新个人密码
15     path('users/register/admin/', views.register_admin, name='register_user_admin'), # 管理员注册用户接口

```

图 A.15 urls.py 部分代码

tasks.py 文件中主要配置的是通知有关的定时任务，如图 A.16~A.18 所示

```

1  from celery import shared_task
2  from django.utils import timezone
3  from .models import Appointments, CarePlans, Notification, Users, Roles
4  from datetime import timedelta, datetime
5  import pytz
6
7  @shared_task
8  def send_admin_notification():
9      """每小时检查未指定员工的预约并通知管理员"""
10     appointments = Appointments.objects.filter(staff__isnull=True, state__in=['待确认', '已预约'])
11     if appointments.exists():
12         admins = Users.objects.filter(role__role_name='管理员')
13         message = f"有 {appointments.count()} 个预约未指定员工，请尽快处理。"
14         for admin in admins:
15             Notification.objects.create(user=admin, message=message)

```

图 A.16 tasks.py 代码其 1

```

17  @shared_task
18  def send_appointment_reminders():
19      """检查即将到来的预约并发送提醒"""
20      tz = pytz.timezone('Asia/Shanghai')
21      now = datetime.now(tz)
22      reminders = [
23          timedelta(hours=24),
24          timedelta(hours=1),
25          timedelta(minutes=30),
26      ]
27      for reminder in reminders:
28          remind_time = now + reminder
29          appointments = Appointments.objects.filter(
30              schedule_date=remind_time.date(),
31              schedule_time__gte=(remind_time - timedelta(minutes=1)).time(),
32              schedule_time__lte=(remind_time + timedelta(minutes=1)).time(),
33              state__in=['待确认', '已预约']
34          )
35          for appointment in appointments:
36              message = f"您有一个预约即将开始，服务：{appointment.service.service_name}，时间：{appointment.schedule_date} {appointment.schedule_time}。"
37              if appointment.staff:
38                  Notification.objects.create(user=appointment.staff.user, message=message)
39              Notification.objects.create(user=appointment.client.user, message=message)

```

图 A.17 tasks.py 代码其 2

```

41 @shared_task
42 def send_care_plan_reminders():
43     """每天早上6点提醒未结束的护理计划"""
44     care_plans = CarePlans.objects.filter(plan_state__in=['待开始', '进行中'])
45     for plan in care_plans:
46         if plan.staff:
47             client_name = f"{plan.client.user.last_name} {plan.client.user.first_name}"
48             message = f"您有未完成的护理计划，客户：{client_name}，开始时间：{plan.start_date}，结束时间：{plan.end_date}。"
49             Notification.objects.create(user=plan.staff.user, message=message)
50
51 @shared_task
52 def clean_old_notifications():
53     """删除7天前的已读通知"""
54     threshold = timezone.now() - timedelta(days=7)
55     Notification.objects.filter(is_read=True, created_at__lt=threshold).delete()

```

图 A.18 tasks.py 代码其 3

utils.py 文件中主要包含的是调用大模型的函数和代谢综合征预测的函数，如图 A.19~A.26 所示。

```

14 def get_diet_recommend(diet_record, taboo_ingredients, favorite_ingredients, recipe_book, nutrition_requirements, gender,
15                        age, medical_history):
16     """
17     根据用户的饮食记录和个人信息生成个性化的饮食建议。
18
19     参数:
20         diet_record: 用户前1-3天的膳食记录
21         taboo_ingredients: 用户忌口的食材列表
22         favorite_ingredients: 用户喜欢的食材列表
23         recipe_book: 可选的菜谱信息，包含食材和营养成分
24         nutrition_requirements: 用户所需的营养成分占比
25         gender: 用户性别
26         age: 用户年龄
27         medical_history: 用户病史
28
29     返回:
30         str: 包含早餐、午餐、晚餐建议的个性化饮食方案
31     """
32     # 系统角色设定
33     system_prompt = '你是一个**营养膳食专家**，专为康养中心的中老年客户服务。'
34
35     # 构建用户提示词
36     user_prompt = (
37         f'请用不超过100个汉字的中文回答以下问题。\\n\\n'
38         f'请参考用户前一到三天的膳食记录，{diet_record}，并在推荐新的一天的饮食时，'
39         f'尽量避免与最近三天内用户食用过的食物重复。\\n\\n'
40         f'在推荐的菜谱中，请避免使用用户的忌口食材，{taboo_ingredients}，'
41         f'并尽量使用用户爱吃的食材，{favorite_ingredients}。\\n\\n'
42         f'请从提供的菜谱中选择适合的菜品，菜谱信息为：{recipe_book}，'
43         f'每道菜的食材和营养成分占比均已提供。\\n\\n'

```

图 A.19 utils.py 文件膳食推荐函数其 1

```

44         f'**请为用户推荐一天的饮食，包括早餐、午餐、晚餐，并对每餐做出解释（人性化解释）'
45         f'（请尽量避免一天出现重复菜品的情况）**，并确保推荐的饮食方案能够满足用户所需的'
46         f'营养成分占比：{nutrition_requirements}（这里的营养占比可能会不符合常理，'
47         f'这种情况下请忽略营养占比）\n\n'
48         f'（用于参考的用户个人信息：\n'
49         f'性别：{gender}，年龄：{age}，病史：{medical_history}。）'
50     )
51
52     # 调用API生成饮食建议
53     completion = client.chat.completions.create(
54         model="qwen-max-latest",
55         messages=[
56             {'role': 'system', 'content': system_prompt},
57             {'role': 'user', 'content': user_prompt}
58         ],
59     )
60
61     # 返回生成的建议
62     return completion.choices[0].message.content

```

图 A.20 utils.py 文件膳食推荐函数其 2

```

65 def get_metrics_assessment(age, sex, marital, income_category, race, vital_signs, mets_probability):
66     """
67     给出身体建议
68
69     参数:
70     age: 年龄
71     sex: 性别 ('Male' 或 'Female')
72     marital: 婚姻状况 ('Single', 'Married', 'Divorced', 'Widowed'等)
73     income_category: 收入类别 ('Low', 'Mid', 'High')
74     race: 种族 ('White', 'Black', 'Asian'等)
75     vital_signs: 身体指标
76     mets_probability: 患代谢综合征的概率
77
78     返回:
79     智能建议
80     """
81     content = f"""
82     请基于以下用户信息和身体指标，提供专业的健康建议：
83
84     基本信息：
85     - 年龄：{age}岁
86     - 性别：{sex}
87     - 婚姻状况：{marital}
88     - 收入水平：{income_category}
89     - 种族：{race}
90
91     身体指标：
92     - BMI：{vital_signs.get('bmi', {}).get('value')} {vital_signs.get('bmi', {}).get('unit')} (正常范围：{vital_signs.get('bmi', {}).get('normal_range')})
93     - 身高：{vital_signs.get('height', {}).get('value')} {vital_signs.get('height', {}).get('unit')}

```

图 A.21 utils.py 文件身体建议函数其 1

```

94 ~ 体重: {vital_signs.get('weight', {}).get('value')} {vital_signs.get('weight', {}).get('unit')}
95 ~ 尿酸: {vital_signs.get('uric_acid', {}).get('value')} {vital_signs.get('uric_acid', {}).get('unit')} (正常范围: {vital_signs.get('uric_acid', {}).get('normal_range')})
96 ~ 心率: {vital_signs.get('heart_rate', {}).get('value')} {vital_signs.get('heart_rate', {}).get('unit')} (正常范围: {vital_signs.get('heart_rate', {}).get('normal_range')})
97 ~ 白蛋白尿: {vital_signs.get('albuminuria', {}).get('value')} {vital_signs.get('albuminuria', {}).get('unit')} (正常范围: {vital_signs.get('albuminuria', {}).get('normal_range')})
98 ~ 血糖: {vital_signs.get('blood_glucose', {}).get('value')} {vital_signs.get('blood_glucose', {}).get('unit')} (正常范围: {vital_signs.get('blood_glucose', {}).get('normal_range')})
99 ~ 甘油三酯: {vital_signs.get('triglycerides', {}).get('value')} {vital_signs.get('triglycerides', {}).get('unit')} (正常范围: {vital_signs.get('triglycerides', {}).get('normal_range')})
100 ~ 血压: 收缩压 {vital_signs.get('blood_pressure', {}).get('systolic', {}).get('value')} {vital_signs.get('blood_pressure', {}).get('systolic', {}).get('unit')} (正常范围: {vital_signs.get('blood_pressure', {}).get('systolic_range')})
101 ~ 舒张压 {vital_signs.get('blood_pressure', {}).get('diastolic', {}).get('value')} {vital_signs.get('blood_pressure', {}).get('diastolic', {}).get('unit')} (正常范围: {vital_signs.get('blood_pressure', {}).get('diastolic_range')})
102 ~ 高密度脂蛋白胆固醇: {vital_signs.get('hdl_cholesterol', {}).get('value')} {vital_signs.get('hdl_cholesterol', {}).get('unit')} (正常范围: {vital_signs.get('hdl_cholesterol', {}).get('normal_range')})
103 ~ 体温: {vital_signs.get('body_temperature', {}).get('value')} {vital_signs.get('body_temperature', {}).get('unit')} (正常范围: {vital_signs.get('body_temperature', {}).get('normal_range')})
104 ~ 呼吸频率: {vital_signs.get('respiratory_rate', {}).get('value')} {vital_signs.get('respiratory_rate', {}).get('unit')} (正常范围: {vital_signs.get('respiratory_rate', {}).get('normal_range')})
105 ~ 血氧饱和度: {vital_signs.get('oxygen_saturation', {}).get('value')} {vital_signs.get('oxygen_saturation', {}).get('unit')} (正常范围: {vital_signs.get('oxygen_saturation', {}).get('normal_range')})
106 ~ 腰围: {vital_signs.get('waist_circumference', {}).get('value')} {vital_signs.get('waist_circumference', {}).get('unit')} (正常范围: {vital_signs.get('waist_circumference', {}).get('normal_range')})
107 ~ 尿白蛋白肌酐比值: {vital_signs.get('urine_albumin_creatinine_ratio', {}).get('value')} {vital_signs.get('urine_albumin_creatinine_ratio', {}).get('unit')} (正常范围: {vital_signs.get('urine_albumin_creatinine_ratio', {}).get('normal_range')})
108
109 代谢综合征风险:
110 ~ 患代谢综合征的概率: {mets_probability*100:.2f}%
111
112 请根据上述信息提供详细的健康建议, 包括:
113 1. 总体健康状况评估
114 2. 需要改善的关键指标
115 3. 具体的生活方式建议 (饮食、运动等)
116 4. 是否需要进一步的医疗咨询
117
118 请用简单易懂的语言回答, 使用中文, 不超过300字。
119 """
120
121 system_prompt = '你是一个**专业医生**, 专精中老年人身体慢性病领域, 擅长通过各项身体数据分析出病人的身体情况。'

```

图 A.22 utils.py 文件身体建议函数其 2

```

121 system_prompt = '你是一个**专业医生**, 专精中老年人身体慢性病领域, 擅长通过各项身体数据分析出病人的身体情况。'
122 completion = client.chat.completions.create(
123     model="qwen-max-latest",
124     messages=[
125         {'role': 'system', 'content': system_prompt},
126         {'role': 'user', 'content': content},
127     ]
128 )
129 return completion.choices[0].message.content

```

图 A.23 utils.py 文件身体建议函数其 3

```

158 def predict_metabolic_syndrome(age, sex, marital, income_category, waist_circ,
159                                bmi, albuminuria, ur_alb_cr, uric_acid, blood_glucose,
160                                hdl, triglycerides, race='Asian'):
161     """
162     预测一个人是否患有代谢综合征
163
164     参数:
165     age: 年龄
166     sex: 性别 ('Male' 或 'Female')
167     marital: 婚姻状况 ('Single', 'Married', 'Divorced', 'Widowed'等)
168     income_category: 收入类别 ('Low', 'Mid', 'High')
169     race: 种族 ('White', 'Black', 'Asian'等)
170     waist_circ: 腰围
171     bmi: 体重指数
172     albuminuria: 白蛋白尿 (0或1)
173     ur_alb_cr: 尿白蛋白肌酐比值
174     uric_acid: 尿酸
175     blood_glucose: 血糖
176     hdl: 高密度脂蛋白胆固醇
177     triglycerides: 甘油三酯
178
179     返回:
180     预测结果 (0: 无代谢综合征, 1: 有代谢综合征)
181     预测概率
182     """

```

图 A.24 utils.py 文件代谢综合征预测函数其 1


```

184     app_config = apps.get_app_config('api')
185     model = app_config.model
186     label_encoders = app_config.label_encoders
187     scaler = app_config.scaler
188     feature_selector = app_config.feature_selector
189     selected_features = app_config.selected_features
190
191
192     # 对分类特征进行编码
193     sex_encoded = label_encoders['Sex'].transform([sex])[0]
194     marital_encoded = label_encoders['Marital'].transform([marital])[0]
195     income_encoded = label_encoders['IncomeCategory'].transform([income_category])[0]
196     race_encoded = label_encoders['Race'].transform([race])[0]
197
198     # 计算额外的特征
199     waist_bmi_ratio = waist_circ / bmi
200     glucose_hdl_ratio = blood_glucose / hdl
201     trig_hdl_ratio = triglycerides / hdl
202
203     # 根据年龄确定年龄组
204     if age < 30:
205         age_group = 0
206     elif age < 45:
207         age_group = 1
208     elif age < 60:
209         age_group = 2
210     else:
211         age_group = 3

```

图 A.25 utils.py 文件代谢综合征预测函数其 2

```

213     # 创建特征向量（包含所有原始特征和新特征）
214     all_features = [age, sex_encoded, marital_encoded, income_encoded, race_encoded,
215                    waist_circ, bmi, albuminuria, ur_alb_cr, uric_acid, blood_glucose,
216                    hdl, triglycerides, waist_bmi_ratio, glucose_hdl_ratio, trig_hdl_ratio,
217                    age_group]
218
219     # 特征标准化
220     features_scaled = scaler.transform([all_features])
221
222     # 应用特征选择
223     features_selected = feature_selector.transform(features_scaled)
224
225     # 预测
226     prediction = model.predict(features_selected)[0]
227     probability = model.predict_proba(features_selected)[0]
228
229     return prediction, probability

```

图 A.26 utils.py 文件代谢综合征预测函数其 3

附录 B：前端关键代码

前端代码整体框架如图 B.1 所示。

```
smart_care_web/
├── public/           # 静态资源目录
├── src/             # 源代码目录
│   ├── assets/      # 资源文件（图片、字体等）
│   ├── components/  # 公共组件
│   ├── config/      # 配置文件
│   ├── router/      # 路由配置
│   ├── utils/       # 工具函数
│   ├── views/       # 页面视图组件
│   ├── App.vue      # 根组件
│   ├── main.js      # 入口文件
│   └── style.css     # 全局样式
├── .vscode/         # VS Code 配置
├── node_modules/    # 依赖包
├── .gitignore       # Git 忽略文件
├── index.html       # HTML 模板
├── package.json     # 项目配置文件
├── package-lock.json # 依赖版本锁定文件
├── README.md        # 项目说明文档
└── vite.config.js   # Vite 配置文件
```

图 B.1 前端整体框架

由于前端代码量总行数超过 10000 行，本章只选取关键文件夹 `components` 和 `views` 及其中的部分文件代码进行展示。`components` 文件夹下存放的是组件代码，`views` 文件夹下存放的是页面代码，其中的具体结构如图 B.2 所示。



图 B.2 components 和 view 下具体文件结构

前端的主要页面主要有两个，分别是 Login.vue 和 Dashboard.vue，其中 Dashboard.vue 页面作为主要的工作台页面，在 Dashboard.vue 页面中通过按钮交互从组件中选取对应组件在页面上显示。

Login.vue 部分代码如图 B.3 所示。

```

1 <!-- src/views/Login.vue -->
2 <template>
3   <div class="login-container">
4     <div class="login-box">
5       <div class="logo-container">
6         
7       </div>
8
9       <!-- 登录表单 -->
10      <div v-if="currentView === 'login'" class="form-container">
11        <h2 class="form-title">欢迎回来</h2>
12        <p class="form-subtitle">请输入您的账号信息进行登录</p>
13
14        <el-form
15          ref="loginFormRef"
16          :model="loginForm"
17          :rules="loginRules"
18          label-position="top"
19          @submit.prevent="handleLogin"
20        >
21          <el-form-item label="用户名" prop="username">
22            <el-input
23              v-model="loginForm.username"
24              placeholder="请输入用户名"
25              prefix-icon="User"
26              class="custom-input"
27            />
28          </el-form-item>
29
30          <el-form-item label="密码" prop="password">
31            <el-input
32              v-model="loginForm.password"
33              type="password"
34              placeholder="请输入密码"
35              prefix-icon="Lock"
36              show-password
37              class="custom-input"

```

图 B.3 Login.vue 部分代码图

Dashboard.vue 部分代码如图 B.4 所示。

```

1 <!-- src/views/Dashboard.vue -->
2 <template>
3   <div class="dashboard-container">
4     <!-- 左侧边栏 -->
5     <div class="sidebar custom-scrollbar">
6       <Logo />
7       <el-menu
8         class="sidebar-menu"
9         :default-active="currentView"
10        background-color="var(--primary-color)"
11        text-color="var(--white)"
12        active-text-color="var(--white)"
13        :default-openeds="openedMenus"
14        @open="handleSubMenuOpen"
15        @close="handleSubMenuClose"
16      >
17        <!-- 首页单独展示 -->
18        <el-menu-item index="home" @click="currentView = 'home'">
19          <el-icon>House /></el-icon>
20          <span>首页</span>
21        </el-menu-item>
22
23        <!-- 客户类别 -->
24        <el-sub-menu index="client-category" v-if="showClientCategory">
25          <template #title>
26            <el-icon>User /></el-icon>
27            <span>客户</span>
28          </template>
29          <el-menu-item
30            v-if="user.role_name === '员工' || user.role_name === '管理员'"
31            index="clients-list"
32            @click="currentView = 'clients-list'"
33          >

```

图 B.4 Dashboard.vue 部分代码图

Home.vue 部分代码如图 B.5 所示。

```

1 <template>
2 <div class="home">
3 <!-- 管理员和员工视图 -->
4 <template v-if="isStaff">
5 <div v-if="!selectedClient">
6 <!-- 页面标题 -->
7 <div class="page-header">
8 <h1>数据概览</h1>
9 <div class="page-description">欢迎使用康养管理系统，以下是系统数据概览</div>
10 </div>
11
12 <!-- 预约数据展示区域 -->
13 <div class="section-container">
14 <div class="section-header">
15 <h2>预约数据展示</h2>
16 <div class="section-actions">
17 <el-button size="small" type="primary" plain @click="refreshAppointmentData">
18 <el-icon>Refresh /></el-icon>
19 刷新数据
20 </el-button>
21 </div>
22 </div>
23 <div class="charts-row">
24 <div class="chart-container">
25 <h3>预约满意度</h3>
26 <div v-if="hasAppointmentData" ref="appointmentSatisfactionChart" class="chart"></div>
27 <el-empty v-else description="暂无数据" />
28 </div>
29 <div class="chart-container">
30 <h3>预约状态</h3>
31 <div v-if="hasAppointmentData" ref="appointmentStateChart" class="chart"></div>
32 <el-empty v-else description="暂无数据" />
33 </div>
34 <div class="chart-container">
35 <h3>服务类型</h3>
36 <div v-if="hasAppointmentData" ref="serviceTypeChart" class="chart"></div>
37 <el-empty v-else description="暂无数据" />

```

图 B.5 Home.vue 部分代码图

Notification.vue 部分代码如图 B.6 所示。

```

1 <template>
2 <div class="notifications-container">
3 <div class="notifications-header">
4 <div class="section-title">我的通知</div>
5 <el-button
6 v-if="notifications.length > 0 && hasUnread"
7 type="primary"
8 size="small"
9 @click="markAllAsRead"
10 >
11 全部标为已读
12 </el-button>
13 </div>
14
15 <div class="notifications-content card-container custom-scrollbar">
16 <el-empty v-if="notifications.length === 0" description="暂无通知" />
17
18 <div v-else class="notification-list">
19 <el-card
20 v-for="notification in notifications"
21 :key="notification.id"
22 class="notification-item"
23 :class="{ 'unread': !notification.is_read }"
24 shadow="hover"
25 >
26 <div class="notification-content">
27 <div class="notification-message">{{ notification.message }}</div>
28 <div class="notification-footer">
29 <div class="notification-time">{{ formatDateTime(notification.created_at) }}</div>
30 <el-button
31 v-if="!notification.is_read"
32 type="primary"
33 size="small"
34 @click.stop="markAsRead(notification.id)"
35 >
36 标为已读
37 </el-button>

```

图 B.6 Notification.vue 部分代码图

UserManagement.vue 部分代码图如图 B.7 所示。

```

1  <template>
2    <div class="user-management">
3      <!-- 搜索和筛选区域 -->
4      <div class="filter-container">
5        <div class="filter-section">
6          <div class="section-title">用户筛选</div>
7          <div class="filter-content">
8            <el-input
9              v-model="searchQuery"
10             placeholder="搜索用户ID/用户名/姓名"
11             class="search-input"
12             clearable
13            >
14              <template #prefix>
15                <el-icon>Search </el-icon>
16              </template>
17            </el-input>
18            <el-select
19              v-model="selectedRole"
20              placeholder="用户身份"
21              clearable
22              class="role-filter"
23            >
24              <el-option
25                v-for="role in roles"
26                :key="role.role_id"
27                :label="role.role_name"
28                :value="role.role_name"
29              />
30            </el-select>
31            <el-button type="primary" @click="showAddUserDialog">
32              <el-icon>Plus </el-icon>新增用户
33            </el-button>
34            <el-button @click="fetchUsers">
35              <el-icon>Refresh </el-icon>刷新数据
36            </el-button>
37          </div>

```

图 B.7 UserManagement.vue 部分代码图

UserProfile.vue 部分代码图如图 B.8 所示。

```

1  <template>
2    <div class="user-profile">
3      <h2>个人信息</h2>
4      <div class="profile-content">
5        <el-form ref="formRef" :model="userInfo" :disabled="!isEditing" label-width="100px">
6          <el-form-item label="用户名">
7            <el-input v-model="userInfo.username" />
8          </el-form-item>
9          <el-form-item label="名">
10             <el-input v-model="userInfo.first_name" />
11           </el-form-item>
12           <el-form-item label="姓">
13             <el-input v-model="userInfo.last_name" />
14           </el-form-item>
15           <el-form-item label="邮箱">
16             <el-input v-model="userInfo.email" type="email" />
17           </el-form-item>
18           <el-form-item label="角色">
19             <el-input v-model="userInfo.role_name" disabled />
20           </el-form-item>
21         </el-form>
22
23         <div class="action-buttons">
24           <el-button
25             type="primary"
26             @click="handleEditClick"
27           >
28             {{ isEditing ? '完成修改' : '修改信息' }}
29           </el-button>
30         </div>
31       </div>
32     </div>
33   </template>
34
35   <script setup>
36     import { ref, onMounted } from 'vue';
37     import { ElMessage } from 'element-plus';

```

图 B.8 UserProfile.vue 部分代码图

ChangePassword.vue 部分代码图如图 B.9 所示。

```

1 <template>
2   <div class="change-password">
3     <h2>修改密码</h2>
4     <div class="password-content">
5       <el-form
6         ref="formRef"
7         :model="passwordForm"
8         :rules="rules"
9         label-width="120px"
10      >
11         <el-form-item label="旧密码" prop="oldPassword">
12           <el-input
13             v-model="passwordForm.oldPassword"
14             type="password"
15             show-password
16             placeholder="请输入旧密码"
17           />
18         </el-form-item>
19         <el-form-item label="新密码" prop="newPassword">
20           <el-input
21             v-model="passwordForm.newPassword"
22             type="password"
23             show-password
24             placeholder="请输入新密码"
25           />
26         </el-form-item>
27         <el-form-item label="确认新密码" prop="confirmPassword">
28           <el-input
29             v-model="passwordForm.confirmPassword"
30             type="password"
31             show-password
32             placeholder="请再次输入新密码"
33           />
34         </el-form-item>
35         <el-form-item>
36           <el-button type="primary" @click="handleSubmit">确认修改</el-button>
37         </el-form-item>

```

图 B.9 ChangePassword.vue 部分代码图

ClientsList.vue 部分代码图如图 B.10 所示。

```

1 <template>
2   <div class="clients-list">
3     <el-scrollbar>
4       <!-- 搜索和筛选区域 -->
5       <div class="filter-container">
6         <div class="filter-section client-filter-section">
7           <div class="section-title">客户筛选</div>
8           <div class="filter-content">
9             <el-input
10               v-model="searchQuery"
11               placeholder="搜索客户姓名或ID"
12               class="search-input"
13               clearable
14             >
15               <template #prefix>
16                 <el-icon>Search</el-icon>
17               </template>
18             </el-input>
19             <el-select
20               v-model="selectedLevel"
21               placeholder="护理等级"
22               clearable
23               class="level-filter"
24             >
25               <el-option
26                 v-for="level in [1, 2, 3, 4, 5]"
27                 :key="level"
28                 :label="'${level}级'"
29                 :value="level"
30               />
31             </el-select>
32           </div>
33         </div>
34       </div>
35       <!-- 客户列表区域 -->
36       <div class="clients-content-wrapper">

```

图 B.10 ClientsList.vue 部分代码图

ClientProfile.vue 部分代码图如图 B.11 所示。

```

1 <template>
2 <div class="client-profile">
3   <div class="profile-header">
4     <h2>客户档案</h2>
5     <el-button type="primary" @click="toggleEdit">{{ isEditing ? '保存' : '修改信息' }}</el-button>
6   </div>
7   <div class="profile-content">
8     <el-descriptions :column="2" border>
9       <el-descriptions-item label="姓">
10        <template v-if="isEditing">
11          <el-input v-model="editedInfo.last_name" />
12        </template>
13        <template v-else>{{ clientInfo.last_name }}</template>
14      </el-descriptions-item>
15      <el-descriptions-item label="名">
16        <template v-if="isEditing">
17          <el-input v-model="editedInfo.first_name" />
18        </template>
19        <template v-else>{{ clientInfo.first_name }}</template>
20      </el-descriptions-item>
21      <el-descriptions-item label="性别">
22        <template v-if="isEditing">
23          <el-select v-model="editedInfo.gender">
24            <el-option label="男" value="男" />
25            <el-option label="女" value="女" />
26          </el-select>
27        </template>
28        <template v-else>{{ clientInfo.gender }}</template>
29      </el-descriptions-item>
30      <el-descriptions-item label="出生日期">
31        <template v-if="isEditing">
32          <el-date-picker
33            v-model="editedInfo.birth_date"
34            type="date"
35            format="YYYY-MM-DD"
36            value-format="YYYY-MM-DD"

```

图 B.11 ClientProfile.vue 部分代码图

HealthRecords.vue 部分代码图如图 B.12 所示。

```

1 <template>
2 <div class="health-records">
3   <!-- 管理员和员工视图：显示客户列表 -->
4   <template v-if="isStaff">
5     <!-- 客户筛选区域 -->
6     <div class="filter-container">
7       <div class="filter-section client-filter-section">
8         <div class="section-title">客户筛选</div>
9         <div class="filter-content">
10          <el-input
11            v-model="searchQuery"
12            placeholder="搜索客户姓名"
13            class="search-input"
14            clearable
15          >
16            <template #prefix>
17              <el-icon>Search /></el-icon>
18            </template>
19          </el-input>
20          <el-input
21            v-model="clientIdFilter"
22            placeholder="客户ID"
23            class="client-id-input"
24            clearable
25          />
26          <el-select
27            v-model="selectedLevel"
28            placeholder="护理等级"
29            clearable
30            class="level-filter"
31          >
32            <el-option
33              v-for="level in [1, 2, 3, 4, 5]"
34              :key="level"
35              :label="'${level}级'"
36              :value="level"
37            />

```

图 B.12 HealthRecords.vue 部分代码图

CarePlan.vue 部分代码图如图 B.13 所示。

```

1  <template>
2  <div class="care-plan">
3    <!-- 客户列表视图 (仅管理员和员工可见) -->
4    <div v-if="isStaff && !selectedClient" class="client-list-view">
5      <div class="filter-section">
6        <el-input
7          v-model="clientFilter"
8          placeholder="搜索客户ID/姓名"
9          clearable
10         class="filter-item"
11       />
12     </div>
13
14     <el-table :data="filteredClients" style="width: 100%" v-loading="loading">
15       <el-table-column label="客户ID" prop="client_id" width="100" />
16       <el-table-column label="姓名">
17         <template #default="{ row }">
18           {{ row.last_name }}{{ row.first_name }}
19         </template>
20       </el-table-column>
21       <el-table-column label="操作" width="200" align="center">
22         <template #default="{ row }">
23           <div class="button-group">
24             <el-button type="primary" size="small" @click="viewClientPlans(row)">
25               查看计划
26             </el-button>
27           </div>
28         </template>
29       </el-table-column>
30     </el-table>
31   </div>
32
33   <!-- 计划列表视图 -->
34   <div v-else class="plan-list-view">
35     <div class="header-section">
36       <div class="back-button" v-if="isStaff">
37         <el-button @click="backToClientList">

```

图 B.13 CarePlan.vue 部分代码图

DietManagement.vue 部分代码图如图 B.14 所示。

```

1  <template>
2  <div class="diet-management">
3    <!-- 管理员/员工视图: 客户列表 -->
4    <div v-if="isStaff" class="client-list-view" v-show="!selectedClient">
5      <div class="panel">
6        <div class="panel-header">
7          <h2>客户列表</h2>
8          <div class="search-bar">
9            <el-input
10              v-model="searchQuery"
11              placeholder="搜索客户ID或姓名"
12              clearable
13              prefix-icon="Search"
14              style="width: 300px"
15            />
16          </div>
17        </div>
18
19        <el-table
20          :data="filteredClients"
21          style="width: 100%"
22          v-loading="loading"
23          border
24          stripe
25          highlight-current-row
26          class="custom-table"
27        >
28          <el-table-column label="客户ID" prop="client_id" width="100" />
29          <el-table-column label="姓名">
30            <template #default="{ row }">
31              {{ row.last_name + row.first_name }}
32            </template>
33          </el-table-column>
34          <el-table-column label="操作" width="150" align="center">
35            <template #default="{ row }">
36              <el-button type="primary" size="small" @click="viewClientDietPlans(row)">
37                查看膳食计划

```

图 B.14 DietManagement.vue 部分代码图

RecipeManagement.vue 部分代码图如图 B.15 所示。

```
1 <template>
2 <div class="recipe-management">
3   <!-- 搜索和筛选区域 -->
4   <div class="filter-section">
5     <div class="filters-container">
6       <el-input
7         v-model="searchQuery"
8         placeholder="搜索菜品名称"
9         clearable
10        class="filter-item"
11      />
12       <el-select
13         v-model="includeIngredients"
14         multiple
15         collapse-tags
16         placeholder="包含食材"
17         clearable
18         class="filter-item"
19       >
20         <el-option
21           v-for="item in ingredients"
22           :key="item.ingredient_id"
23           :label="item.ingredient_name"
24           :value="item.ingredient_name"
25         />
26       </el-select>
27       <el-select
28         v-model="excludeIngredients"
29         multiple
30         collapse-tags
31         placeholder="不包含食材"
32         clearable
33         class="filter-item"
34       >
35         <el-option
36           v-for="item in ingredients"
37           :key="item.ingredient_id"
```

图 B.15 RecipeManagement.vue 部分代码图

IngredientManagement.vue 部分代码图如图 B.16 所示。

```
1 <template>
2 <div class="ingredient-management">
3   <!-- 搜索栏 -->
4   <div class="filter-section">
5     <el-input
6       v-model="searchQuery"
7       placeholder="搜索食材名称"
8       clearable
9       class="filter-item"
10    />
11    <el-button type="primary" class="add-button" @click="createIngredient">
12      新增食材
13    </el-button>
14  </div>
15
16  <!-- 食材列表 -->
17  <div class="ingredient-list">
18    <el-table :data="filteredIngredients" style="width: 100%" v-loading="loading">
19      <el-table-column label="食材名称" prop="ingredient_name" />
20      <el-table-column label="操作" width="250" align="center">
21        <template #default="{ row }">
22          <div class="button-group">
23            <el-button type="primary" size="small" @click="viewDetail(row)">
24              详情
25            </el-button>
26            <el-button type="primary" size="small" @click="editIngredient(row)">
27              编辑
28            </el-button>
29            <el-button type="danger" size="small" @click="deleteIngredient(row)">
30              删除
31            </el-button>
32          </div>
33        </template>
34      </el-table-column>
35    </el-table>
36  </div>
```

图 B.16 IngredientManagement.vue 部分代码图

StaffSchedule.vue 部分代码图如图 B.17 所示。

```

1 <template>
2 <div class="staff-schedule">
3   <!-- 筛选区域 -->
4   <div class="filter-container">
5     <div class="filter-section">
6       <div class="section-title">排班筛选</div>
7       <div class="filter-content">
8         <template v-if="isAdmin">
9           <el-input
10            v-model="staffIdFilter"
11            placeholder="员工ID"
12            class="staff-filter"
13            clearable
14          />
15           <el-input
16            v-model="staffNameFilter"
17            placeholder="员工姓名"
18            class="staff-filter"
19            clearable
20          />
21         </template>
22         <el-date-picker
23          v-model="dateRange"
24          type="daterange"
25          range-separator="至"
26          start-placeholder="开始日期"
27          end-placeholder="结束日期"
28          format="YYYY-MM-DD"
29          value-format="YYYY-MM-DD"
30          class="date-range-picker"
31        />
32       </div>
33     </div>
34   </div>
35   <!-- 操作区域 (仅管理员可见) -->
36   <div v-if="isAdmin" class="operation-container">

```

图 B.17 StaffSchedule.vue 部分代码图

ShiftTemplates.vue 部分代码图如图 B.18 所示。

```

1 <template>
2 <div class="shift-templates-container">
3   <!-- 页面标题 -->
4   <div class="page-header">
5     <h1>排班模板管理</h1>
6     <div class="page-description">管理排班模板，用于员工排班</div>
7   </div>
8   <!-- 模板列表和操作区域 -->
9   <div class="templates-section">
10     <div class="section-header">
11       <h2>模板列表</h2>
12       <el-button type="primary" @click="openCreateDialog">
13         <el-icon>Plus /></el-button>
14         新建模板
15       </el-button>
16     </div>
17     <div class="templates-list">
18       <el-table :data="templates" style="width: 100%" v-loading="loading">
19         <el-table-column prop="template_id" label="ID" width="80" />
20         <el-table-column prop="shift_name" label="班次名称" width="180" />
21         <el-table-column label="时间段" min-width="200">
22           <template #default="{ row }">
23             {{ row.start_time }} - {{ row.end_time }}
24           </template>
25         </el-table-column>
26         <el-table-column label="颜色标识" width="120">
27           <template #default="{ row }">
28             <div class="color-preview" :style="{ backgroundColor: row.color_code }"></div>
29             <span class="color-code">{{ row.color_code }}</span>
30           </template>
31         </el-table-column>
32         <el-table-column label="操作" width="200" fixed="right">
33           <template #default="{ row }">
34             <el-button type="primary" size="small" @click="openEditDialog(row)">
35               <el-icon>Edit /></el-button>
36           </template>
37         </el-table-column>

```

图 B.18 ShiftTemplates.vue 部分代码图

ServiceManagement.vue 部分代码图如图 B.19 所示。

```

1 <template>
2 <div class="service-management">
3   <!-- 筛选区域 -->
4   <div class="filter-container">
5     <div class="filter-section">
6       <div class="section-title">服务筛选</div>
7       <div class="filter-content">
8         <el-input
9           v-model="searchQuery"
10          placeholder="搜索服务名称"
11          class="search-input"
12          clearable
13        />
14      </div>
15    </div>
16  </div>
17  <!-- 操作区域 -->
18  <div class="operation-container" v-if="hasPermission">
19    <el-button type="primary" @click="showAddDialog">
20      添加服务
21    </el-button>
22    <el-icon>Plus </el-icon>
23  </div>
24  </div>
25  <!-- 服务列表 -->
26  <div class="service-table">
27    <el-table :data="filteredServices" border style="width: 100%">
28      <el-table-column prop="service_name" label="服务名称" />
29      <el-table-column label="时长">
30        <template #default="scope">
31          {{ scope.row.duration }} 分钟
32        </template>
33      </el-table-column>
34      <el-table-column label="操作" width="200" v-if="hasPermission">
35        <template #default="scope">
36          <div class="button-group">

```

图 B.19 ServiceManagement.vue 部分代码图

AppointmentManagement.vue 部分代码图如图 B.20 所示。

```

1 <template>
2 <div class="appointment-management">
3   <!-- 筛选区域 -->
4   <div class="filter-section">
5     <el-input
6       v-model="clientNameFilter"
7       placeholder="客户姓名"
8       clearable
9       class="filter-item"
10      v-if="isStaff"
11    />
12    <el-input
13      v-model="staffNameFilter"
14      placeholder="员工姓名"
15      clearable
16      class="filter-item"
17    />
18    <el-date-picker
19      v-model="dateRangeFilter"
20      type="daterange"
21      range-separator="至"
22      start-placeholder="开始日期"
23      end-placeholder="结束日期"
24      format="YYYY-MM-DD"
25      value-format="YYYY-MM-DD"
26      clearable
27      class="filter-item date-range-filter"
28    />
29    <el-select
30      v-model="stateFilter"
31      placeholder="预约状态"
32      clearable
33      class="filter-item"
34    >
35      <el-option
36        v-for="state in ['待确认', '已预约', '已结束', '已取消']"
37        :key="state"

```

图 B.20 AppointmentManagement.vue 部分代码图

附录 C：代谢综合征预测代码

代谢综合征预测代码框架如图 C.1 所示。

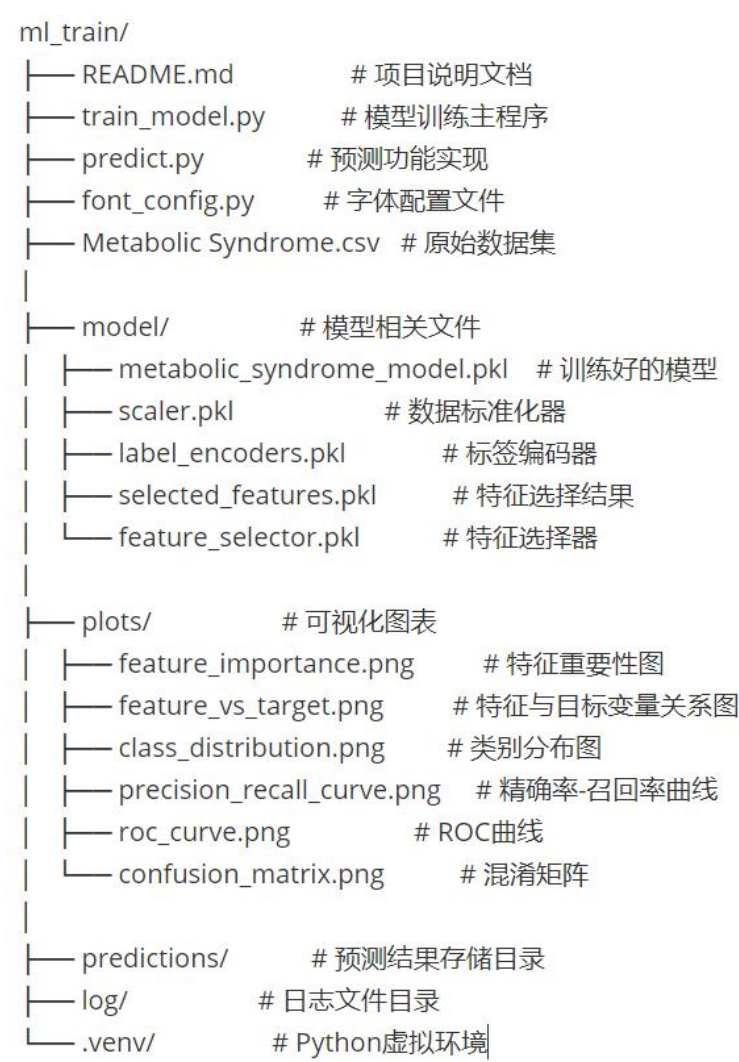


图 C.1 代谢综合征预测代码框架

整体代码包含模型训练代码（train_model.py）、模型预测代码（predict.py）和用于协助 matplotlib 库进行包含中文绘图的工具函数代码（font_config.py）。本章将仅展示核心代码 train_model.py。

train_model.py 代码见图 C.2~图 C.13。

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score, StratifiedKFold
4 from sklearn.preprocessing import LabelEncoder, StandardScaler
5 from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, VotingClassifier, AdaBoostClassifier, ExtraTreesClassifier
6 from sklearn.linear_model import LogisticRegression
7 from sklearn.svm import SVC
8 from sklearn.neighbors import KNeighborsClassifier
9 from sklearn.tree import DecisionTreeClassifier
10 from sklearn.naive_bayes import GaussianNB
11 from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, roc_curve, auc, precision_recall_curve
12 from sklearn.feature_selection import SelectFromModel
13 import pickle
14 import warnings
15 import matplotlib.pyplot as plt
16 import seaborn as sns
17 import os
18 import matplotlib
19 from matplotlib.font_manager import FontProperties
20 import sys
21 import datetime
22 import logging
23
24 # 配置日志
25 current_time = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
26 log_file = f'./log/log_{current_time}.txt'
27 logging.basicConfig(
28     level=logging.INFO,
29     format='%(asctime)s - %(levelname)s - %(message)s',
30     handlers=[
31         logging.FileHandler(log_file, encoding='utf-8'),
32         logging.StreamHandler(sys.stdout)
33     ]
34 )

```

图 C.2 train_model.py 代码其 1

```

36 # 导入字体配置工具
37 import font_config
38
39 # 配置中文字体
40 chinese_font = font_config.configure_chinese_font()
41
42 warnings.filterwarnings('ignore')
43
44 # 创建保存模型和图表的目录
45 if not os.path.exists('./model'):
46     os.makedirs('./model')
47 if not os.path.exists('./plots'):
48     os.makedirs('./plots')
49
50 # 读取数据集
51 logging.info("读取数据集...")
52 df = pd.read_csv('Metabolic Syndrome.csv')
53
54 # 数据预处理
55 logging.info("数据预处理...")
56 # 查看数据基本信息
57 logging.info(f"数据集形状: {df.shape}")
58 logging.info("数据集前5行:")
59 logging.info(df.head())
60
61 # 处理缺失值
62 logging.info("\n处理缺失值...")
63 logging.info(f"缺失值统计: \n{df.isnull().sum()}")
64 # 用中位数填充数值型特征的缺失值
65 for col in df.select_dtypes(include=['float64', 'int64']).columns:
66     if df[col].isnull().sum() > 0:
67         df[col] = df[col].fillna(df[col].median())
68

```

图 C.3 train_model.py 代码其 2

```

69 # Income分类处理
70 logging.info("\n处理Income字段...")
71 # 先处理Income列的缺失值
72 df['Income'] = pd.to_numeric(df['Income'], errors='coerce')
73 df['Income'] = df['Income'].fillna(df['Income'].median())
74
75 # 根据要求将Income分为三类
76 def categorize_income(income):
77     if income < 3000:
78         return 'Low'
79     elif 3000 <= income <= 6000:
80         return 'Mid'
81     else:
82         return 'High'
83
84 df['IncomeCategory'] = df['Income'].apply(categorize_income)
85 logging.info(f"Income分类统计:\n{df['IncomeCategory'].value_counts()}")
86
87 # 对分类特征进行编码
88 logging.info("\n对分类特征进行编码...")
89 categorical_cols = ['Sex', 'Marital', 'IncomeCategory', 'Race']
90 label_encoders = {}
91
92 for col in categorical_cols:
93     le = LabelEncoder()
94     df[col + '_Encoded'] = le.fit_transform(df[col])
95     label_encoders[col] = le
96

```

图 C.4 train_model.py 其 3

```

97 # 特征工程 - 创建新特征
98 logging.info("\n特征工程 - 创建新特征...")
99 # 创建BMI和腰围的比率特征
100 df['WaistBMIRatio'] = df['WaistCirc'] / df['BMI']
101 # 创建血糖与HDL的比率
102 df['GlucoseHDLRatio'] = df['BloodGlucose'] / df['HDL']
103 # 创建甘油三酯与HDL的比率
104 df['TrigHDLRatio'] = df['Triglycerides'] / df['HDL']
105 # 创建年龄分组
106 df['AgeGroup'] = pd.cut(df['Age'], bins=[0, 30, 45, 60, 100], labels=[0, 1, 2, 3])
107
108 # 特征选择
109 logging.info("\n特征选择...")
110 # 使用所有特征包括新创建的特征
111 features = ['Age', 'Sex_Encoded', 'Marital_Encoded', 'IncomeCategory_Encoded',
112            'Race_Encoded', 'WaistCirc', 'BMI', 'Albuminuria', 'UrAlbCr',
113            'UricAcid', 'BloodGlucose', 'HDL', 'Triglycerides',
114            'WaistBMIRatio', 'GlucoseHDLRatio', 'TrigHDLRatio', 'AgeGroup']
115 X = df[features]
116 y = df['MetabolicSyndrome']
117
118 logging.info(f"特征列表: {features}")
119
120 # 数据标准化
121 logging.info("\n数据标准化...")
122 scaler = StandardScaler()
123 X_scaled = scaler.fit_transform(X)
124
125 # 特征选择 - 使用随机森林特征重要性
126 logging.info("\n使用随机森林进行特征选择...")
127 selector = RandomForestClassifier(n_estimators=100, random_state=42)
128 selector.fit(X_scaled, y)
129

```

图 C.5 train_model.py 其 4

```

130 # 特征重要性可视化
131 feature_importance = selector.feature_importances_
132 indices = np.argsort(feature_importance)[::-1]
133 plt.figure(figsize=(10, 6))
134 plt.title('特征重要性')
135 plt.bar(range(X.shape[1]), feature_importance[indices], align='center')
136 plt.xticks(range(X.shape[1]), [features[i] for i in indices], rotation=90)
137 plt.tight_layout()
138 plt.savefig('./plots/feature_importance.png')
139
140 # 选择重要特征
141 logging.info("\n选择重要特征...")
142 sfm = SelectFromModel(selector, threshold=0.03)
143 sfm.fit(X_scaled, y)
144 X_selected = sfm.transform(X_scaled)
145 selected_features_indices = sfm.get_support(indices=True)
146 selected_features = [features[i] for i in selected_features_indices]
147 logging.info(f"选择的重要特征: {selected_features}")
148
149 # 分割训练集和测试集
150 logging.info("\n分割数据集...")
151 X_train, X_test, y_train, y_test = train_test_split(X_selected, y, test_size=0.2, random_state=42, stratify=y)
152 logging.info(f"训练集大小: {X_train.shape}, 测试集大小: {X_test.shape}")
153
154 # 交叉验证评估不同模型
155 logging.info("\n交叉验证评估不同模型...")
156 models = {
157     "随机森林": RandomForestClassifier(random_state=42),
158     "梯度提升": GradientBoostingClassifier(random_state=42),
159     "逻辑回归": LogisticRegression(random_state=42, max_iter=1000),
160     "支持向量机": SVC(probability=True, random_state=42),
161     "K近邻": KNeighborsClassifier(),
162     "决策树": DecisionTreeClassifier(random_state=42),
163     "朴素贝叶斯": GaussianNB(),
164     "AdaBoost": AdaBoostClassifier(random_state=42),
165     "ExtraTrees": ExtraTreesClassifier(random_state=42)
166 }

```

图 C.6 train_model.py 其 5

```

168 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
169 cv_results = {}
170
171 for name, model in models.items():
172     cv_scores = cross_val_score(model, X_train, y_train, cv=cv, scoring='accuracy')
173     cv_results[name] = (cv_scores.mean(), cv_scores.std())
174     logging.info(f"{name} 交叉验证准确率: {cv_scores.mean():.4f} ± {cv_scores.std():.4f}")
175
176 # 选择最佳单一模型
177 best_model_name = max(cv_results.items(), key=lambda x: x[1][0])[0]
178 logging.info(f"\n最佳单一模型: {best_model_name}")
179
180 # 网格搜索优化最佳模型超参数
181 logging.info("\n网格搜索优化超参数...")
182 if best_model_name == "随机森林":
183     param_grid = {
184         'n_estimators': [100, 200, 300],
185         'max_depth': [None, 10, 20, 30],
186         'min_samples_split': [2, 5, 10],
187         'min_samples_leaf': [1, 2, 4]
188     }
189     best_model = RandomForestClassifier(random_state=42)
190 elif best_model_name == "梯度提升":
191     param_grid = {
192         'n_estimators': [100, 200, 300],
193         'learning_rate': [0.01, 0.1, 0.2],
194         'max_depth': [3, 5, 7],
195         'min_samples_split': [2, 5, 10]
196     }
197     best_model = GradientBoostingClassifier(random_state=42)
198 elif best_model_name == "逻辑回归":
199     param_grid = {
200         'C': [0.01, 0.1, 1, 10, 100],
201         'penalty': ['l1', 'l2'],
202         'solver': ['liblinear', 'saga']
203     }
204     best_model = LogisticRegression(random_state=42, max_iter=1000)

```

图 C.7 train_model.py 其 6


```

205 elif best_model_name == "支持向量机":
206     param_grid = {
207         'C': [0.1, 1, 10],
208         'kernel': ['linear', 'rbf', 'poly'],
209         'gamma': ['scale', 'auto', 0.1, 1]
210     }
211     best_model = SVC(probability=True, random_state=42)
212 elif best_model_name == "K近邻":
213     param_grid = {
214         'n_neighbors': [3, 5, 7, 9],
215         'weights': ['uniform', 'distance'],
216         'p': [1, 2]
217     }
218     best_model = KNeighborsClassifier()
219 elif best_model_name == "决策树":
220     param_grid = {
221         'max_depth': [None, 10, 20, 30],
222         'min_samples_split': [2, 5, 10],
223         'min_samples_leaf': [1, 2, 4]
224     }
225     best_model = DecisionTreeClassifier(random_state=42)
226 elif best_model_name == "AdaBoost":
227     param_grid = {
228         'n_estimators': [50, 100, 200],
229         'learning_rate': [0.01, 0.1, 1.0]
230     }
231     best_model = AdaBoostClassifier(random_state=42)
232 elif best_model_name == "ExtraTrees":
233     param_grid = {
234         'n_estimators': [100, 200, 300],
235         'max_depth': [None, 10, 20, 30],
236         'min_samples_split': [2, 5, 10]
237     }
238     best_model = ExtraTreesClassifier(random_state=42)

```

图 C.8 train_model.py 其 7

```

239 else: # 朴素贝叶斯
240     param_grid = {
241         'var_smoothing': [1e-9, 1e-8, 1e-7]
242     }
243     best_model = GaussianNB()
244
245 grid_search = GridSearchCV(best_model, param_grid, cv=cv, scoring='accuracy')
246 grid_search.fit(X_train, y_train)
247 logging.info(f"最佳参数: {grid_search.best_params_}")
248 logging.info(f"最佳交叉验证得分: {grid_search.best_score_:.4f}")
249
250 # 创建集成模型
251 logging.info("\n创建集成模型...")
252 estimators = []
253 for name, model in models.items():
254     if name == best_model_name:
255         estimators.append((name, grid_search.best_estimator_))
256     else:
257         estimators.append((name, model))
258
259 ensemble_model = VotingClassifier(estimators=estimators, voting='soft')
260 ensemble_model.fit(X_train, y_train)

```

图 C.9 train_model.py 其 8

```

262 # 在测试集上评估单一最佳模型和集成模型
263 logging.info("\n在测试集上评估模型...")
264 best_single_model = grid_search.best_estimator_
265 y_pred_best = best_single_model.predict(X_test)
266 y_pred_ensemble = ensemble_model.predict(X_test)
267
268 logging.info("\n最佳单一模型评估:")
269 acc_best = accuracy_score(y_test, y_pred_best)
270 logging.info(f"准确率: {acc_best:.4f}")
271 logging.info("\n分类报告:")
272 logging.info(classification_report(y_test, y_pred_best))
273 logging.info("\n混淆矩阵:")
274 conf_matrix_best = confusion_matrix(y_test, y_pred_best)
275 logging.info(conf_matrix_best)
276
277 logging.info("\n集成模型评估:")
278 acc_ensemble = accuracy_score(y_test, y_pred_ensemble)
279 logging.info(f"准确率: {acc_ensemble:.4f}")
280 logging.info("\n分类报告:")
281 logging.info(classification_report(y_test, y_pred_ensemble))
282 logging.info("\n混淆矩阵:")
283 conf_matrix_ensemble = confusion_matrix(y_test, y_pred_ensemble)
284 logging.info(conf_matrix_ensemble)
285
286 # 选择最终模型
287 final_model = ensemble_model if acc_ensemble > acc_best else best_single_model
288 final_model_name = "集成模型" if acc_ensemble > acc_best else "最佳单一模型"
289 logging.info(f"\n最终选择模型: {final_model_name}")
290 final_acc = acc_ensemble if acc_ensemble > acc_best else acc_best
291 final_conf_matrix = conf_matrix_ensemble if acc_ensemble > acc_best else conf_matrix_best

```

图 C.10 train_model.py 其 9

```

293 # 保存特征选择器、所选择的特征列表和最终模型
294 logging.info("\n保存模型和特征相关信息...")
295 with open('./model/feature_selector.pkl', 'wb') as f:
296     pickle.dump(sfm, f)
297
298 with open('./model/selected_features.pkl', 'wb') as f:
299     pickle.dump(selected_features, f)
300
301 with open('./model/metabolic_syndrome_model.pkl', 'wb') as f:
302     pickle.dump(final_model, f)
303
304 with open('./model/label_encoders.pkl', 'wb') as f:
305     pickle.dump(label_encoders, f)
306
307 with open('./model/scaler.pkl', 'wb') as f:
308     pickle.dump(scaler, f)
309
310 # 可视化结果
311 logging.info("\n可视化结果...")
312
313 # 1. 混淆矩阵可视化
314 plt.figure(figsize=(8, 6))
315 sns.heatmap(final_conf_matrix, annot=True, fmt='d', cmap='Blues',
316             xticklabels=['无代谢综合征', '有代谢综合征'],
317             yticklabels=['无代谢综合征', '有代谢综合征'])
318 plt.xlabel('预测标签')
319 plt.ylabel('真实标签')
320 plt.title(f'{final_model_name}混淆矩阵 (准确率: {final_acc:.4f})')
321 plt.tight_layout()
322 plt.savefig('./plots/confusion_matrix.png')

```

图 C.11 train_model.py 其 10

```

324 # 2. ROC曲线
325 plt.figure(figsize=(8, 6))
326 y_pred_prob = final_model.predict_proba(X_test)[: , 1]
327 fpr, tpr, _ = roc_curve(y_test, y_pred_prob)
328 roc_auc = auc(fpr, tpr)
329 plt.plot(fpr, tpr, lw=2, label=f'ROC曲线 (AUC = {roc_auc:.4f})')
330 plt.plot([0, 1], [0, 1], 'k--', lw=2)
331 plt.xlim([0.0, 1.0])
332 plt.ylim([0.0, 1.05])
333 plt.xlabel('假阳性率')
334 plt.ylabel('真阳性率')
335 plt.title('ROC曲线')
336 plt.legend(loc="lower right")
337 plt.savefig('./plots/roc_curve.png')
338
339 # 3. 精确率-召回率曲线
340 plt.figure(figsize=(8, 6))
341 precision, recall, _ = precision_recall_curve(y_test, y_pred_prob)
342 plt.plot(recall, precision, lw=2)
343 plt.xlabel('召回率')
344 plt.ylabel('精确率')
345 plt.title('精确率-召回率曲线')
346 plt.savefig('./plots/precision_recall_curve.png')
347
348 # 4. 类别分布可视化
349 plt.figure(figsize=(8, 6))
350 class_counts = df['MetabolicSyndrome'].value_counts()
351 plt.bar(['无代谢综合征 (0)', '有代谢综合征 (1)'], class_counts.values)
352 plt.ylabel('样本数量')
353 plt.title('代谢综合征类别分布')
354 for i, v in enumerate(class_counts.values):
355     plt.text(i, v + 5, str(v), ha='center')
356 plt.savefig('./plots/class_distribution.png')
357

```

图 C.12 train_model.py 其 11

```

358 # 5. 特征与目标变量的关系
359 numeric_features = ['Age', 'WaistCirc', 'BMI', 'BloodGlucose', 'HDL', 'Triglycerides']
360 fig, axes = plt.subplots(2, 3, figsize=(18, 10))
361 axes = axes.flatten()
362
363 for i, feature in enumerate(numeric_features):
364     sns.boxplot(x='MetabolicSyndrome', y=feature, data=df, ax=axes[i])
365     axes[i].set_title(f'{feature} vs MetabolicSyndrome')
366     axes[i].set_xlabel('代谢综合征 (0: 无, 1: 有)')
367
368 plt.tight_layout()
369 plt.savefig('./plots/feature_vs_target.png')
370
371 logging.info("\n模型训练完成! 可视化结果已保存到'plots'文件夹。")
372 logging.info(f"最终模型准确率: {final_acc:.4f}")

```

图 C.13 train_model.py 其 12

致 谢

这是本篇论文的最后一个部分也是我本科生涯的句点。

时光荏苒，许多人和事也渐淡出我的脑海。为免今后再无机会，于此向我过往生活中发生的一切人和事，向我的父母、我的朋友们、我的老师们表示感谢。

在论文完成的过程中，从系统的设计到落地实现，我的指导老师石锐提供了颇多教导，在此特向她表示感谢。

原创性声明

郑重声明：所呈交的论文（设计）《康养中心信息化管理系统的设计与实现》，是本人在导师的指导下，独立进行研究取得的成果。除论文（设计）中已经标注引用的内容外，本论文（设计）不包含其他人或集体已经发表或撰写过的作品成果。对本文的研究做出贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律后果，并承诺因本声明而产生的法律结果由本人承担。

论文（设计）作者签名：_____刘锐

日期：_____2025年5月10日

使用授权书

本论文（设计）作者完全了解学校有关保留、使用论文（设计）的规定，同意学校保留并向国家有关部门或机构送交论文（设计）复印件和电子版，允许论文（设计）被查阅和借阅。本人授权重庆大学将本论文（设计）的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制方式保存和汇编本论文（设计）。

本论文（设计）属于：

保 密 ☐ 在_____年解密后适用本授权书

不保密 ☒

论文（设计）作者签名：_____刘锐 指导教师签名：_____石锐

日期：_____2025年5月10日

日期：_____2025年5月10日